DEVOPS ENGINEERING MASTERY

CLOUD ENGINEERING AMAZON WEB SERVICES (AWS)

AWS Architecture, IAM, VPC, EC2, S3, Load Balancing, Auto Scaling, Databases, Containers, Serverless, Security & Production Cloud Operations

VOLUME 11 • FIRST EDITION

```
nahid@technova:~$ aws sts get-caller-identity
✓ Account 111122223333  •  arn:aws:iam::111122223333:user/nahid
nahid@technova:~$ aws ec2 describe-instances --filters Name=tag:Env,Values=prod
✓ 4 instances running  •  2 AZs  •  IMDSv2 enforced
nahid@technova:~$ aws s3 ls s3://technova-app-assets --recursive
```

IAM • VPC • EC2 • S3 • RDS • ECS • Lambda • MERN

First Edition • Published 9 November 2026 • Dhaka, Bangladesh

Companion Workbook • 24 Labs • 3 Projects • Capstone

AWS Lab Workbook

DEVOPS ENGINEERING MASTERY — Volume 11 (Companion Workbook). First Edition, 9 November 2026. AWS CLI v2, AWS Management Console, Terraform and CloudFormation with a MERN example application. This workbook collects all 24 guided labs, 15 assignments, the 3 major projects and the capstone, so you can work through them beside your AWS account without carrying the whole book. Do the labs in order; each lab states its prerequisites and its cost risk. Check Free Tier eligibility before every lab and delete every resource at the end of the lab — never leave paid resources running. The instructor-style solution guide for the capstone is at the end — attempt the capstone before reading it.

ওয়ার্কবুক • হাতে-কলমে ল্যাব • TechNova প্রেক্ষাপট

AWS Lab Workbook

এই ওয়ার্কবুক DEVOPS ENGINEERING MASTERY Volume 11: Cloud Engineering — Amazon Web Services (AWS)-এর হাতে-কলমে অংশ। বইয়ের প্রতিটি অধ্যায়ে যে ধারণা শিখেছেন, এখানে তা নিজের হাতে AWS-এ বানিয়ে, ভেঙে, ঠিক করে এবং মুছে ফেলে শিখবেন। প্রতিটি ল্যাব একটি ছোট, সম্পূর্ণ কাজ — শুরু থেকে শেষ পর্যন্ত করলে আপনি একটি বাস্তব দক্ষতা অর্জন করবেন।

১. এই ওয়ার্কবুক কীভাবে ব্যবহার করবেন

ল্যাবগুলো ক্রমানুসারে সাজানো হয়েছে — Lab 1 থেকে Lab 24 পর্যন্ত একটার উপর আরেকটা দাঁড়িয়ে আছে। প্রথমবার পড়ার সময় ক্রম মেনে চলুন। প্রতিটি ল্যাবে একই কাঠামো আছে: **উদ্দেশ্য** → **পূর্বশর্ত** → **আর্কিটেকচার** → **ধাপে ধাপে** → **যাচাই** → **ট্রাবলশুটিং** → **পরিষ্কার করা** → **চিন্তার প্রশ্ন**। কোনো ল্যাব শুরু করার আগে তার উদ্দেশ্য ও পূর্বশর্ত পড়ুন; শেষ করার আগে পরিষ্কার করা অংশটি অবশ্যই সম্পূর্ণ করুন।

প্রতিটি ল্যাবে **Console** (ওয়েব কনসোল) ও **AWS CLI** — দুইভাবেই নির্দেশনা দেওয়া আছে। প্রথমবার Console দিয়ে করলে কী হচ্ছে তা চোখে দেখা যায়; দ্বিতীয়বার একই কাজ CLI দিয়ে করলে তা অটোমেশনের ভিত্তি হয়। সময় বাঁচাতে চাইলে সরাসরি CLI পথটি নিতে পারেন, তবে Console-এর স্ক্রিনশট/নোট নিজের খাতায় রাখুন — চাকরির ইন্টারভিউতে এগুলোই আপনার প্রমাণ।

প্রতিটি ল্যাবের শেষে চিন্তার প্রশ্ন আছে। উত্তর মুখস্থ করবেন না — নিজের ল্যাবের অভিজ্ঞতা থেকে উত্তর লিখুন। প্রজেক্ট ও ক্যাপস্টোনে এই উত্তরগুলোই আপনার ডকুমেন্টেশনের ভিত্তি হবে।

২. ল্যাব নিরাপত্তা নিয়ম — অবশ্যই মানুন

> ⚠️ **সোনার নিয়ম: প্রতিটি ল্যাব শেষে সব রিসোর্স মুছে ফেলুন।** AWS-এ ভুলে চালু রেখে দেওয়া একটি `NAT Gateway`, `ALB`, `RDS` বা `EC2` ইনস্ট্যান্স কয়েক দিনেই বড় বিল তৈরি করতে পারে। ল্যাব শেষ মানেই *Cleanup* শেষ — এটা ল্যাবেরই অংশ, ঐচ্ছিক নয়।

- **Free Tier যাচাই:** প্রতিটি ল্যাব শুরুর আগে `Billing Dashboard → Free Tier` দেখুন — কতটুকু ব্যবহার বাকি আছে। Free Tier সীমা পেরিয়ে গেলে চার্জ শুরু হয়, AWS থেমে যায় না।
- **বিলিং অ্যালার্ট:** Lab 1-এই বিলিং অ্যালার্ট বসাবেন। $5 ও $10 — দুটি থ্রেশহোল্ডে অ্যালার্ট সেট করুন। অ্যালার্ট ছাড়া কখনো ল্যাব করবেন না।
- **কস্ট রিস্ক:** প্রতিটি ল্যাবে *Low* / *Medium* / *High* ঝুঁকি দেওয়া আছে। High ঝুঁকির ল্যাব (NAT Gateway, ALB, RDS, EKS, ECS) করার দিনই শেষ করুন — রাতে চালু রেখে ঘুমাবেন না।
- **দাম কখনো অনুমান করবেন না:** এই বইয়ে কোথাও নির্দিষ্ট AWS মূল্য দেওয়া হয়নি — মূল্য বদলায়। যেকোনো রিসোর্স বানানোর আগে **AWS Pricing Calculator-এ যাচাই করুন**।
- **Root অ্যাকাউন্ট ব্যবহার নয়:** দৈনন্দিন কাজে root ব্যবহার করবেন না। Lab 1-এ MFA চালু করে IAM অ্যাডমিন ব্যবহারকারী বানান — তারপর সব কাজ সেই ব্যবহারকারী দিয়ে।
- **ক্রেডেনশিয়াল কখনো শেয়ার নয়:** Access Key, Secret Key, পাসওয়ার্ড — কোথাও (GitHub, চ্যাট, স্ক্রিনশট, কোড) পেস্ট করবেন না। এই বইয়ে শুধু `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে।
- **ডিসপোজেবল পরিবেশ:** ল্যাবের জন্য আলাদা টেস্ট পরিবেশ ব্যবহার করুন। নিজের আসল প্রোডাকশন অ্যাকাউন্টে পরীক্ষা করবেন না।
- **রিজিয়ন ঠিক রাখুন:** সব ল্যাবে রিজিয়ন `ap-south-1` (Mumbai)। Console-এর উপরে ডানদিকে রিজিয়ন দেখে নিন — ভুল রিজিয়নে বানানো রিসোর্স খুঁজে পাবেন না, কিন্তু বিল আসবে।

৩. প্রয়োজনীয় টুলস

টুল	কেন লাগবে	যাচাই কমান্ড
AWS অ্যাকাউন্ট + MFA	সব ল্যাবের ভিত্তি; MFA ছাড়া root সুরক্ষিত নয়	Console → IAM → MFA status
AWS CLI v2	টার্মিনাল থেকে AWS চালানো, স্ক্রিপ্ট ও অটোমেশন	`aws --version`
টার্মিনাল / PowerShell	CLI কমান্ড চালানো	—
SSH ক্লায়েন্ট + কী পেয়ার	EC2-তে নিরাপদে ঢোকা (Lab 7)	`ssh -V`
Docker	Lab 17–18: ইমেজ বানানো ও চালানো	`docker --version`
Terraform ≥ 1.5	Lab 19: Infrastructure as Code	`terraform version`
Git + GitHub অ্যাকাউন্ট	Lab 20: CI/CD; কোড সংরক্ষণ	`git --version`
Node.js 20 + npm	Lab 14–15, 24: MERN অ্যাপ	`node --version`
kubectl	Lab 18: EKS ক্লাস্টার পরিচালনা	`kubectl version --client`

ইনস্টল যাচাই: টার্মিনালে উপরের কমান্ডগুলো চালিয়ে দেখুন। কোনো টুল না থাকলে তার অফিসিয়াল সাইট থেকে ইনস্টল করুন — তৃতীয় পক্ষের অচেনা ইনস্টলার ব্যবহার করবেন না।

৪. TechNova — কাল্পনিক প্রেক্ষাপট

সব ল্যাবে আমরা **TechNova** নামের একটি কাল্পনিক স্টার্টআপের ইঞ্জিনিয়ার। TechNova একটি MERN (MongoDB / Express / React / Node.js) ওয়েব অ্যাপ চালায়। বাস্তব কাজের মতো করে শেখার জন্য সব ল্যাবে একই নাম, ডোমেইন ও নেটওয়ার্ক ব্যবহার করা হয়েছে:

আইটেম	ল্যাবে ব্যবহৃত মান	মন্তব্য
কোম্পানি	TechNova	কাল্পনিক; আসল কোম্পানি নয়
ডোমেইন	`technova.example`	`.example` — বাস্তবে রেজিস্টার করা যায় না, তাই নিরাপদ
AWS অ্যাকাউন্ট ID	`111122223333`	কাল্পনিক উদাহরণ ID; নিজের আসল ID ব্যবহার করুন
রিজিয়ন	`ap-south-1` (Mumbai)	সব ল্যাবে একই রিজিয়ন
VPC CIDR	`10.0.0.0/16`	Lab 4 থেকে ব্যবহার হবে
পাবলিক সাবনেট	`10.0.1.0/24` (AZ-a), `10.0.2.0/24` (AZ-b)	ALB / NAT থাকে এখানে
প্রাইভেট সাবনেট	`10.0.11.0/24` (AZ-a), `10.0.12.0/24` (AZ-b)	EC2 অ্যাপ / RDS থাকে এখানে
উদাহরণ IP রেঞ্জ	`203.0.113.x`	ডকুমেন্টেশন-উদ্দেশ্যে সংরক্ষিত রেঞ্জ (RFC 5737)
নামকরণ নিয়ম	`technova-<পরিবেশ>-<সেবা>`	যেমন `technova-dev-web`, `technova-prod-db`

> **নামকরণই অর্ধেক DevOps:** প্রতিটি রিসোর্সে `Name` ট্যাগ দিন — `technova-dev-web-01` -এর মতো। ট্যাগ ছাড়া রিসোর্স পরে খুঁজে পাওয়া যায় না, আর বিলে কোন টিম কত খরচ করল তা বোঝা যায় না। প্রতিটি ল্যাবে `Project=TechNova` ও `Env=dev` ট্যাগ ব্যবহার করুন।

৫. ল্যাব শেষে যা জমা দেবেন (নিজের পোর্টফোলিও)

প্রতিটি ল্যাব শেষে তিনটি জিনিস সংরক্ষণ করুন: (১) কী বানালেন তার স্ক্রিনশট বা CLI আউটপুট, (২) কী সমস্যা হয়েছিল ও কীভাবে ঠিক করলেন — দুই লাইন, (৩) Cleanup-এর প্রমাণ (রিসোর্স মুছে ফেলার পর Console-এর খালি তালিকার স্ক্রিনশট)। এই তিনটি মিলেই আপনার প্রমাণ — ক্যাপস্টোনে এগুলো জমা দেবেন।

> **পরামর্শ:** একটি নোটবুক (Notion / Obsidian / সাধারণ Markdown ফাইল) খুলুন — `technova-lab-notes.md`। প্রতিটি ল্যাবের তারিখ, খরচ, সমস্যা ও সমাধান লিখে রাখুন। তিন মাস পরে এই নোটই আপনার সবচেয়ে মূল্যবান সম্পদ হবে।

ল্যাব ১ • অ্যাকাউন্ট নিরাপত্তা • ৪৫ মিনিট • কস্ট রিস্ক: Low

Lab 1 — Secure AWS Account Setup, MFA and Billing Alerts

১. উদ্দেশ্য

- AWS অ্যাকাউন্টের root ব্যবহারকারীকে MFA দিয়ে সুরক্ষিত করতে পারবেন।
- দৈনন্দিন কাজের জন্য IAM অ্যাডমিন ব্যবহারকারী তৈরি করতে পারবেন।
- বিলিং অ্যালার্ট ও বাজেট সেট করে অপ্রত্যাশিত খরচের আগেই সতর্ক হতে পারবেন।
- অ্যাকাউন্টের নিরাপত্তা অবস্থা যাচাই করতে পারবেন।

২. পূর্বশর্ত

একটি নতুন AWS অ্যাকাউন্ট (ইমেইল + ফোন ভেরিফিকেশন সম্পন্ন), একটি অথেনটিকেটর অ্যাপ (Google Authenticator / Authy / Microsoft Authenticator) ফোনে ইনস্টল করা, এবং ইন্টারনেট সংযোগ।

৩. প্রয়োজনীয় AWS সেবা

`IAM`, `AWS Budgets`, `CloudWatch Alarms (Billing)`, `AWS Cost Explorer`।

৪. আর্কিটেকচার

```
[Root User + MFA] ──► [IAM Admin User: technova-admin + MFA]
                          │                          │
                          └─► [Billing Alerts] ──► [Email: admin@technova.example]
                                                   $5 alarm / $10 budget
```

নিয়ম: `Root` শুধু বিলিং ও অ্যাকাউন্ট বন্ধের কাজে; দৈনন্দিন সব কাজ `IAM Admin` দিয়ে।

৫. ধাপে ধাপে

ধাপ ১ — Root-এ MFA চালু করা (Console)

Root দিয়ে সাইন ইন করুন → উপরে ডানদিকে অ্যাকাউন্ট নাম → *Security credentials* → *Multi-factor authentication (MFA)* → *Assign MFA device* → *Authenticator app* নির্বাচন করুন। QR কোড স্ক্যান করে অ্যাপে যোগ করুন, পরপর দুটি কোড দিয়ে যাচাই করুন। সফল হলে MFA status *Assigned* দেখাবে।

ধাপ ২ — Root Access Key আছে কি না যাচাই

একই *Security credentials* পাতায় *Access keys* দেখুন। Root-এর Access Key থাকলে তা **এখনই মুছে ফেলুন** — root-এর Access Key কখনোই ব্যবহার করা উচিত নয়।

ধাপ ৩ — IAM অ্যাডমিন ব্যবহারকারী তৈরি (Console)

`IAM → Users → Create user` → নাম `technova-admin` → *Provide user access to the AWS Management Console* টিক দিন → পাসওয়ার্ড দিন (শক্তিশালী, `CHANGE-ME-Strong-Password-123!`-এর মতো নিজেরটি) → *Next* → *Attach policies directly* → `AdministratorAccess` যোগ করুন → *Create user*। তারপর ব্যবহারকারীর পাতায় গিয়ে MFA-ও চালু করুন (Lab 1-এর ধাপ ১-এর মতো)।

ধাপ ৪ — IAM অ্যাডমিন দিয়ে সাইন ইন যাচাই (CLI)

```
aws --version
aws configure --profile technova-admin
# AWS Access Key ID:     [IAM ব্যবহারকারীর Access Key]
# AWS Secret Access Key: [CHANGE-ME-SECRET-KEY]
# Default region name:   ap-south-1
# Default output format: json
aws sts get-caller-identity --profile technova-admin
```

প্রত্যাশিত আউটপুটে `Account: 111122223333` (নিজের আসল ID) ও `Arn`-এ `user/technova-admin` দেখাবে।

ধাপ ৫ — বিলিং অ্যালার্ট চালু করা

Root দিয়ে: `Billing → Billing preferences → Receive Billing Alerts` টিক দিন → Save। তারপর `CloudWatch → Alarms → Create alarm` → মেট্রিক `Billing → Total Estimated Charge`, Currency USD, Threshold `5` → অ্যালার্মের নাম `technova-billing-5-usd` → SNS/ইমেইল নোটিফিকেশন যোগ করুন। একইভাবে `$10`-এর অ্যালার্ম বানান।

ধাপ ৬ — বাজেট তৈরি (AWS Budgets)

`AWS Budgets → Create budget → Cost budget` → নাম `technova-monthly-10-usd` → Amount `10` USD → Alert threshold 80% ও 100%-এ ইমেইল অ্যালার্ট।

৬. যাচাই

- Root ও `technova-admin` — দুজনেরই MFA *Assigned*।
- `aws sts get-caller-identity` সঠিক অ্যাকাউন্ট দেখায়।
- CloudWatch-এ দুটি বিলিং অ্যালার্ম *OK* অবস্থায় আছে।
- Budgets-এ মাসিক বাজেট দেখা যায়।

৭. ট্রাবলশুটিং

সমস্যা	কারণ ও সমাধান
MFA কোড কাজ করে না	ফোনের সময় ভুল। ফোনের সময় Automatic করুন; অথেনটিকেটর অ্যাপে সময় সিঙ্ক করুন।
`InvalidClientTokenId`	Access Key ভুল বা নিষ্ক্রিয়। IAM থেকে নতুন কী বানান, পুরোনোটি মুছুন।
বিলিং অ্যালার্ম অপশন দেখা যায় না	বিলিং অ্যালার্ট শুধু `us-east-1` রিজিয়নে কাজ করে। CloudWatch কনসোলে রিজিয়ন `us-east-1` করুন।

৮. নিরাপত্তা বিবেচনা

Root পাসওয়ার্ড পাসওয়ার্ড ম্যানেজারে রাখুন, কাগজে লিখে ডেস্কে রাখবেন না। অ্যাডমিন ব্যবহারকারীর পাসওয়ার্ডও আলাদা রাখুন। Access Key হারিয়ে গেলে সঙ্গে সঙ্গে IAM থেকে নিষ্ক্রিয় (Deactivate) করুন, তারপর মুছুন।

৯. পরিষ্কার করা

এই ল্যাবে কোনো বিলযোগ্য রিসোর্স বানানো হয়নি — মুছতে কিছু নেই। তবে যাচাই করুন: Root-এর কোনো Access Key নেই, বিলিং অ্যালার্ট চালু আছে। এই অ্যালার্টগুলো **মুছবেন না** — এগুলো পরের সব ল্যাবের নিরাপত্তা জাল।

১০. চিন্তার প্রশ্ন

1. Root দিয়ে দৈনন্দিন কাজ করলে কী কী ঝুঁকি থাকে?
2. MFA চালু থাকলেও পাসওয়ার্ড চুরি গেলে আক্রমণকারী কী করতে পারবে, কী পারবে না?
3. বিলিং অ্যালার্ট $5-এ সেট করলে কী সুবিধা — $100-এ সেট করলে কী সমস্যা হতো?

ল্যাব ২ • IAM • ৬০ মিনিট • কস্ট রিস্ক: Low

Lab 2 — IAM Role and Least-Privilege Policy

১. উদ্দেশ্য

- Least-privilege নীতিতে IAM পলিসি লিখতে পারবেন — প্রয়োজনের বেশি অনুমতি নয়।
- ব্যবহারকারী, গ্রুপ ও রোলের পার্থক্য বুঝে সঠিকটি ব্যবহার করতে পারবেন।
- EC2-র জন্য IAM রোল বানিয়ে Access Key ছাড়াই S3 ব্যবহার করতে পারবেন।

২. পূর্বশর্ত

Lab 1 সম্পন্ন (অ্যাডমিন ব্যবহারকারী + MFA)। IAM-এর মৌলিক ধারণা: User, Group, Role, Policy।

৩. প্রয়োজনীয় AWS সেবা

`IAM`, `S3` (পরীক্ষার বাকেট), `EC2` (রোল যাচাইয়ে, Lab 7-এ বিস্তারিত)।

৪. আর্কিটেকচার

```
[Developer Group] ──► Policy: S3 Read-only on technova-lab-* only
[EC2 Instance] ──► Role: technova-ec2-s3-role ──► S3 bucket (read/write)
[Admin] ──► AdministratorAccess (শুধু Lab 1-এর অ্যাডমিন)
নীতি: কেউই প্রয়োজনের বেশি কিছু করতে পারবে না।
```

৫. ধাপে ধাপে

ধাপ ১ — Least-Privilege পলিসি তৈরি (Console)

`IAM → Policies → Create policy → JSON` ট্যাবে নিচের JSON বসান — খেয়াল করুন, শুধু `technova-lab-*` বাকেটেই অনুমতি, সব বাকেটে নয়:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:ListBucket"],
      "Resource": "arn:aws:s3:::technova-lab-*"
    },
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::technova-lab-*/**"
    }
```

```
  ]
}
```

নাম দিন `TechnovaLabS3Limited` → Create।

ধাপ ২ — গ্রুপ বানিয়ে পলিসি যুক্ত করা

`IAM → User groups → Create group` → নাম `technova-developers` → পলিসি `TechnovaLabS3Limited` যুক্ত করুন → Create।

ধাপ ৩ — ডেভেলপার ব্যবহারকারী তৈরি

`IAM → Users → Create user` → নাম `technova-dev-01` → Console access দিন → গ্রুপ `technova-developers`-এ যোগ করুন। এই ব্যবহারকারীর কোনো `AdministratorAccess` নেই — এটাই উদ্দেশ্য।

ধাপ ৪ — EC2-র জন্য IAM রোল (CLI)

```
aws iam create-role --role-name technova-ec2-s3-role \
  --assume-role-policy-document file://trust-policy.json
# trust-policy.json-এ Service: ec2.amazonaws.com-কে AssumeRole দিতে হবে
aws iam attach-role-policy --role-name technova-ec2-s3-role \
  --policy-arn arn:aws:iam::111122223333:policy/TechnovaLabS3Limited
aws iam create-instance-profile \
  --instance-profile-name technova-ec2-profile
aws iam add-role-to-instance-profile \
  --instance-profile-name technova-ec2-profile \
  --role-name technova-ec2-s3-role
```

`trust-policy.json`-এর বিষয়বস্তু:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"Service": "ec2.amazonaws.com"},
    "Action": "sts:AssumeRole"
  }]
}
```

ধাপ ৫ — অনুমতি পরীক্ষা (নেগেটিভ টেস্ট)

`technova-dev-01` দিয়ে সাইন ইন করে EC2 বানানোর চেষ্টা করুন — ব্যর্থ হওয়া উচিত (`AccessDenied`)। S3-তে `technova-lab-test` বাকেটের ফাইল দেখা যাওয়া উচিত। নেগেটিভ টেস্ট সফল হওয়া মানেই পলিসি ঠিক আছে।

৬. যাচাই

- `aws iam get-user --user-name technova-dev-01` ব্যবহারকারী দেখায়।
- `aws iam list-attached-user-policies`-এ শুধু সীমিত পলিসি আছে।
- ডেভেলপার EC2 বানাতে পারে না, কিন্তু নির্দিষ্ট S3 বাকেট পড়তে পারে।
- রোলের Trust Policy-তে শুধু `ec2.amazonaws.com` আছে।

৭. ট্রাবলশুটিং

সমস্যা	কারণ ও সমাধান
`AccessDenied` — S3-তেও	বাকেটের নাম পলিসির `technova-lab-*` প্যাটার্নে মেলে কি না দেখুন। ARN-এ টাইপো সবচেয়ে সাধারণ ভুল।
রোল তৈরি হয় কিন্তু EC2-তে কাজ করে না	Instance Profile যুক্ত করেছেন কি না দেখুন। রোল একা যথেষ্ট নয় — Instance Profile লাগে।
পলিসি সেভ হয় না — JSON error	কমা বা ব্র্যাকেট ভুল। IAM Policy Simulator-এ যাচাই করুন।

> **IAM Policy Simulator:** `policysim.aws.amazon.com` -এ গিয়ে ব্যবহারকারী/রোল ও অ্যাকশন বেছে পরীক্ষা করুন — আসল রিসোর্স না বানিয়েই অনুমতি যাচাই করা যায়।

৮. নিরাপত্তা বিবেচনা

`*` (সবকিছু) Resource দিয়ে পলিসি লিখবেন না — সবসময় নির্দিষ্ট ARN দিন। `AdministratorAccess` শুধু ভাঙা-গড়ার কাজে, দৈনন্দিন ডেভেলপার অ্যাকাউন্টে নয়। পলিসি বদলালে `Access Advisor` দেখে নিন — কোন অনুমতি আসলে ব্যবহৃত হয়নি।

৯. পরিষ্কার করা

```
aws iam remove-user-from-group --group-name technova-developers \
  --user-name technova-dev-01
aws iam delete-user --user-name technova-dev-01
aws iam delete-group --group-name technova-developers
aws iam delete-policy \
  --policy-arn arn:aws:iam::111122223333:policy/TechnovaLabS3Limited
aws iam remove-role-from-instance-profile \
  --instance-profile-name technova-ec2-profile \
  --role-name technova-ec2-s3-role
aws iam delete-instance-profile \
  --instance-profile-name technova-ec2-profile
aws iam detach-role-policy --role-name technova-ec2-s3-role \
  --policy-arn arn:aws:iam::111122223333:policy/TechnovaLabS3Limited
aws iam delete-role --role-name technova-ec2-s3-role
```

নিজের শেখার জন্য পলিসি JSON-টি নোটবুকে কপি করে রাখুন, তারপর মুছুন।

১০. চিন্তার প্রশ্ন

1. Access Key দিয়ে EC2 থেকে S3 ব্যবহার করা আর IAM রোল ব্যবহার করার মধ্যে নিরাপত্তার পার্থক্য কী?
2. একজন ডেভেলপারকে `s3:*` দিলে কী কী ক্ষতি হতে পারে?
3. Trust Policy ভুল লিখলে রোল ব্যবহার করা যাবে না কেন?

ল্যাব ৩ • AWS CLI • ৪০ মিনিট • কস্ট রিস্ক: Low

Lab 3 — AWS CLI Profiles and Credential Management

১. উদ্দেশ্য

- একাধিক AWS প্রোফাইল (dev / prod) সেটআপ ও ব্যবহার করতে পারবেন।
- ক্রেডেনশিয়াল ফাইলের গঠন ও নিরাপত্তা বুঝবেন।
- SSO/পরিবেশ ভেরিয়েবল ও প্রোফাইল চেইন ব্যবহার করতে পারবেন।
- Access Key রোটেশন (ঘোরানো) করতে পারবেন।

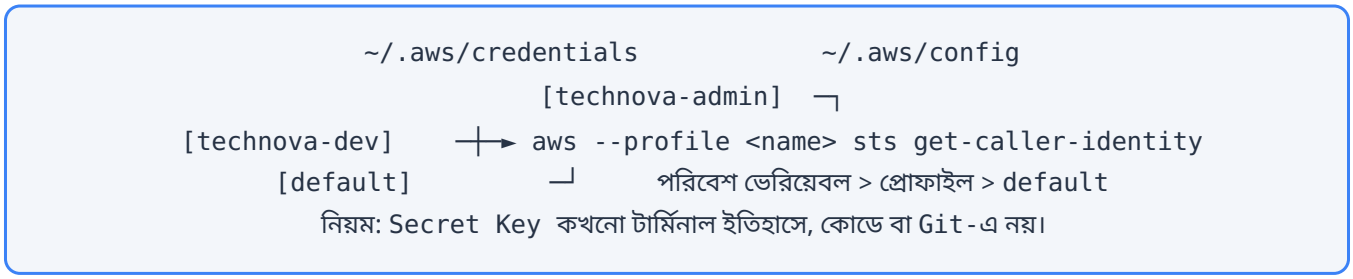
২. পূর্বশর্ত

Lab 1–2 সম্পন্ন। AWS CLI v2 ইনস্টল করা। দুটি IAM ব্যবহারকারী (অ্যাডমিন ও ডেভেলপার) তৈরি থাকা ভালো।

৩. প্রয়োজনীয় AWS সেবা

`IAM`, `STS` (পরিচয় যাচাই), `AWS CLI v2`।

৪. আর্কিটেকচার



৫. ধাপে ধাপে

ধাপ ১ — CLI ইনস্টল যাচাই ও আপডেট

```
aws --version
# প্রত্যাশিত: aws-cli/2.x Python/3.x ...
aws configure list
```

ধাপ ২ — দুটি প্রোফাইল তৈরি

```
aws configure --profile technova-admin
aws configure --profile technova-dev
cat ~/.aws/credentials
cat ~/.aws/config
```

Console নির্দেশনা: প্রোফাইল আসলে দুটি ফাইলে থাকে — `~/.aws/credentials` (গোপন কী) ও `~/.aws/config` (রিজিয়ন, আউটপুট ফরম্যাট)। ফাইল দুটি খুলে দেখুন, কিন্তু Secret Key কাউকে দেখাবেন না।

ধাপ ৩ — প্রোফাইল দিয়ে কাজ করা

```
aws sts get-caller-identity --profile technova-admin
aws sts get-caller-identity --profile technova-dev
aws s3 ls --profile technova-dev
export AWS_PROFILE=technova-dev
aws sts get-caller-identity
unset AWS_PROFILE
```

ধাপ ৪ — পরিবেশ ভেরিয়েবল দিয়ে অস্থায়ী ক্রেডেনশিয়াল

```
export AWS_ACCESS_KEY_ID=CHANGE-ME-ACCESS-KEY
export AWS_SECRET_ACCESS_KEY=CHANGE-ME-SECRET-KEY
export AWS_DEFAULT_REGION=ap-south-1
aws sts get-caller-identity
# কাজ শেষে অবশ্যই মুছুন:
unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY
```

> **সতর্কতা:** `export` করা Secret Key টার্মিনাল ইতিহাসে থেকে যায়। শেয়ার্ড মেশিনে এভাবে কাজ করবেন না। স্থায়ী কাজে প্রোফাইল ব্যবহার করুন।

ধাপ ৫ — Access Key রোটেশন

```
aws iam create-access-key --user-name technova-dev-01
# নতুন কী দিয়ে প্রোফাইল আপডেট করুন, পুরোনো কী দিয়ে কাজ হচ্ছে যাচাই করুন
aws iam list-access-keys --user-name technova-dev-01
aws iam update-access-key --user-name technova-dev-01 \
  --access-key-id CHANGE-ME-OLD-KEY-ID --status Inactive
aws iam delete-access-key --user-name technova-dev-01 \
  --access-key-id CHANGE-ME-OLD-KEY-ID
```

ধাপ ৬ — `.gitignore` সুরক্ষা

যেকোনো প্রজেক্টে `.gitignore`-এ এই লাইনগুলো আছে কি না যাচাই করুন:

```
.env
.aws/
*.pem
credentials
config
```

৬. যাচাই

- দুটি প্রোফাইল দিয়ে `get-caller-identity` আলাদা ARN দেখায়।
- `AWS_PROFILE` সেট করলে প্রোফাইল ছাড়া কমান্ডও সঠিক অ্যাকাউন্টে যায়।
- পুরোনো Access Key নিষ্ক্রিয় করার পর তা দিয়ে কমান্ড ব্যর্থ হয়।

- Git রিপোজিটরিতে কোনো Secret Key নেই (`git log --all -p | grep -i secret` খালি)।

৭. ট্রাবলশুটিং

সমস্যা	কারণ ও সমাধান
`ProfileNotFound`	প্রোফাইলের নাম ভুল টাইপ হয়েছে। `aws configure list-profiles` দিয়ে নাম দেখুন।
ভুল অ্যাকাউন্টে কমান্ড চলছে	`AWS_PROFILE` পরিবেশ ভেরিয়েবল সেট আছে কি না দেখুন — এটি `--profile`-এর চেয়ে অগ্রাধিকার পায় না, কিন্তু default-কে ছাপিয়ে যায়।
`ExpiredToken`	অস্থায়ী (STS) ক্রেডেনশিয়ালের মেয়াদ শেষ। নতুন করে লগইন/AssumeRole করুন।

৮. নিরাপত্তা বিবেচনা

`~/.aws/credentials` ফাইলের অনুমতি `600` রাখুন (`chmod 600 ~/.aws/credentials`)। Access Key ৯০ দিনের বেশি পুরোনো হলে রোটেট করুন। সম্ভব হলে দীর্ঘমেয়াদি Access Key-এর বদলে IAM Identity Center (SSO) ব্যবহার করুন — তাতে কী সংরক্ষণই করতে হয় না।

৯. পরিষ্কার করা

পরীক্ষার জন্য বানানো অতিরিক্ত Access Key মুছে ফেলুন (ধাপ ৫ দেখুন)। `technova-dev` প্রোফাইলটি রেখে দিতে পারেন — পরের ল্যাবে লাগবে। পরিবেশ ভেরিয়েবলে কোনো Secret Key সেট আছে কি না যাচাই করুন: `env | grep AWS_` খালি হওয়া উচিত।

১০. চিন্তার প্রশ্ন

1. একই মেশিনে dev ও prod প্রোফাইল থাকলে ভুল করে prod-এ কমান্ড চালানো কীভাবে ঠেকাবেন?
2. Access Key GitHub-এ ফাঁস হলে প্রথম ৫ মিনিটে কী কী করবেন?
3. SSO কেন দীর্ঘমেয়াদি Access Key-এর চেয়ে নিরাপদ?

ল্যাব ৪ • নেটওয়ার্কিং • ৭৫ মিনিট • কস্ট রিস্ক: Low

Lab 4 — Custom VPC and Subnet Design

১. উদ্দেশ্য

- নিজের CIDR পরিকল্পনা করে কাস্টম VPC বানাতে পারবেন।
- দুটি Availability Zone-এ পাবলিক ও প্রাইভেট সাবনেট ভাগ করতে পারবেন।
- রাউট টেবিলের কাজ বুঝে সাবনেটকে পাবলিক/প্রাইভেট হিসেবে চিহ্নিত করতে পারবেন।
- DNS হোস্টনেম ও DNS সাপোর্ট সেটিংস যাচাই করতে পারবেন।

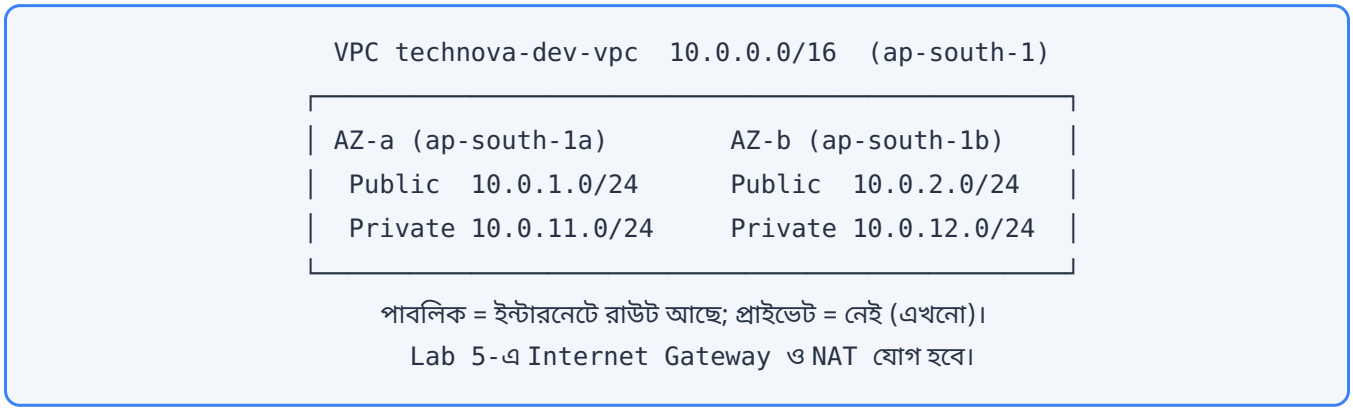
২. পূর্বশর্ত

Lab 1–3 সম্পন্ন। IP অ্যাড্রেসিং ও CIDR-এর মৌলিক ধারণা (যেমন `/16` মানে ৬৫,৫৩৬টি ঠিকানা)। ক্যালকুলেটর হিসেবে `10.0.0.0/16` ভাগ করার পরিকল্পনা হাতে লিখে রাখুন।

৩. প্রয়োজনীয় AWS সেবা

`VPC`, `Subnets`, `Route Tables`, `Availability Zones (ap-south-1a / ap-south-1b)`।

৪. আর্কিটেকচার



পাবলিক = ইন্টারনেটে রাউট আছে; প্রাইভেট = নেই (এখনো)।
Lab 5-এ `Internet Gateway` ও `NAT` যোগ হবে।

সাবনেট	CIDR	AZ	ধরন	ব্যবহার
`technova-dev-public-a`	`10.0.1.0/24`	ap-south-1a	পাবলিক	ALB, NAT Gateway
`technova-dev-public-b`	`10.0.2.0/24`	ap-south-1b	পাবলিক	ALB (দ্বিতীয় AZ)
`technova-dev-private-a`	`10.0.11.0/24`	ap-south-1a	প্রাইভেট	EC2 অ্যাপ, RDS
`technova-dev-private-b`	`10.0.12.0/24`	ap-south-1b	প্রাইভেট	EC2 অ্যাপ, RDS (স্ট্যান্ডবাই)

৫. ধাপে ধাপে

ধাপ ১ — VPC তৈরি (Console)

`VPC → Create VPC → VPC only` → নাম `technova-dev-vpc` → IPv4 CIDR `10.0.0.0/16` → DNS hostnames: Enable, DNS resolution: Enable → Create।

ধাপ ২ — চারটি সাবনেট তৈরি (Console)

`VPC → Subnets → Create subnet` → VPC নির্বাচন করুন → প্রতিটি সাবনেটের জন্য নাম, AZ ও CIDR দিন (উপরের টেবিল)। প্রতিটি সাবনেট তৈরির পর তার পাতায় গিয়ে *Auto-assign public IPv4 address* — পাবলিক সাবনেটে Enable, প্রাইভেটে Disable — যাচাই করুন।

ধাপ ৩ — রাউট টেবিল তৈরি ও যুক্ত করা (Console)

`VPC → Route tables → Create route table` → নাম `technova-dev-public-rt` → VPC নির্বাচন → Create → *Subnet associations* → *Edit* → দুটি পাবলিক সাবনেট যুক্ত করুন। একইভাবে `technova-dev-private-rt` বানিয়ে দুটি প্রাইভেট সাবনেট যুক্ত করুন। এখনো কোনো রাউটে `0.0.0.0/0` নেই — সেটা Lab 5-এ যোগ হবে।

ধাপ ৪ — একই কাজ CLI দিয়ে (দ্বিতীয় VPC-তে অনুশীলন, তারপর মুছে ফেলুন)

```
aws ec2 create-vpc --cidr-block 10.1.0.0/16 \
  --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=technova-cli-test}]'
VPC_ID=CHANGE-ME-VPC-ID
aws ec2 create-subnet --vpc-id $VPC_ID \
  --cidr-block 10.1.1.0/24 --availability-zone ap-south-1a \
  --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=technova-cli-test-a}]'
aws ec2 describe-vpcs --vpc-ids $VPC_ID
aws ec2 describe-subnets --filters Name=vpc-id,Values=$VPC_ID
```

ধাপ ৫ — DNS সেটিংস যাচাই

```
aws ec2 describe-vpc-attribute --vpc-id $VPC_ID \
  --attribute enableDnsHostnames
aws ec2 describe-vpc-attribute --vpc-id $VPC_ID \
  --attribute enableDnsSupport
```

দুটোই `true` হওয়া উচিত — না হলে EC2-র পাবলিক DNS নাম কাজ করবে না।

৬. যাচাই

- VPC-র CIDR `10.0.0.0/16` ও state *Available*।
- চারটি সাবনেট সঠিক AZ ও CIDR-এ আছে।
- পাবলিক রাউট টেবিলে দুটি পাবলিক সাবনেট, প্রাইভেটে দুটি প্রাইভেট সাবনেট যুক্ত।
- সাবনেটগুলোর CIDR পরস্পর ওভারল্যাপ করে না।

৭. ট্রাবলশুটিং

সমস্যা	কারণ ও সমাধান
`InvalidSubnet.Range`	CIDR VPC-র রেঞ্জের বাইরে বা অন্য সাবনেটের সঙ্গে ওভারল্যাপ। CIDR পরিকল্পনা আবার লিখুন।
সাবনেট তৈরি হয় কিন্তু EC2-তে IP পায় না	সাবনেটে যথেষ্ট ফ্রি IP নেই বা Auto-assign সেটিং ভুল। সাবনেটের Available IPs দেখুন।
রাউট টেবিল যুক্ত হচ্ছে না	একটি সাবনেট একসময়ে একটি রাউট টেবিলেই থাকতে পারে। আগের অ্যাসোসিয়েশন সরান।

৮. নিরাপত্তা বিবেচনা

ডাটাবেস ও অ্যাপ সার্ভার সবসময় প্রাইভেট সাবনেটে রাখুন — পাবলিক সাবনেটে শুধু লোড ব্যালান্সার ও NAT। ভবিষ্যতে VPC Flow Logs চালু করে নেটওয়ার্ক ট্র্যাফিক নিরীক্ষণ করুন (Lab 21)।

৯. পরিষ্কার করা

এই ল্যাবের VPC মুছবেন না — Lab 5–10, 13, 24-এ একই VPC ব্যবহার হবে। শুধু ধাপ ৪-এর CLI টেস্ট VPC (`10.1.0.0/16`) মুছুন:

```
aws ec2 delete-subnet --subnet-id CHANGE-ME-SUBNET-ID
aws ec2 delete-vpc --vpc-id $VPC_ID
```

মূল VPC রেখে দেওয়ার কারণ নোটবুকে লিখুন। ক্যাপস্টোনে এই VPC-ই প্রোডাকশন ভিত্তি হবে।

১০. চিন্তার প্রশ্ন

1. `10.0.0.0/16`-কে `/24` সাবনেটে ভাগ করলে কয়টি সাবনেট পাওয়া যায়? ভবিষ্যতের বৃদ্ধির জন্য কতটুকু ফাঁকা রাখা উচিত?
2. একটি সাবনেটকে "পাবলিক" বলা হয় কেন — রাউট টেবিলের কোন লাইনটি তাকে পাবলিক করে?
3. দুটি AZ ব্যবহার করলে একটি AZ ডাউন হলেও অ্যাপ কেন চলবে?

ল্যাব ৫ • নেটওয়ার্কিং • ৬০ মিনিট • কস্ট রিস্ক: Medium — NAT Gateway চার্জ হয়, ল্যাব শেষেই মুছুন

Lab 5 — Internet Gateway, NAT and VPC Endpoints

১. উদ্দেশ্য

- Internet Gateway (IGW) দিয়ে পাবলিক সাবনেটকে ইন্টারনেটে সংযুক্ত করতে পারবেন।
- NAT Gateway দিয়ে প্রাইভেট সাবনেট থেকে আউটবাউন্ড ইন্টারনেট (আপডেট, প্যাকেজ) চালু করতে পারবেন।
- VPC Endpoint (Gateway) দিয়ে S3-তে প্রাইভেটভাবে পৌঁছাতে পারবেন — ইন্টারনেট ছাড়াই।
- প্রতিটি পথের খরচ ও নিরাপত্তা পার্থক্য ব্যাখ্যা করতে পারবেন।

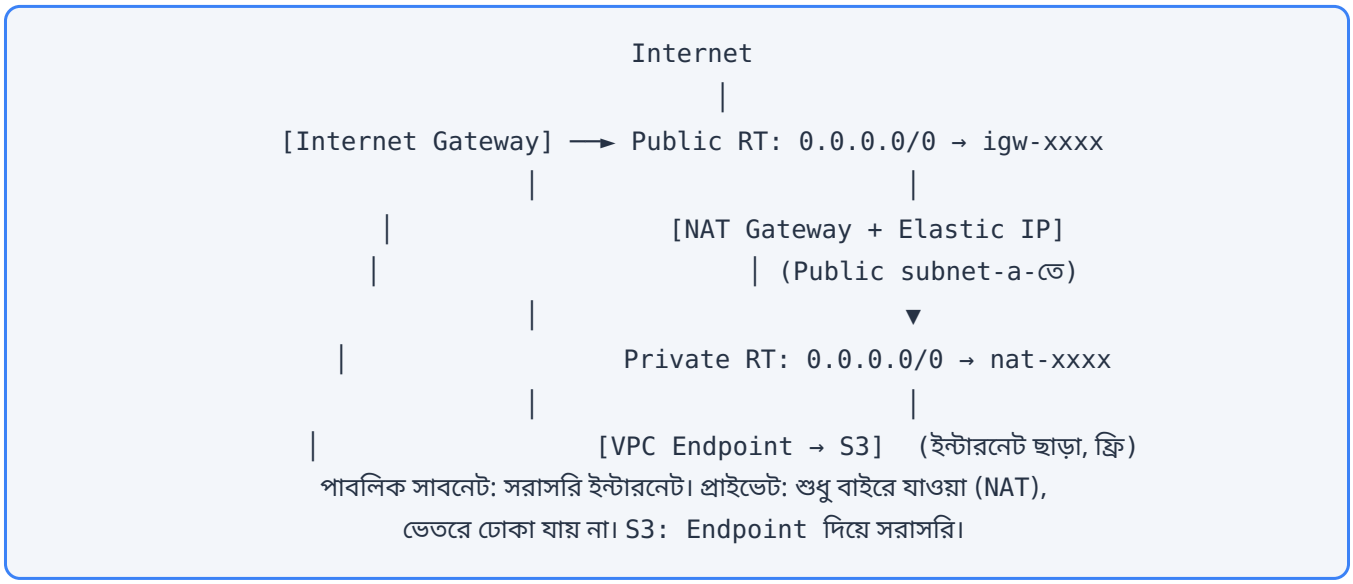
২. পূর্বশর্ত

Lab 4-এর VPC ও চারটি সাবনেট তৈরি আছে। Elastic IP সম্পর্কে ধারণা।

৩. প্রয়োজনীয় AWS সেবা

`VPC Internet Gateway`, `NAT Gateway`, `Elastic IP`, `VPC Endpoints (S3 Gateway)`।

৪. আর্কিটেকচার



৫. ধাপে ধাপে

ধাপ ১ — Internet Gateway তৈরি ও যুক্ত করা (Console)

`VPC → Internet gateways → Create internet gateway` → নাম `technova-dev-igw` → Create → *Attach to VPC* → `technova-dev-vpc` নির্বাচন করুন।

ধাপ ২ — পাবলিক রাউট টেবিলে রাউট যোগ

`Route tables → technova-dev-public-rt → Routes → Edit routes → Add route` → Destination `0.0.0.0/0`, Target: Internet Gateway `technova-dev-igw` → Save। এখন পাবলিক সাবনেট সত্যিই পাবলিক।

ধাপ ৩ — NAT Gateway তৈরি (Console)

`VPC → NAT gateways → Create NAT gateway` → Subnet: `technova-dev-public-a` → Connectivity: Public → *Allocate Elastic IP* → নাম `technova-dev-nat-a` → Create। তৈরি হতে ২–৫ মিনিট লাগে — state *Available* হওয়া পর্যন্ত অপেক্ষা করুন।

> **খরচ সতর্কতা:** NAT Gateway প্রতি ঘণ্টায় চার্জ হয়, আর ডেটা প্রসেসিংয়ের জন্যও চার্জ হয়। এই ল্যাবে একটি NAT-ই যথেষ্ট (শুধু AZ-a-তে)। প্রোডাকশনে প্রতি AZ-তে একটি লাগে, কিন্তু ল্যাবে একটিই রাখুন — আর ল্যাব শেষে মুছে ফেলুন। দাম AWS Pricing Calculator-এ যাচাই করুন।

ধাপ ৪ — প্রাইভেট রাউট টেবিলে NAT রাউট

`Route tables → technova-dev-private-rt → Edit routes → Add route` → Destination `0.0.0.0/0`, Target: NAT Gateway `technova-dev-nat-a` → Save।

ধাপ ৫ — S3 Gateway VPC Endpoint (CLI)

```
aws ec2 create-vpc-endpoint --vpc-id CHANGE-ME-VPC-ID \
  --service-name com.amazonaws.ap-south-1.s3 \
  --route-table-ids CHANGE-ME-PRIVATE-RT-ID \
  --vpc-endpoint-type Gateway \
  --tag-specifications 'ResourceType=vpc-endpoint,Tags=[{Key=Name,Value=technova-dev-s3-
endpoint}]'
aws ec2 describe-vpc-endpoints --filters Name=vpc-id,Values=CHANGE-ME-VPC-ID
```

ধাপ ৬ — সংযোগ পরীক্ষা (EC2 ছাড়া যাচাই)

রাউট টেবিল দুটি খুলে দেখুন: পাবলিক RT-তে `0.0.0.0/0 → igw`, প্রাইভেট RT-তে `0.0.0.0/0 → nat` এবং S3-র জন্য Prefix List রাউট (`pl-xxxx → vpce-xxxx`) আছে কি না। পূর্ণ পরীক্ষা Lab 7-এ EC2 দিয়ে হবে — প্রাইভেট EC2 থেকে `curl` করে ইন্টারনেট যাবে, কিন্তু বাইরে থেকে SSH যাবে না।

৬. যাচাই

- IGW state *Attached*, VPC সঠিক।
- পাবলিক RT-তে IGW রাউট, প্রাইভেট RT-তে NAT রাউট আছে।
- NAT Gateway state *Available*, Elastic IP যুক্ত।
- S3 Endpoint state *Available* ও প্রাইভেট RT-তে তার রাউট আছে।

৭. ট্রাবলশুটিং

সমস্যা	কারণ ও সমাধান

NAT Gateway তৈরি হচ্ছে না	Elastic IP লিমিট শেষ (ডিফল্ট ৫টি)। অব্যবহৃত EIP ছেড়ে দিন বা লিমিট বাড়ানোর অনুরোধ করুন।
প্রাইভেট সাবনেট থেকে ইন্টারনেট যায় না	প্রাইভেট RT-তে NAT রাউট আছে কি না, NAT state Available কি না, এবং Security Group-এ আউটবাউন্ড অনুমতি আছে কি না — তিনটি দেখুন।
S3 Endpoint-এর পরও S3 ধীর/ব্যর্থ	Endpoint-এর রাউট টেবিল অ্যাসোসিয়েশন দেখুন; বাকেট পলিসি VPC Endpoint-কে ব্লক করছে কি না দেখুন।

৮. নিরাপত্তা বিবেচনা

NAT Gateway শুধু আউটবাউন্ড — বাইরে থেকে কেউ প্রাইভেট সাবনেটে ঢুকতে পারে না। এটাই এর নিরাপত্তা। S3 Gateway Endpoint ব্যবহার করলে S3 ট্র্যাফিক পাবলিক ইন্টারনেটে যায় না — সংবেদনশীল ডেটার জন্য এটা বড় সুবিধা। Interface Endpoint (যেমন SSM, ECR)-এর ক্ষেত্রে Security Group দিয়ে Endpoint-কে সুরক্ষিত করুন।

৯. পরিষ্কার করা

এই ল্যাবে NAT Gateway মুছে ফেলুন (খরচ বাঁচাতে) — তবে IGW ও S3 Endpoint রেখে দিন, Lab 7–10-এ লাগবে। NAT মুছতে:

```
aws ec2 delete-nat-gateway --nat-gateway-id CHANGE-ME-NAT-ID
# NAT মুছে গেলে Elastic IP ছেড়ে দিন:
aws ec2 release-address --allocation-id CHANGE-ME-EIP-ALLOC-ID
# প্রাইভেট RT থেকে NAT রাউটটি সরান (Console → Edit routes → Delete)
```

পরের ল্যাবে NAT দরকার হলে আবার বানাবেন — প্রতিবার বানানো-মোছা অনুশীলনই শেখার অংশ।

১০. চিন্তার প্রশ্ন

1. প্রাইভেট সাবনেটের সার্ভারকে প্যাকেজ আপডেট করতে হলে NAT কেন লাগে, IGW দিয়ে কেন হবে না?
2. NAT Gateway-এর বদলে NAT Instance ব্যবহার করলে কী সুবিধা-অসুবিধা?
3. S3 Endpoint থাকলে প্রাইভেট সাবনেট থেকে S3-তে ডেটা পাঠাতে NAT-এর ডেটা চার্জ কেন বাঁচে?

ল্যাব ৬ • নেটওয়ার্ক নিরাপত্তা • ৬০ মিনিট • কস্ট রিস্ক: Low

Lab 6 — Security Groups and NACLs

১. উদ্দেশ্য

- Security Group (stateful) ও NACL (stateless)-এর পার্থক্য হাতে-কলমে বুঝবেন।
- ওয়েব, অ্যাপ ও ডাটাবেস — তিন স্তরের জন্য আলাদা Security Group বানাতে পারবেন।
- সবচেয়ে কম অনুমতির নিয়ম লিখে পরীক্ষা করতে পারবেন।
- ভুল নিয়মের কারণে সংযোগ ব্যর্থ হলে নির্ণয় করতে পারবেন।

২. পূর্বশর্ত

Lab 4–5-এর VPC। TCP পোর্টের ধারণা (22 SSH, 80 HTTP, 443 HTTPS, 3306 MySQL, 5432 PostgreSQL, 27017 MongoDB)।

৩. প্রয়োজনীয় AWS সেবা

`VPC Security Groups`, `Network ACLs`, `VPC Flow Logs` (ঐচ্ছিক যাচাই)।

৪. আর্কিটেকচার

```
Internet → [SG-web: 80/443 from 0.0.0.0/0] → ALB / Web EC2
                → [SG-app: 3000 only from SG-web] → App EC2
                  → [SG-db: 5432 only from SG-app] → RDS
NACL (subnet স্তর): পাবলিক সাবনেটে 80/443 ইন, সব আউট;
প্রাইভেট সাবনেটে শুধু VPC ভেতরের ট্র্যাফিক + ephemeral পোর্ট।
SG = ইনস্ট্যান্সের দরজা; NACL = সাবনেটের গেট।
```

Security Group	Inbound	Outbound
`technova-sg-web`	80, 443 — `0.0.0.0/0`; 22 — শুধু নিজের IP `203.0.113.10/32`	সব (ডিফল্ট)
`technova-sg-app`	3000 — শুধু `technova-sg-web` থেকে; 22 — শুধু `technova-sg-web` থেকে	সব
`technova-sg-db`	5432 — শুধু `technova-sg-app` থেকে	সব

৫. ধাপে ধাপে

ধাপ ১ — তিনটি Security Group তৈরি (Console)

`EC2 → Security Groups → Create security group` → প্রতিটির নাম, বর্ণনা ও VPC (`technova-dev-vpc`) দিন। Inbound নিয়ম উপরের টেবিল অনুযায়ী যোগ করুন। খেয়াল করুন: Source-এ Security Group ID লেখা যায় — IP লিখতে হয় না। এটাই সঠিক পদ্ধতি: IP বদলালেও নিয়ম ভাঙে না।

ধাপ ২ — একই কাজ CLI দিয়ে

```
VPC_ID=CHANGE-ME-VPC-ID
WEB_SG=$(aws ec2 create-security-group --group-name technova-sg-web \
  --description "TechNova web tier" --vpc-id $VPC_ID \
  --query 'GroupId' --output text)
aws ec2 authorize-security-group-ingress --group-id $WEB_SG \
  --protocol tcp --port 80 --cidr 0.0.0.0/0
aws ec2 authorize-security-group-ingress --group-id $WEB_SG \
  --protocol tcp --port 443 --cidr 0.0.0.0/0
aws ec2 authorize-security-group-ingress --group-id $WEB_SG \
  --protocol tcp --port 22 --cidr 203.0.113.10/32
aws ec2 describe-security-groups --group-ids $WEB_SG
```

> **নিজের IP ব্যবহার করুন:** উদাহরণ IP `203.0.113.10/32` একটি ডকুমেন্টেশন IP — SSH নিয়মে নিজের আসল পাবলিক IP (`/32` সহ) দিন। `0.0.0.0/0` দিয়ে SSH খুলবেন না — এটি সবচেয়ে সাধারণ ও বিপজ্জনক ভুল।

ধাপ ৩ — NACL তৈরি ও নিয়ম লেখা

`VPC → Network ACLs → Create network ACL` → নাম `technova-nacl-public` → VPC নির্বাচন → সাবনেট অ্যাসোসিয়েশনে দুটি পাবলিক সাবনেট যুক্ত করুন। Inbound নিয়ম: Rule 100 — TCP 80 `0.0.0.0/0` Allow; Rule 110 — TCP 443 Allow; Rule 120 — TCP 22 শুধু নিজের IP Allow; Rule 130 — TCP 1024–65535 (ephemeral, ফেরত ট্র্যাফিকের জন্য) Allow। Outbound: Rule 100 — সব Allow।

ধাপ ৪ — Stateful বনাম Stateless পরীক্ষা (ধারণা যাচাই)

Security Group stateful: ইনবাউন্ড অনুমতি দিলে ফেরত ট্র্যাফিক আপনা-আপনি যায় — আলাদা আউটবাউন্ড নিয়ম লাগে না। NACL stateless: ইনবাউন্ড খুললে ফেরত পথের জন্য আউটবাউন্ডেও ephemeral পোর্ট খুলতে হয় — না হলে সংযোগ হবে না। এই পার্থক্যটি নোটবুকে লিখুন; ইন্টারভিউতে প্রায়ই জিজ্ঞেস করা হয়।

ধাপ ৫ — নিয়ম পরীক্ষা ও ভুল ধরার অনুশীলন

ইচ্ছা করে `technova-sg-app`-এর 3000 পোর্টের Source `0.0.0.0/0` করে দেখুন — তারপর আবার `technova-sg-web`-এ ফিরিয়ে আনুন। প্রতিবার বদলের পর `describe-security-groups` দিয়ে যাচাই করুন। ভুল নিয়ম কীভাবে ধরা যায়, সেটাই এই ধাপের শিক্ষা।

৬. যাচাই

- তিনটি SG সঠিক VPC-তে, সঠিক ইনবাউন্ড নিয়মসহ আছে।
- `technova-sg-db`-তে শুধু `technova-sg-app` থেকে 5432 খোলা — `0.0.0.0/0` কোথাও নেই।
- NACL-এ ইনবাউন্ড ও আউটবাউন্ড — দুদিকেই ephemeral নিয়ম আছে।
- SSH শুধু নিজের IP থেকে খোলা।

৭. ট্রাবলশুটিং

সমস্যা	কারণ ও সমাধান

SSH হয় না, যদিও SG-তে 22 খোলা	NACL-এ 22 বা ephemeral পোর্ট বন্ধ কি না দেখুন। SG ও NACL — দুটোই পাস করতে হয়।
ওয়েবসাইট খোলে না	SG-তে 80/443, NACL-এ 80/443 + ephemeral, রাউট টেবিলে IGW — তিনটি ধাপে ধাপে দেখুন।
`InvalidPermission.Duplicate`	একই নিয়ম দুবার যোগ করা হয়েছে। আগে `describe` করে দেখুন।

৮. নিরাপত্তা বিবেচনা

Security Group-কে "ভার্চুয়াল ফায়ারওয়াল" ভাবুন — ডিফল্ট Deny, শুধু যা বলবেন তাই Allow। ডাটাবেস পোর্ট কখনো ইন্টারনেটে খুলবেন না। নিয়মিত `AWS Trusted Advisor` বা `Security Hub`-এর "unrestricted SSH" সতর্কতা দেখুন।

৯. পরিষ্কার করা

এই ল্যাবের Security Group ও NACL **রেখে দিন** — Lab 7–10, 13-এ ব্যবহার হবে। শুধু ধাপ ৫-এর পরীক্ষামূলক ভুল নিয়মটি ঠিক করে রাখুন (`technova-sg-app`-এর Source আবার `technova-sg-web` করুন)। NACL-এর ডিফল্ট NACL মুছবেন না।

১০. চিন্তার প্রশ্ন

1. Security Group stateful হওয়ার সুবিধা কী? Stateless হলে কী অসুবিধা হতো?
2. ডাটাবেস SG-র Source-এ IP-র বদলে App SG ব্যবহার করলে কী লাভ?
3. একটি সাবনেটে একসঙ্গে কয়টি NACL যুক্ত করা যায়? একটি NACL কয়টি সাবনেটে যুক্ত করা যায়?

ল্যাব ৭ • কম্পিউট • ৭৫ মিনিট • কস্ট রিস্ক: Medium — EC2 চালু থাকলে চার্জ; শেষে Terminate করুন

Lab 7 — EC2 Deployment and Secure Access

১. উদ্দেশ্য

- Free Tier-যোগ্য EC2 ইনস্ট্যান্স চালু করতে পারবেন।
- Key Pair দিয়ে নিরাপদ SSH অ্যাক্সেস সেটআপ করতে পারবেন।
- User Data স্ক্রিপ্ট দিয়ে ইনস্ট্যান্স চালুর সময়ই ওয়েব সার্ভার বসাতে পারবেন।
- IAM রোল যুক্ত করে Access Key ছাড়াই AWS সেবা ব্যবহার করতে পারবেন।

২. পূর্বশর্ত

Lab 4–6 সম্পন্ন (VPC, সাবনেট, Security Group)। SSH ক্লায়েন্ট ও `.pem` কী ফাইল সংরক্ষণের জায়গা।

৩. প্রয়োজনীয় AWS সেবা

`EC2`, `Key Pairs`, `Security Groups`, `IAM Roles`।

৪. আর্কিটেকচার

```
[আপনার ল্যাপটপ] ──SSH (22, শুধু আপনার IP)──► [EC2 technova-dev-web-01]
   Public subnet-a (10.0.1.0/24), Amazon Linux 2023, t2.micro/t3.micro
   IAM Role: technova-ec2-s3-role (Lab 2) — S3 পড়তে পারে, কী লাগে না
   User Data: httpd ইনস্টল + "Hello from TechNova" পাতা
দ্বিতীয় ইনস্ট্যান্স: Private subnet-a — বাইরে থেকে SSH নয়, শুধু Bastion/SSM দিয়ে।
```

৫. ধাপে ধাপে

ধাপ ১ — Key Pair তৈরি (Console)

`EC2 → Key pairs → Create key pair` → নাম `technova-dev-key` → Type: RSA, Format: `.pem` → Create। `.pem` ফাইলটি একবারই ডাউনলোড হবে — নিরাপদ জায়গায় রাখুন। তারপর অনুমতি ঠিক করুন:

```
chmod 400 ~/technova-dev-key.pem
ls -l ~/technova-dev-key.pem
```

ধাপ ২ — পাবলিক EC2 চালু (Console)

`EC2 → Launch instance` → নাম `technova-dev-web-01` → AMI: *Amazon Linux* 2023 → Instance type: `t2.micro` (Free Tier) → Key pair: `technova-dev-key` → Network: `technova-dev-vpc`, Subnet: `technova-dev-public-a`, Auto-assign public IP: Enable → Security Group: `technova-sg-web` → Storage: 8 GB gp3 → *Advanced details* → *IAM instance profile*: `technova-ec2-profile` (Lab 2) → User Data-তে নিচের স্ক্রিপ্ট:

```
#!/bin/bash
dnf update -y
dnf install -y httpd
systemctl start httpd
systemctl enable httpd
echo "<h1>Hello from TechNova - $(hostname)</h1>" \
  > /var/www/html/index.html
```

Launch করুন। ২–৩ মিনিট অপেক্ষা করে Status Checks 2/2 *passed* দেখুন।

ধাপ ৩ — SSH করে যাচাই (CLI)

```
ssh -i ~/technova-dev-key.pem ec2-user@CHANGE-ME-PUBLIC-IP
# ইনস্ট্যান্সের ভেতরে:
whoami
curl localhost
aws s3 ls
exit
```

`aws s3 ls` Access Key ছাড়াই চলা উচিত — IAM রোলের প্রমাণ। ব্রাউজারে `http://CHANGE-ME-PUBLIC-IP` খুললে "Hello from TechNova" দেখা যাবে।

ধাপ ৪ — প্রাইভেট EC2 চালু (CLI) ও SSM দিয়ে সংযোগ

```
aws ec2 run-instances --image-id CHANGE-ME-AMAZON-LINUX-2023-AMI-ID \
  --instance-type t2.micro --key-name technova-dev-key \
  --security-group-ids CHANGE-ME-SG-APP-ID \
  --subnet-id CHANGE-ME-PRIVATE-A-SUBNET-ID \
  --iam-instance-profile Name=technova-ec2-profile \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=technova-dev-app-01}]'
aws ec2 describe-instances --filters Name=tag:Name,Values=technova-dev-app-01 \
  --query 'Reservations[].Instances[].State.Name'
```

প্রাইভেট ইনস্ট্যান্সে পাবলিক IP নেই — SSH সরাসরি হবে না। Console-এ ইনস্ট্যান্স নির্বাচন করে *Connect → Session Manager* দিয়ে সংযোগ করুন (SSM Agent Amazon Linux 2023-এ আগে থেকে থাকে; Instance Profile-এ `AmazonSSMManagedInstanceCore` পলিসি যুক্ত থাকতে হবে)।

ধাপ ৫ — Elastic IP (ঐচ্ছিক, বুঝে করুন)

পাবলিক IP প্রতিবার রিস্টার্টে বদলায়। স্থায়ী IP চাইলে Elastic IP Allocate করে Associate করুন — তবে মনে রাখবেন, ব্যবহার না করা EIP-তেও চার্জ হয়। ল্যাব শেষে Release করতে ভুলবেন না।

৬. যাচাই

- পাবলিক EC2-র Status Checks 2/2 passed।
- ব্রাউজারে পাবলিক IP-তে "Hello from TechNova" দেখা যায়।
- SSH-এ `curl localhost` একই পাতা দেখায়।
- IAM রোল দিয়ে `aws s3 ls` চলে — কোনো Access Key ফাইলে নেই।
- প্রাইভেট EC2-তে সরাসরি SSH ব্যর্থ হয়, Session Manager-এ সংযোগ হয়।

৭. ট্রাবলশুটিং

সমস্যা	কারণ ও সমাধান
`Permission denied (publickey)`	কী ফাইলের অনুমতি 400 নয়, ভুল ব্যবহারকারী (`ec2-user` নয়), বা ভুল কী। `ssh -v` দিয়ে দেখুন।
ওয়েবসাইট খোলে না, SSH হয়	httpd চলছে কি না দেখুন: `systemctl status httpd`। User Data লগ: `/var/log/cloud-init-output.log`।
Session Manager-এ সংযোগ হয় না	Instance Profile-এ SSM পলিসি আছে কি না, প্রাইভেট সাবনেটে NAT বা SSM Endpoint আছে কি না দেখুন।
ইনস্ট্যান্স চালু হচ্ছে না — `InsufficientInstanceCapacity`	অন্য AZ বা অন্য instance type (t3.micro) চেষ্টা করুন।

৮. নিরাপত্তা বিবেচনা

`.pem` ফাইল কখনো Git-এ তুলবেন না, কাউকে পাঠাবেন না। SSH পোর্ট শুধু নিজের IP থেকে খোলা রাখুন। পাসওয়ার্ড-ভিত্তিক SSH বন্ধ রাখুন (Amazon Linux-এ ডিফল্ট বন্ধ)। নিয়মিত `dnf update` দিয়ে প্যাচ দিন — পুরোনো AMI-তে পরিচিত দুর্বলতা থাকে।

৯. পরিষ্কার করা

```
aws ec2 terminate-instances --instance-ids CHANGE-ME-WEB-ID CHANGE-ME-APP-ID
aws ec2 wait instance-terminated --instance-ids CHANGE-ME-WEB-ID CHANGE-ME-APP-ID
# Elastic IP ব্যবহার করে থাকলে Release করুন:
aws ec2 release-address --allocation-id CHANGE-ME-EIP-ALLOC-ID
```

Console-এ গিয়ে দুটি ইনস্ট্যান্সের State *Terminated* দেখে নিন। Key Pair ফাইলটি নিজের মেশিনে রেখে দিতে পারেন — পরের ল্যাবে আবার লাগবে — কিন্তু AWS-এ Key Pair মুছতে চাইলে `EC2 → Key pairs → Delete` করুন।

১০. চিন্তার প্রশ্ন

1. প্রাইভেট সাবনেটের সার্ভারে সরাসরি SSH না গিয়ে Session Manager দিয়ে যাওয়া কেন নিরাপদ?
2. User Data স্ক্রিপ্ট প্রতিবার রিবুটে চলে, নাকি শুধু প্রথমবার? প্রমাণ কী?
3. AMI, Instance Type ও Key Pair — তিনটির ভূমিকা এক লাইনে লিখুন।

ল্যাব ৮ • স্টোরেজ • ৬০ মিনিট • কস্ট রিস্ক: Low

Lab 8 — EBS Volumes, Snapshots and Recovery

১. উদ্দেশ্য

- EC2-তে অতিরিক্ত EBS ভলিউম যুক্ত করে ফরম্যাট ও মাউন্ট করতে পারবেন।
- স্ন্যাপশট নিয়ে ডেটা ব্যাকআপ করতে পারবেন।
- স্ন্যাপশট থেকে নতুন ভলিউম বানিয়ে ডেটা পুনরুদ্ধার (recovery) করতে পারবেন।
- ভলিউমের ধরন (gp3/io2/st1) ও এনক্রিপশনের পার্থক্য বুঝবেন।

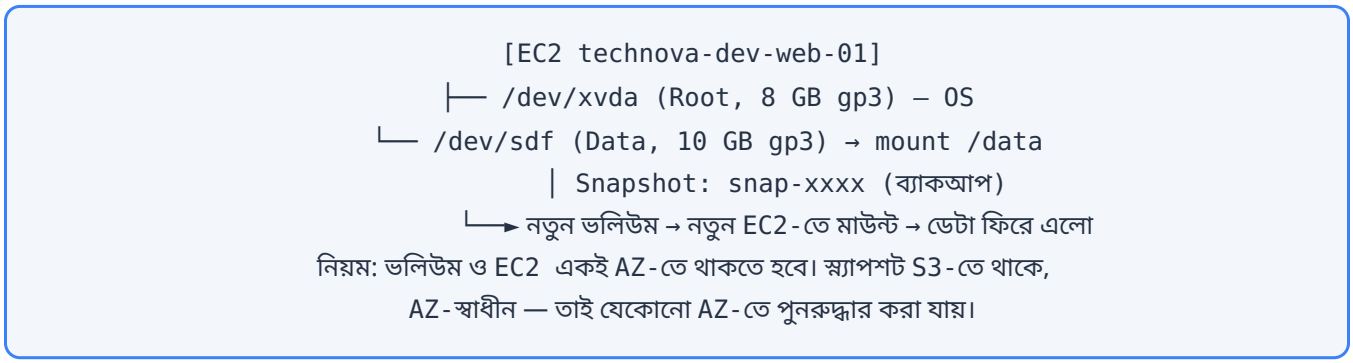
২. পূর্বশর্ত

Lab 7-এর EC2 চালু আছে (বা নতুন করে চালু করুন)। Linux-এ `mkfs`, `mount`-এর মৌলিক ধারণা।

৩. প্রয়োজনীয় AWS সেবা

`EC2`, `EBS Volumes`, `EBS Snapshots`, `KMS` (এনক্রিপশন)।

৪. আর্কিটেকচার



৫. ধাপে ধাপে

ধাপ ১ — অতিরিক্ত ভলিউম তৈরি (Console)

`EC2 → Volumes → Create volume` → Volume type: `gp3`, Size: 10 GiB, Availability Zone: EC2-র AZ-ই (`ap-south-1a`) → Encryption: Enable → নাম `technova-dev-data-01` → Create।

ধাপ ২ — ভলিউম যুক্ত করা (Console + CLI)

ভলিউম নির্বাচন → *Actions* → *Attach volume* → Instance: `technova-dev-web-01`, Device: `/dev/sdf` → Attach। CLI-তে:

```
aws ec2 create-volume --volume-type gp3 --size 10 \
  --availability-zone ap-south-1a --encrypted \
  --tag-specifications 'ResourceType=volume,Tags=[{Key=Name,Value=technova-dev-data-01}]'
```

```
aws ec2 attach-volume --volume-id CHANGE-ME-VOL-ID \
  --instance-id CHANGE-ME-INSTANCE-ID --device /dev/sdf
```

ধাপ ৩ — ফরম্যাট ও মাউন্ট (SSH-এর ভেতরে)

```
lsblk
sudo mkfs -t ext4 /dev/xvdf
sudo mkdir /data
sudo mount /dev/xvdf /data
df -h /data
echo "TechNova important data - $(date)" | sudo tee /data/important.txt
cat /data/important.txt
```

> **সতর্কতা:** `mkfs` ভলিউমের সব ডেটা মুছে ফেলে। ভুল ডিভাইসে (`/dev/xvda` — root) চালাবেন না। `lsblk` দিয়ে আগে নিশ্চিত হয়ে নিন কোনটি নতুন ভলিউম।

ধাপ ৪ — স্ন্যাপশট নেওয়া

`EC2 → Volumes → technova-dev-data-01 → Actions → Create snapshot` → বর্ণনা `technova-data-backup-day1` → Create। অথবা CLI:

```
aws ec2 create-snapshot --volume-id CHANGE-ME-VOL-ID \
  --description "technova-data-backup-day1" \
  --tag-specifications 'ResourceType=snapshot,Tags=[{Key=Name,Value=technova-data-backup-
day1}]'
aws ec2 describe-snapshots --snapshot-ids CHANGE-ME-SNAP-ID \
  --query 'Snapshots[].State'
```

State *completed* হওয়া পর্যন্ত অপেক্ষা করুন।

ধাপ ৫ — ডেটা "হারিয়ে" পুনরুদ্ধার (Recovery অনুশীলন)

```
# EC2-তে: ফাইল মুছে ফেলুন (ভান করুন ডেটা হারিয়েছে)
sudo rm /data/important.txt
ls /data/
# এবার স্ন্যাপশট থেকে নতুন ভলিউম বানান (Console বা CLI):
aws ec2 create-volume --snapshot-id CHANGE-ME-SNAP-ID \
  --availability-zone ap-south-1a --volume-type gp3 \
  --tag-specifications 'ResourceType=volume,Tags=[{Key=Name,Value=technova-dev-data-
restored}]'
# নতুন ভলিউম /dev/sdg হিসেবে Attach করুন, মাউন্ট করুন:
sudo mkdir /data-restored
sudo mount /dev/xvdg /data-restored
cat /data-restored/important.txt
```

ফাইলটি ফিরে এসেছে — এটাই স্ন্যাপশট-ভিত্তিক পুনরুদ্ধার।

৬. যাচাই

- `df -h`-তে `/data` 10 GB দেখায়।

- স্ন্যাপশট State *completed*।
- পুনরুদ্ধার করা ভলিউমে `important.txt` ফিরে এসেছে।
- ভলিউম Encrypted: Yes দেখায়।

৭. ট্রাবলশুটিং

সমস্যা	কারণ ও সমাধান
ভলিউম Attach হয় না	ভলিউম ও EC2 ভিন্ন AZ-তে। একই AZ-তে নতুন ভলিউম বানান।
`mount: wrong fs type`	ফরম্যাট করা হয়নি। আগে `mkfs` করুন — তবে স্ন্যাপশট থেকে বানানো ভলিউমে `mkfs` করবেন না, তাতে ডেটা মুছে যাবে।
রিবুটের পর মাউন্ট উধাও	`/etc/fstab` -এ এন্ট্রি দেননি। UUID দিয়ে fstab-এ যোগ করুন, ডিভাইস নাম দিয়ে নয়।

৮. নিরাপত্তা বিবেচনা

সব ভলিউম এনক্রিপ্ট করুন (KMS)। স্ন্যাপশট শেয়ার করার আগে ভাবুন — স্ন্যাপশটে পুরো ডিস্কের ডেটা থাকে, পাসওয়ার্ডসহ। স্ন্যাপশটেরও লাইফসাইকেল (পুরোনো স্ন্যাপশট মুছে ফেলা) ঠিক করুন — না হলে স্টোরেজ খরচ বাড়ে।

৯. পরিষ্কার করা

```
sudo umount /data /data-restored
aws ec2 detach-volume --volume-id CHANGE-ME-VOL-ID
aws ec2 detach-volume --volume-id CHANGE-ME-RESTORED-VOL-ID
aws ec2 delete-volume --volume-id CHANGE-ME-VOL-ID
aws ec2 delete-volume --volume-id CHANGE-ME-RESTORED-VOL-ID
aws ec2 delete-snapshot --snapshot-id CHANGE-ME-SNAP-ID
aws ec2 terminate-instances --instance-ids CHANGE-ME-INSTANCE-ID
```

স্ন্যাপশট মুছে ফেলার আগে নোটবুকে তার ID ও বর্ণনা লিখে রাখুন — কী শিখলেন, সেটাই আসল ব্যাকআপ।

১০. চিন্তার প্রশ্ন

1. স্ন্যাপশট নেওয়ার সময় EC2 চালু থাকলে কী ঝুঁকি থাকে? `fsfreeze` বা অ্যাপ বন্ধ করে নেওয়া কেন নিরাপদ?
2. gp3, io2 ও st1 — কোন কাজে কোনটি? একটি ডাটাবেসের জন্য কোনটি বেছে নেবেন?
3. স্ন্যাপশট থেকে EC2-র root ভলিউমও পুনরুদ্ধার করা যায় কি? কীভাবে?

ল্যাব ৯ • লোড ব্যালান্সিং • ৭৫ মিনিট • কস্ট রিস্ক: Medium — ALB প্রতি ঘণ্টায় চার্জ; শেষে মুছুন

Lab 9 — Application Load Balancer

১. উদ্দেশ্য

- দুটি AZ-তে Application Load Balancer (ALB) বানাতে পারবেন।
- Target Group ও Health Check-এর কাজ বুঝে সেটআপ করতে পারবেন।
- ALB-র DNS নাম দিয়ে ট্র্যাফিক দুটি EC2-তে ভাগ হতে দেখবেন।
- অসুস্থ (unhealthy) ইনস্ট্যান্স থেকে ট্র্যাফিক সরিয়ে নেওয়া যাচাই করবেন।

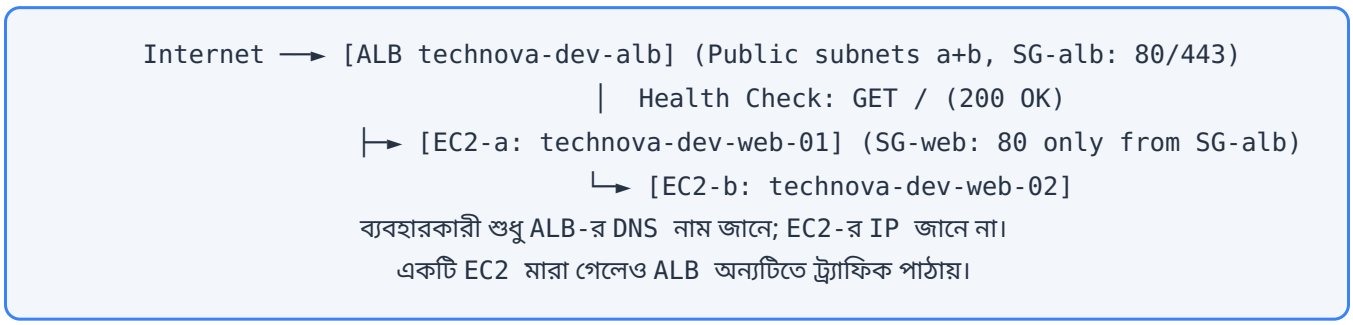
২. পূর্বশর্ত

Lab 4–7 সম্পন্ন। দুটি EC2 (AZ-a ও AZ-b, পাবলিক সাবনেটে) — প্রতিটিতে httpd চলছে ও ভিন্ন ভিন্ন "Hello" পাতা আছে।

৩. প্রয়োজনীয় AWS সেবা

`Elastic Load Balancing (ALB)`, `Target Groups`, `EC2`, `Security Groups`।

৪. আর্কিটেকচার



৫. ধাপে ধাপে

ধাপ ১ — ALB-র জন্য Security Group

`technova-sg-alb` বানান: Inbound 80 ও 443 — `0.0.0.0/0` থেকে। তারপর `technova-sg-web`-এর Inbound নিয়ম বদলে 80-এর Source `0.0.0.0/0`-এর বদলে `technova-sg-alb` করুন — এখন ওয়েব সার্ভারে সরাসরি ইন্টারনেট থেকে ঢোকা যাবে না, শুধু ALB দিয়ে যাবে।

ধাপ ২ — Target Group তৈরি (Console)

`EC2 → Target Groups → Create target group` → Type: Instances → নাম `technova-dev-web-tg` → Protocol HTTP, Port 80 → VPC: `technova-dev-vpc` → Health check path: `/`, Healthy threshold 2, Unhealthy threshold 2, Interval 30s → Next → দুটি EC2 নির্বাচন করে *Include as pending below* → Create।

ধাপ ৩ — ALB তৈরি (Console)

`EC2 → Load Balancers → Create load balancer → Application Load Balancer` → নাম `technova-dev-alb` → Scheme: Internet-facing → VPC ও দুটি পাবলিক সাবনেট (AZ-a, AZ-b) → Security Group: `technova-sg-alb` → Listener: HTTP 80 → Forward to: `technova-dev-web-tg` → Create। তৈরি হতে ২–৩ মিনিট লাগে।

ধাপ ৪ — CLI দিয়ে একই কাজ (রেফারেন্স)

```
aws elbv2 create-target-group --name technova-dev-web-tg \
  --protocol HTTP --port 80 --vpc-id CHANGE-ME-VPC-ID \
  --health-check-path / --target-type instance
aws elbv2 register-targets --target-group-arn CHANGE-ME-TG-ARN \
  --targets Id=CHANGE-ME-EC2-A-ID Id=CHANGE-ME-EC2-B-ID
aws elbv2 create-load-balancer --name technova-dev-alb \
  --subnets CHANGE-ME-PUB-A CHANGE-ME-PUB-B \
  --security-groups CHANGE-ME-SG-ALB-ID --scheme internet-facing \
  --type application
aws elbv2 create-listener --load-balancer-arn CHANGE-ME-ALB-ARN \
  --protocol HTTP --port 80 \
  --default-actions Type=forward,TargetGroupArn=CHANGE-ME-TG-ARN
```

ধাপ ৫ — ট্র্যাফিক পরীক্ষা

```
ALB_DNS=CHANGE-ME-ALB-DNS-NAME
for i in 1 2 3 4 5 6; do curl -s http://$ALB_DNS/; echo; sleep 1; done
aws elbv2 describe-target-health --target-group-arn CHANGE-ME-TG-ARN
```

ছয়বার `curl` করলে দুটি সার্ভারের নাম পালাক্রমে আসা উচিত — এটাই রাউন্ড-রবিন লোড ব্যালান্সিং। দুটি টার্গেটেরই Health *healthy* দেখাবে।

ধাপ ৬ — Failover পরীক্ষা

একটি EC2-তে SSH করে `sudo systemctl stop httpd` করুন। ৩০–৬০ সেকেন্ড পর Target Health-এ সেটি *unhealthy* হবে, আর সব `curl` অন্যটিতে যাবে। তারপর আবার `systemctl start httpd` করে সুস্থ করুন।

৬. যাচাই

- ALB state *Active*, DNS নামে ব্রাউজারে পাতা খোলে।
- দুটি টার্গেটই *healthy*।
- বারবার রিফ্রেশে দুটি সার্ভারের উত্তর পালাক্রমে আসে।
- একটি সার্ভার বন্ধ করলে অন্যটি সব ট্র্যাফিক নেয় — ডাউনটাইম নেই।

৭. ট্রাবলশুটিং

সমস্যা	কারণ ও সমাধান
টার্গেট *unhealthy* — 502/503	Health Check পাথ `/` -তে 200 আসে কি না দেখুন। Security Group-এ ALB থেকে EC2-র 80 খোলা আছে কি না দেখুন — সবচেয়ে সাধারণ ভুল এটাই।

ALB DNS-এ সংযোগ হয় না	ALB-র SG-তে 80 খোলা আছে কি না, সাবনেট দুটি পাবলিক কি না দেখুন। ALB তৈরি হতে সময় লাগে — state Active হওয়া পর্যন্ত অপেক্ষা করুন।
সব ট্র্যাফিক এক সার্ভারে যায়	দুটি সার্ভারের "Hello" পাতা একই কি না দেখুন — User Data-তে hostname আলাদা কি না যাচাই করুন। Sticky sessions চালু থাকলে বন্ধ করুন।

৮. নিরাপত্তা বিবেচনা

EC2-র Security Group-এ সরাসরি `0.0.0.0/0` থেকে 80 খুলবেন না — শুধু ALB SG থেকে। প্রোডাকশনে HTTP Listener-এর পাশাপাশি HTTPS Listener (ACM সার্টিফিকেট, Lab 12) যোগ করুন ও HTTP → HTTPS রিডাইরেক্ট করুন। ALB Access Logs চালু করে S3-তে রাখুন — আক্রমণ তদন্তে লাগে।

৯. পরিষ্কার করা

```
aws elbv2 delete-listener --listener-arn CHANGE-ME-LISTENER-ARN
aws elbv2 delete-load-balancer --load-balancer-arn CHANGE-ME-ALB-ARN
aws elbv2 deregister-targets --target-group-arn CHANGE-ME-TG-ARN \
  --targets Id=CHANGE-ME-EC2-A-ID Id=CHANGE-ME-EC2-B-ID
aws elbv2 delete-target-group --target-group-arn CHANGE-ME-TG-ARN
aws ec2 terminate-instances --instance-ids CHANGE-ME-EC2-A-ID CHANGE-ME-EC2-B-ID
```

ALB মুছে ফেলার পর Target Group মুছুন — উল্টো করলে ALB মুছতে সমস্যা হয়। Console-এ Load Balancers তালিকা খালি দেখে নিশ্চিত হোন।

১০. চিন্তার প্রশ্ন

1. ALB, NLB ও CLB-র মধ্যে পার্থক্য কী? TechNova-র MERN অ্যাপে কোনটি কেন?
2. Health Check ব্যর্থ হলে ALB কী করে — আর কেন এটি "স্ব-নিরাময়" (self-healing)-এর ভিত্তি?
3. ALB-র পেছনের EC2-র IP বদলে গেলেও ব্যবহারকারীর কিছু করতে হয় না কেন?

ল্যাব ১০ • অটো স্কেলিং • ৭৫ মিনিট • কস্ট রিস্ক: Medium — স্কেল-আউটে EC2 বাড়ে; শেষে Desired 0 করুন

Lab 10 — Auto Scaling Group

১. উদ্দেশ্য

- Launch Template দিয়ে একইরকম EC2 বারবার বানাতে পারবেন।
- Auto Scaling Group (ASG) দিয়ে চাহিদা অনুযায়ী ইনস্ট্যান্স বাড়ানো-কমানো সেটআপ করতে পারবেন।
- CPU-ভিত্তিক স্কেলিং পলিসি ও Health Check প্রতিস্থাপন (replacement) পরীক্ষা করবেন।
- ALB-র সঙ্গে ASG যুক্ত করে স্বয়ংক্রিয়ভাবে ট্র্যাফিক পাওয়া দেখবেন।

২. পূর্বশর্ত

Lab 9-এর ALB ও Target Group-এর ধারণা। একটি কাজের AMI (Amazon Linux 2023 + httpd User Data)।

৩. প্রয়োজনীয় AWS সেবা

`EC2 Auto Scaling`, `Launch Templates`, `CloudWatch Alarms`, `Elastic Load Balancing`।

৪. আর্কিটেকচার

```
[CloudWatch Alarm: CPU > 60% for 2 min] ──► Scale OUT (+1)
[CloudWatch Alarm: CPU < 30% for 5 min] ──► Scale IN (-1)
   [ASG technova-dev-web-asg: min 1, desired 2, max 4]
          ├── AZ-a (public-a) ── EC2 (Launch Template)
          └── AZ-b (public-b) ── EC2 (Launch Template)
সব ইনস্ট্যান্স auto-register হয় Target Group-এ; অসুস্থ হলে ASG নতুন বানায়।
```

৫. ধাপে ধাপে

ধাপ ১ — Launch Template তৈরি (Console)

`EC2 → Launch Templates → Create launch template` → নাম `technova-dev-web-lt` → AMI: Amazon Linux 2023 → Instance type: `t2.micro` → Key pair: `technova-dev-key` → Security Group: `technova-sg-web` → IAM instance profile: `technova-ec2-profile` → User Data: Lab 7-এর httpd স্ক্রিপ্ট (hostname-সহ) → Create। *Version* 1 তৈরি হবে।

ধাপ ২ — Auto Scaling Group তৈরি (Console)

`EC2 → Auto Scaling Groups → Create Auto Scaling group` → নাম `technova-dev-web-asg` → Launch Template নির্বাচন → VPC: `technova-dev-vpc`, Subnets: দুটি পাবলিক সাবনেট → *Attach to an existing load balancer*: Target Group `technova-dev-web-tg` নির্বাচন → Health check type: ELB, Grace period 120s → Group size: Desired 2, Min 1, Max 4 → Create।

ধাপ ৩ — স্কেলিং পলিসি যোগ (Console)

ASG-র পাতায় *Automatic scaling* → *Create dynamic scaling policy* → Policy type: Target tracking → Metric: Average CPU utilization → Target value: 60 → Create। এটি CPU 60%-এর উপরে গেলে ইনস্ট্যান্স বাড়াবে, নিচে নামলে কমাবে।

ধাপ ৪ — CLI দিয়ে অবস্থা দেখা

```
aws autoscaling describe-auto-scaling-groups \
  --auto-scaling-group-names technova-dev-web-asg \
  --query 'AutoScalingGroups[].Instances[].
{Id:InstanceId,State:LifecycleState,Health:HealthStatus}'
aws elbv2 describe-target-health --target-group-arn CHANGE-ME-TG-ARN
```

দুটি ইনস্ট্যান্স *InService* ও *Healthy* হওয়া পর্যন্ত অপেক্ষা করুন (৩–৫ মিনিট)।

ধাপ ৫ — Self-healing পরীক্ষা

একটি ইনস্ট্যান্স Console থেকে Terminate করুন। ASG ১–২ মিনিটের মধ্যে নতুন ইনস্ট্যান্স চালু করবে — Desired 2 বজায় রাখতে। Activity History-তে `Launching a new EC2 instance` দেখুন। নতুন ইনস্ট্যান্স Target Group-এ আপনা-আপনি *healthy* হবে।

ধাপ ৬ — লোড দিয়ে স্কেল-আউট (ঐচ্ছিক)

একটি ইনস্ট্যান্সে SSH করে CPU চাপ দিন:

```
sudo dnf install -y stress
stress --cpu 2 --timeout 300 &
# CloudWatch-এ CPU 60% পেরিয়ে গেলে ASG নতুন ইনস্ট্যান্স বাড়াবে।
# পরীক্ষা শেষে: killall stress; Desired আবার 2-এ নামান।
```

> **খরচ সতর্কতা:** স্কেল-আউট পরীক্ষায় অতিরিক্ত EC2 চালু হয় — প্রতিটি ঘণ্টায় চার্জ হয়। পরীক্ষা শেষে ASG-র Desired capacity 0 করুন অথবা ASG মুছে ফেলুন। Max 4 রেখে ভুলে গেলে বিল বাড়তেই থাকবে।

৬. যাচাই

- ASG-তে 2টি ইনস্ট্যান্স InService, দুটি AZ-তে ভাগ করা।
- Target Group-এ দুটিই healthy; ALB DNS-এ পাতা খোলে।
- একটি Terminate করলে ASG নিজে নতুন বানায় (Activity History-তে প্রমাণ)।
- স্কেলিং পলিসি Target tracking 60% CPU-তে সেট আছে।

৭. ট্রাবলশুটিং

সমস্যা	কারণ ও সমাধান
ASG ইনস্ট্যান্স বানায়, কিন্তু সঙ্গে সঙ্গে Terminate করে	Health Check (ELB) ব্যর্থ — httpd চলছে না বা SG-তে ALB থেকে 80 বন্ধ। Launch Template-এর User Data লগ দেখুন।

স্কেল-আউট হয় না	CloudWatch অ্যালার্মের থ্রেশহোল্ড ও সময় দেখুন; Max capacity-তে পৌঁছে গেছে কি না দেখুন।
ইনস্ট্যান্স এক AZ-তেই জমা হয়	ASG-তে দুটি সাবনেটই নির্বাচিত আছে কি না দেখুন; AZ-র ক্যাপাসিটি সমস্যা হলে অন্য AZ-তে যায়।

৮. নিরাপত্তা বিবেচনা

Launch Template-এ কোনো Secret Key বা পাসওয়ার্ড লিখবেন না — User Data সবাই (যাদের EC2 দেখার অনুমতি আছে) পড়তে পারে। গোপন তথ্য Secrets Manager বা Parameter Store থেকে নিন (Lab 20, Project 02)। Termination Protection প্রোডাকশন ডেটাবেসে চালু করুন, stateless ওয়েব সার্ভারে নয়।

৯. পরিষ্কার করা

```
aws autoscaling update-auto-scaling-group \
  --auto-scaling-group-name technova-dev-web-asg \
  --min-size 0 --max-size 0 --desired-capacity 0
aws autoscaling delete-auto-scaling-group \
  --auto-scaling-group-name technova-dev-web-asg --force-delete
aws ec2 delete-launch-template --launch-template-name technova-dev-web-lt
```

Console-এ ASG তালিকা খালি ও EC2 তালিকায় কোনো চলমান ইনস্ট্যান্স নেই — নিশ্চিত হয়ে নিন। ALB ও Target Group Lab 9-এর মতো মুছুন (যদি এখনো থাকে)।

১০. চিন্তার প্রশ্ন

1. Desired, Min ও Max — তিনটির ভূমিকা কী? Min 0 রাখলে কী হয়?
2. ELB Health Check বনাম EC2 Health Check — কখন কোনটি ব্যবহার করবেন?
3. হঠাৎ ট্র্যাফিক বাড়লে ASG-র প্রতিক্রিয়া জানাতে কত সময় লাগে? এই দেরি কীভাবে কমানো যায়?

ল্যাব ১১ • স্টোরেজ • ৬০ মিনিট • কস্ট রিস্ক: Low

Lab 11 — S3 Bucket Policies, Versioning and Lifecycle

১. উদ্দেশ্য

- নিরাপদ S3 বাকেট বানাতে পারবেন (Block Public Access, এনক্রিপশন, ভার্সনিং)।
- Bucket Policy লিখে নির্দিষ্ট ব্যবহারকারী/সেবাকে অনুমতি দিতে পারবেন।
- লাইফসাইকেল নিয়ম দিয়ে পুরোনো ফাইল স্বয়ংক্রিয়ভাবে সস্তা স্টোরেজে পাঠাতে পারবেন।
- ভুল করে মোছা ফাইল ভার্সনিং দিয়ে ফিরিয়ে আনতে পারবেন।

২. পূর্বশর্ত

Lab 2-এর IAM ধারণা। একটি অনন্য বাকেট নাম (সব AWS-এ বাকেট নাম বিশ্বব্যাপী অনন্য হতে হয়)।

৩. প্রয়োজনীয় AWS সেবা

`S3`, `IAM Policies`, `S3 Lifecycle`, `CloudFront (OAC — Lab 12-এ)`।

৪. আর্কিটেকচার

```
                [S3 Bucket: technova-lab-assets-CHANGE-ME]
                  ├── Versioning: Enabled (মোছা ফাইলও ফেরে)
                      ├── Encryption: SSE-S3 (AES-256)
                      ├── Block Public Access: ON (সব)
        ├── Bucket Policy: শুধু CloudFront OAC + নির্দিষ্ট IAM Role পড়তে পারে
  └── Lifecycle: 30 দিন পর Standard-IA → 90 দিন পর Glacier → 365 দিন পর মুছে ফেলা
        Public ওয়েবসাইট হোস্টিং এই ল্যাবে নয় — Lab 12-এ CloudFront দিয়ে হবে।
```

৫. ধাপে ধাপে

ধাপ ১ — বাকেট তৈরি (CLI)

```
BUCKET=technova-lab-assets-CHANGE-ME
aws s3api create-bucket --bucket $BUCKET --region ap-south-1 \
  --create-bucket-configuration LocationConstraint=ap-south-1
aws s3api put-public-access-block --bucket $BUCKET \
  --public-access-block-configuration \
  BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
aws s3api put-bucket-versioning --bucket $BUCKET \
  --versioning-configuration Status=Enabled
aws s3api put-bucket-encryption --bucket $BUCKET \
  --server-side-encryption-configuration \
```

```
  '{"Rules":[{"ApplyServerSideEncryptionByDefault":{"SSEAlgorithm":"AES256"}}]}'
aws s3api get-bucket-versioning --bucket $BUCKET
```

ধাপ ২ — ফাইল আপলোড ও ভার্সনিং পরীক্ষা

```
echo "TechNova v1" > hello.txt
aws s3 cp hello.txt s3://$BUCKET/hello.txt
echo "TechNova v2 - updated" > hello.txt
aws s3 cp hello.txt s3://$BUCKET/hello.txt
aws s3api list-object-versions --bucket $BUCKET --prefix hello.txt \
  --query 'Versions[].{VersionId:VersionId,IsLatest:IsLatest}'
```

দুটি ভার্সন দেখা যাবে — নতুনটি Latest।

ধাপ ৩ — মোছা ফাইল ফিরিয়ে আনা

```
aws s3 rm s3://$BUCKET/hello.txt
aws s3 ls s3://$BUCKET/
# ফাইল "মুছে গেছে" — কিন্তু ভার্সন আছে:
aws s3api list-object-versions --bucket $BUCKET --prefix hello.txt
# Delete Marker-টি মুছে ফেলুন — ফাইল ফিরে আসবে:
aws s3api delete-object --bucket $BUCKET --key hello.txt \
  --version-id CHANGE-ME-DELETE-MARKER-VERSION-ID
aws s3 cp s3://$BUCKET/hello.txt ./recovered.txt
cat recovered.txt
```

> **Delete Marker:** ভার্সনিং চালু থাকলে `rm` আসলে ফাইল মোছে না — একটি Delete Marker বসায়। Marker মুছে দিলেই আগের ভার্সন আবার দেখা যায়। এটাই ভার্সনিংয়ের জাদু — আর এজন্যই বাকেট খালি করতে হলে সব ভার্সন মুছতে হয়।

ধাপ ৪ — Bucket Policy (শুধু নির্দিষ্ট রোল পড়তে পারে)

`S3 → Bucket → Permissions → Bucket policy → Edit`-এ নিচের নীতি বসান (নিজের অ্যাকাউন্ট ID ও রোল নাম বসিয়ে):

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"AWS": "arn:aws:iam::111122223333:role/technova-ec2-s3-role"},
    "Action": ["s3:GetObject"],
    "Resource": "arn:aws:s3:::technova-lab-assets-CHANGE-ME/*"
  }]
}
```

ধাপ ৫ — লাইফসাইকেল নিয়ম (Console)

`S3 → Bucket → Management → Create lifecycle rule` → নাম `technova-archive-old` → Scope: সব অবজেক্ট → Transition: 30 দিন পর *Standard-IA*, 90 দিন পর *Glacier Flexible Retrieval* → Expiration: 365 দিন পর মুছে ফেলা → Create।

ধাপ ৬ — প্রি-সাইন্ড URL (অস্থায়ী শেয়ার)

```
aws s3 presign s3://$BUCKET/hello.txt --expires-in 3600
# ১ ঘণ্টা মেয়াদি URL পাবেন — ব্রাউজারে খুলে দেখুন।
# মেয়াদ শেষে URL কাজ করবে না — বাকেট পাবলিক না করেও শেয়ার করার উপায়।
```

৬. যাচাই

- Block Public Access চারটি সেটিংই ON।
- Versioning Enabled; দুটি ভার্সন তালিকায় আছে।
- মোছা ফাইল Delete Marker সরিয়ে ফিরিয়ে আনা গেছে।
- লাইফসাইকেল নিয়ম তালিকায় আছে।
- প্রি-সাইন্ড URL ১ ঘণ্টা কাজ করে, তারপর ব্যর্থ হয়।

৭. ট্রাবলশুটিং

সমস্যা	কারণ ও সমাধান
`BucketAlreadyExists`	নামটি অন্য কেউ নিয়েছে (বিশ্বব্যাপী অনন্য)। নামে নিজের নাম/তারিখ যোগ করুন।
বাকেট খালি করা যাচ্ছে না	ভার্সনসহ সব অবজেক্ট মুছতে হয়। Console-এ *Empty* বোতাম ভার্সনসহ মোছে — অথবা CLI-তে প্রতিটি VersionId ধরে মুছুন।
`AccessDenied` — GetObject	Bucket Policy-র Principal ভুল, বা Block Public Access পলিসিকে আটকাচ্ছে। Policy Simulator দিয়ে যাচাই করুন।

৮. নিরাপত্তা বিবেচনা

বাকেট কখনো পাবলিক করবেন না — পাবলিক ফাইল দরকার হলে CloudFront + OAC ব্যবহার করুন (Lab 12)। সংবেদনশীল বাকেটে *Default encryption* ও *Bucket Key* চালু করুন। `aws s3 ls`-এর আউটপুটে গোপন ফাইলের নামও ফাঁস হতে পারে — নামকরণে সতর্ক থাকুন।

৯. পরিষ্কার করা

```
aws s3 rm s3://$BUCKET --recursive
# ভার্সনসহ খালি করুন (Console → Empty), তারপর:
aws s3api delete-bucket --bucket $BUCKET
aws s3api list-buckets --query 'Buckets[].Name' | grep technova-lab-assets
```

বাকেট মুছে ফেলার আগে লাইফসাইকেল নিয়মের স্ক্রিনশট নোটবুকে রাখুন।

১০. চিন্তার প্রশ্ন

1. ভার্সনিং চালু থাকলে স্টোরেজ খরচ কেন বাড়ে? লাইফসাইকেল দিয়ে কীভাবে কমাবেন?
2. Bucket Policy বনাম IAM Policy — কখন কোনটি ব্যবহার করবেন?

3. প্রি-সাইন্ড URL কেন পাবলিক বাকেটের চেয়ে নিরাপদ?

ল্যাব ১২ • CDN ও DNS • ৭৫ মিনিট • কস্ট রিস্ক: Low — Route 53 হোস্টেড জোন চার্জ হয়; .example ডোমেইন ব্যবহার করুন

Lab 12 — CloudFront, Route 53 and HTTPS

১. উদ্দেশ্য

- S3-কে Origin করে CloudFront Distribution বানাতে পারবেন।
- Origin Access Control (OAC) দিয়ে S3-কে শুধু CloudFront-এর জন্য খুলতে পারবেন।
- Route 53-এ DNS রেকর্ড বানিয়ে নিজের ডোমেইনে সাইট চালাতে পারবেন।
- ACM দিয়ে বিনামূল্যে HTTPS সার্টিফিকেট নিয়ে HTTP → HTTPS চালু করতে পারবেন।

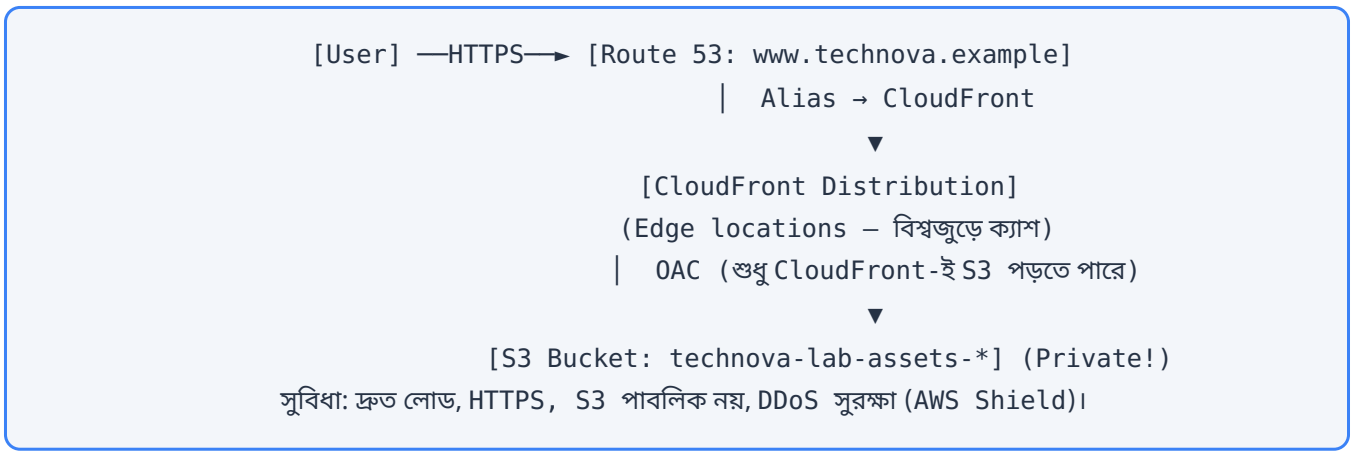
২. পূর্বশর্ত

Lab 11-এর S3 বাকেট (স্ট্যাটিক ফাইলসহ)। একটি ডোমেইন নাম থাকলে ভালো — না থাকলে `technova.example` দিয়ে অনুশীলন করুন (বাস্তবে রাউট হবে না, কিন্তু কনফিগারেশন শিখবেন)।

৩. প্রয়োজনীয় AWS সেবা

`CloudFront`, `Route 53`, `ACM (Certificate Manager)`, `S3`।

৪. আর্কিটেকচার



৫. ধাপে ধাপে

ধাপ ১ — S3-তে স্ট্যাটিক সাইট ফাইল

```
cat > index.html <<'EOF'
<!DOCTYPE html><html><head><meta charset="utf-8">
<title>TechNova</title></head><body>
<h1>TechNova — CloudFront দিয়ে পরিবেশিত হচ্ছে!</h1>
</body></html>
```

```
EOF
aws s3 cp index.html s3://technova-lab-assets-CHANGE-ME/index.html
```

ধাপ ২ — CloudFront Distribution (Console)

`CloudFront → Create distribution` → Origin domain: S3 বাকেট নির্বাচন করুন → Origin access: *Origin access control settings* (*recommended*) → Create new OAC → Viewer protocol policy: *Redirect HTTP to HTTPS* → Default root object: `index.html` → Create distribution। তৈরি হতে ৫–১০ মিনিট লাগে — Distribution domain name (`d111111abcdef8.cloudfront.net`-এর মতো) কপি করুন।

> **OAC নীতি কপি করুন:** Distribution তৈরির পর CloudFront একটি সতর্কতা দেখাবে — S3 Bucket Policy আপডেট করতে হবে। *Copy policy* করে S3 বাকেটের Bucket Policy-তে বসান। না করলে 403 Forbidden আসবে।

ধাপ ৩ — CloudFront URL যাচাই

```
curl -I https://CHANGE-ME-CLOUDFRONT-DOMAIN/
# HTTP/2 200, x-cache: Hit from cloudfront (দ্বিতীয়বার) দেখা উচিত।
curl -s https://CHANGE-ME-CLOUDFRONT-DOMAIN/ | head -5
```

প্রথমবার `Miss from cloudfront`, দ্বিতীয়বার `Hit` — ক্যাশ কাজ করছে।

ধাপ ৪ — ACM সার্টিফিকেট (আসল ডোমেইন থাকলে)

`ACM → Request certificate → Request a public certificate` → ডোমেইন: `www.technova.example` (নিজের আসল ডোমেইন) → Validation: DNS → Request। Route 53-এ CNAME রেকর্ড যোগ করে ভ্যালিডেট করুন। **মনে রাখবেন:** CloudFront-এর সার্টিফিকেট `us-east-1` রিজিয়নে নিতে হয় — অন্য রিজিয়নে নিলে CloudFront-এ দেখা যাবে না।

ধাপ ৫ — Route 53 রেকর্ড (আসল ডোমেইন থাকলে)

`Route 53 → Hosted zones → নিজের ডোমেইন` → *Create record* → Record name: `www` → Record type: A → Alias: ON → Route traffic to: CloudFront distribution → Distribution নির্বাচন → Create। কয়েক মিনিট পর `https://www.technova.example` খুলবে।

ধাপ ৬ — ক্যাশ ইনভ্যালিডেশন

```
aws cloudfront create-invalidation --distribution-id CHANGE-ME-DIST-ID \
  --paths "/*"
# ফাইল বদলালে CloudFront-কে জানাতে হয় — না হলে পুরোনো ক্যাশ দেখাবে।
```

৬. যাচাই

- CloudFront URL-এ HTTPS-এ পাতা খোলে, `x-cache: Hit` দেখা যায়।
- S3 বাকেটের URL সরাসরি খুললে 403 আসে (OAC কাজ করছে)।
- HTTP খুললে HTTPS-এ রিডাইরেক্ট হয়।
- (আসল ডোমেইন থাকলে) নিজের ডোমেইনে HTTPS-এ সাইট খোলে।

৭. ট্রাবলশুটিং

সমস্যা	কারণ ও সমাধান
403 Forbidden	OAC-র Bucket Policy বাকেটে বসানো হয়নি, বা Origin-এর নাম ভুল। CloudFront-এর দেওয়া পলিসিটি হুবহু বসান।
পুরোনো পাতা দেখাচ্ছে	ক্যাশ। Invalidation করুন অথবা কয়েক মিনিট অপেক্ষা করুন।
ACM সার্টিফিকেট CloudFront-এ দেখা যায় না	সার্টিফিকেট `us-east-1` -এ নেওয়া হয়নি। সঠিক রিজিয়নে নতুন করে নিন।
DNS রেকর্ড কাজ করে না	ডোমেইনের Name Server Route 53-র দিকে আছে কি না দেখুন। DNS ছড়াতে ৫–৩০ মিনিট লাগতে পারে।

৮. নিরাপত্তা বিবেচনা

CloudFront-এ *Viewer protocol policy*: *Redirect HTTP to HTTPS* সবসময় চালু রাখুন। S3 Origin-এ OAC ব্যবহার করুন — পুরোনো OAI-র বদলে এটাই বর্তমান মান। WAF (Web Application Firewall) যোগ করলে SQL Injection ও bot আক্রমণ ঠেকানো যায় — Project 02-এ যোগ করবেন।

৯. পরিষ্কার করা

```
aws cloudfront update-distribution --id CHANGE-ME-DIST-ID \
  --distribution-config file://disabled-config.json
# অথবা Console → Distribution → Disable → (Disabled হলে) Delete
# Route 53 রেকর্ড মুছুন; Hosted Zone মুছতে চাইলে সব রেকর্ড আগে মুছুন।
aws s3 rm s3://technova-lab-assets-CHANGE-ME --recursive
aws s3api delete-bucket --bucket technova-lab-assets-CHANGE-ME
```

CloudFront Distribution মুছতে আগে Disable করতে হয় — Disable হতে ১০–১৫ মিনিট লাগতে পারে। এই ল্যাবের Distribution মুছে ফেলুন, তবে S3 বাকেটটি Lab 11-এর মতোই মুছুন।

১০. চিন্তার প্রশ্ন

1. CloudFront ছাড়া সরাসরি S3 ওয়েবসাইট হোস্টিং করলে কী কী অসুবিধা?
2. Edge Location কী? ঢাকার ব্যবহারকারী কোন Edge থেকে ফাইল পাবে?
3. ক্যাশ ইনভ্যালিডেশন না করে ফাইল আপডেট করলে ব্যবহারকারী কী দেখবে — কেন?

ল্যাব ১৩ • ডাটাবেস • ৭৫ মিনিট • কস্ট রিস্ক: High — RDS চালু থাকলে প্রতি ঘণ্টায় চার্জ; ল্যাব শেষেই মুছুন

Lab 13 — RDS Database Deployment and Backup

১. উদ্দেশ্য

- প্রাইভেট সাবনেটে Multi-AZ নয় — প্রথমে Single-AZ RDS PostgreSQL বানাতে পারবেন।
- Security Group দিয়ে শুধু অ্যাপ সার্ভার থেকে ডাটাবেসে ঢোকার নিয়ম করতে পারবেন।
- psql দিয়ে সংযোগ করে টেবিল বানিয়ে ডেটা ঢোকাতে পারবেন।
- অটোমেটেড ব্যাকআপ ও ম্যানুয়াল স্ন্যাপশট নিয়ে পুনরুদ্ধার করতে পারবেন।

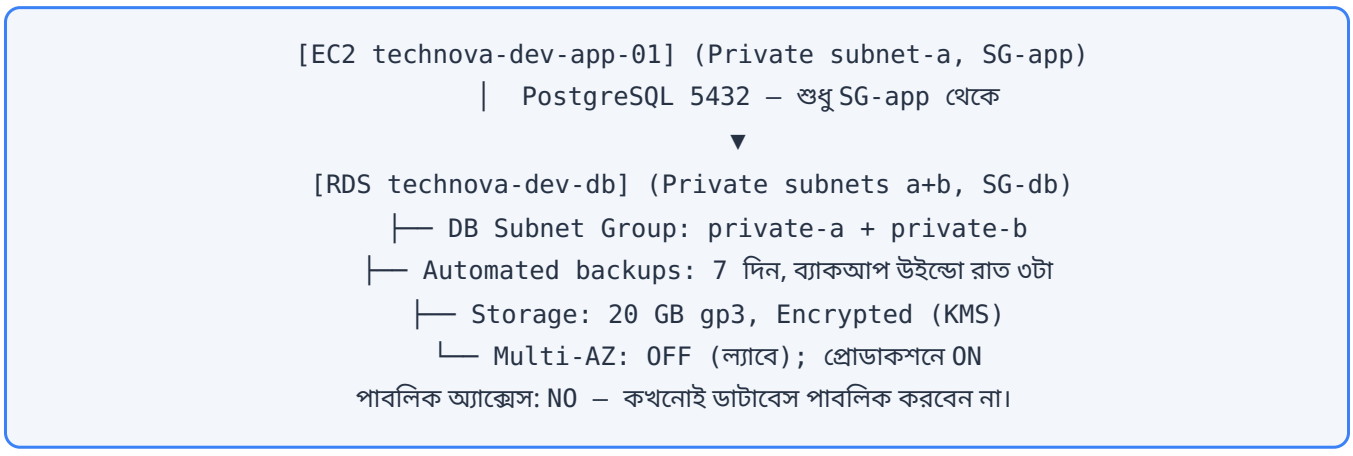
২. পূর্বশর্ত

Lab 4–6-এর VPC ও Security Group (`technova-sg-db`, `technova-sg-app`)। Lab 7-এর EC2 (অ্যাপ সার্ভার হিসেবে)।

৩. প্রয়োজনীয় AWS সেবা

`RDS (PostgreSQL)`, `DB Subnet Group`, `Security Groups`, `RDS Snapshots`।

৪. আর্কিটেকচার



৫. ধাপে ধাপে

ধাপ ১ — DB Subnet Group তৈরি

`RDS → Subnet groups → Create DB subnet group` → নাম `technova-dev-db-subnet` → VPC: `technova-dev-vpc` → Subnets: দুটি প্রাইভেট সাবনেট (AZ-a, AZ-b) → Create। CLI:

```
aws rds create-db-subnet-group --db-subnet-group-name technova-dev-db-subnet \
  --db-subnet-group-description "TechNova dev DB subnets" \
  --subnet-ids CHANGE-ME-PRIV-A CHANGE-ME-PRIV-B
```

ধাপ ২ — RDS ইনস্ট্যান্স তৈরি (Console)

`RDS → Create database → Standard create` → Engine: PostgreSQL 16 → Templates: *Free tier* (বা Dev/Test) → DB instance identifier: `technova-dev-db` → Master username: `technova_admin` → Master password: `CHANGE-ME-Db-Password-123!` (নিজের শক্তিশালীটি) → Instance: `db.t3.micro` → Storage: 20 GB gp3 → VPC, Subnet group ও Security Group: `technova-sg-db` → Public access: *No* → Backup retention: 7 দিন → Encryption: Enable → Create database। তৈরি হতে ৫–১০ মিনিট লাগে।

ধাপ ৩ — অ্যাপ সার্ভার থেকে সংযোগ (psql)

EC2-তে SSH/Session Manager দিয়ে:

```
sudo dnf install -y postgresql15
DB_HOST=CHANGE-ME-RDS-ENDPOINT
psql -h $DB_HOST -U technova_admin -d postgres
-- psql-এর ভেতরে:
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(200) UNIQUE NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
INSERT INTO users (name, email) VALUES
  ('Ayesha Rahman', 'ayesha@technova.example'),
  ('Tanvir Hasan', 'tanvir@technova.example');
SELECT * FROM users;
\q
```

ধাপ ৪ — ম্যানুয়াল স্ন্যাপশট ও পুনরুদ্ধার

```
aws rds create-db-snapshot --db-instance-identifier technova-dev-db \
  --db-snapshot-identifier technova-dev-db-snap-01
aws rds describe-db-snapshots --db-snapshot-identifier technova-dev-db-snap-01 \
  --query 'DBSnapshots[].Status'
# Status available হলে স্ন্যাপশট থেকে নতুন DB বানিয়ে যাচাই করুন:
aws rds restore-db-instance-from-db-snapshot \
  --db-instance-identifier technova-dev-db-restored \
  --db-snapshot-identifier technova-dev-db-snap-01 \
  --db-instance-class db.t3.micro
```

পুনরুদ্ধার করা DB-তে psql দিয়ে `SELECT * FROM users;` চালিয়ে দেখুন — দুটি রো ফিরে এসেছে।

ধাপ ৫ — Parameter Group ও লগ (ধারণা)

Console-এ DB-র *Configuration* ট্যাবে Parameter Group দেখুন। `log_min_duration_statement` বদলে ধীর কুয়েরি লগ করা যায় — তবে ল্যাবে বদলাবেন না, শুধু দেখুন।

৬. যাচাই

- RDS status *Available*, Endpoint থেকে psql সংযোগ হয়।
- `users` টেবিলে 2টি রো আছে।

- EC2 ছাড়া নিজের ল্যাপটপ থেকে সরাসরি psql ব্যর্থ হয় (প্রাইভেট — এটাই সঠিক)।
- স্ন্যাপশট *available*, পুনরুদ্ধার করা DB-তেও ডেটা আছে।
- অটোমেটেড ব্যাকআপ retention 7 দিন দেখায়।

৭. ট্রাবলশুটিং

সমস্যা	কারণ ও সমাধান
`Connection timed out`	SG-db-তে 5432-র Source SG-app কি না দেখুন; RDS ও EC2 একই VPC-তে কি না দেখুন।
`password authentication failed`	মাস্টার পাসওয়ার্ড ভুল। RDS Console থেকে *Modify* → *Master password* বদলে নিন (Apply immediately)।
RDS তৈরি হতে অনেক সময় লাগছে	স্বাভাবিক — ৫–১০ মিনিট লাগে। Status *Creating* থাকলে অপেক্ষা করুন।
স্ন্যাপশট থেকে Restore ব্যর্থ	স্ন্যাপশট *available* হওয়ার আগেই Restore চেষ্টা করেছেন। Status দেখে নিন।

৮. নিরাপত্তা বিবেচনা

মাস্টার পাসওয়ার্ড কখনো কোডে লিখবেন না — Secrets Manager-এ রাখুন (Project 02)। ডাটাবেস পাবলিক করবেন না, এমনকি "সাময়িক পরীক্ষার জন্য"ও নয় — পরীক্ষা EC2-র ভেতর থেকে করুন। Storage এনক্রিপশন সবসময় ON রাখুন। নিয়মিত ব্যাকআপ পুনরুদ্ধার পরীক্ষা করুন — ব্যাকআপ নেওয়া হয়নি, এমন ধরে নিন যতক্ষণ না Restore পরীক্ষা করেছেন।

৯. পরিষ্কার করা

> **RDS মুছতে ভুলবেন না — এটি এই ওয়ার্কবুকের সবচেয়ে ব্যয়বহুল ভুল।** RDS চালু থাকলে প্রতি ঘণ্টায় চার্জ হয়, এমনকি ব্যবহার না করলেও।

```
aws rds delete-db-instance --db-instance-identifier technova-dev-db-restored \
  --skip-final-snapshot
aws rds delete-db-instance --db-instance-identifier technova-dev-db \
  --skip-final-snapshot
aws rds delete-db-snapshot --db-snapshot-identifier technova-dev-db-snap-01
aws rds delete-db-subnet-group --db-subnet-group-name technova-dev-db-subnet
aws ec2 terminate-instances --instance-ids CHANGE-ME-APP-ID
```

Console-এ RDS → Databases তালিকা খালি দেখে নিন। Final snapshot নিতে চাইলে `--skip-final-snapshot`-এর বদলে `--final-db-snapshot-identifier` দিন — তবে স্ন্যাপশটের স্টোরেজ চার্জ মনে রাখবেন।

১০. চিন্তার প্রশ্ন

1. Multi-AZ চালু করলে কী বদলায় — আর কেন এটি ব্যাকআপের বিকল্প নয়?
2. অটোমেটেড ব্যাকআপ বনাম ম্যানুয়াল স্ন্যাপশট — কোনটি কখন ব্যবহার করবেন?

3. ডাটাবেসের পাসওয়ার্ড ঘোরানো (rotation) কেন দরকার? Secrets Manager কীভাবে সাহায্য করে?

ল্যাব ১৪ • NoSQL • ৬০ মিনিট • কস্ট রিস্ক: Low — On-demand মোডে ব্যবহার করুন

Lab 14 — DynamoDB Table and Application Integration

১. উদ্দেশ্য

- DynamoDB টেবিল বানিয়ে Partition Key ও Sort Key-এর ধারণা প্রয়োগ করতে পারবেন।
- Node.js অ্যাপ থেকে AWS SDK দিয়ে Item লেখা, পড়া, আপডেট ও মোছা করতে পারবেন।
- Global Secondary Index (GSI) দিয়ে দ্বিতীয়ভাবে খোঁজ করতে পারবেন।
- On-demand বনাম Provisioned ক্যাপাসিটির পার্থক্য বুঝবেন।

২. পূর্বশর্ত

Node.js 20 ইনস্টল করা। Lab 2-এর IAM রোল/ব্যবহারকারী (DynamoDB অনুমতিসহ)।

৩. প্রয়োজনীয় AWS সেবা

`DynamoDB`, `IAM`, `CloudWatch` (মেট্রিক)।

৪. আর্কিটেকচার

```
[Node.js অ্যাপ — ল্যাপটপ বা EC2]
  |  AWS SDK v3 (DynamoDBClient)
  ▼
[DynamoDB Table: technova-users]
  Partition Key: userId (String)
  Sort Key:      createdAt (String, ISO তারিখ)
  GSI: EmailIndex — Partition Key: email
  Billing: On-demand (ল্যাবে নিরাপদ — ব্যবহার অনুযায়ী বিল)
উদাহরণ Item: { userId: "u-001", email: "ayesha@technova.example",
  name: "Ayesha Rahman", createdAt: "2026-11-09T10:00:00Z" }
```

৫. ধাপে ধাপে

ধাপ ১ — টেবিল তৈরি (CLI)

```
aws dynamodb create-table --table-name technova-users \
  --attribute-definitions AttributeName=userId,AttributeType=S \
    AttributeName=createdAt,AttributeType=S \
    AttributeName=email,AttributeType=S \
  --key-schema AttributeName=userId,KeyType=HASH \
    AttributeName=createdAt,KeyType=RANGE \
  --billing-mode PAY_PER_REQUEST \
```

```
  --global-secondary-indexes \

'IndexName=EmailIndex,KeySchema=[{AttributeName=email,KeyType=HASH}],Projection={ProjectionTyp
e=ALL}' \
  --tags Key=Project,Value=TechNova Key=Env,Value=dev
aws dynamodb wait table-exists --table-name technova-users
aws dynamodb describe-table --table-name technova-users \
  --query 'Table.TableStatus'
```

ধাপ ২ — IAM পলিসি (অ্যাপের জন্য)

অ্যাপের IAM ব্যবহারকারী/রোলে শুধু এই টেবিলের অনুমতি দিন:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["dynamodb:GetItem", "dynamodb:PutItem",
               "dynamodb:UpdateItem", "dynamodb:DeleteItem",
               "dynamodb:Query", "dynamodb:Scan"],
    "Resource": [
      "arn:aws:dynamodb:ap-south-1:111122223333:table/technova-users",
      "arn:aws:dynamodb:ap-south-1:111122223333:table/technova-users/index/*"
    ]
  }]
}
```

ধাপ ৩ — Node.js অ্যাপ (SDK v3)

```
mkdir technova-dynamo-app && cd technova-dynamo-app
npm init -y
npm install @aws-sdk/client-dynamodb @aws-sdk/lib-dynamodb
```

`app.mjs` ফাইল তৈরি করুন:

```
import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
import { DynamoDBDocumentClient, PutCommand,
  GetCommand, QueryCommand } from "@aws-sdk/lib-dynamodb";

const client = DynamoDBDocumentClient.from(
  new DynamoDBClient({ region: "ap-south-1" }));

// 1. লেখা
await client.send(new PutCommand({
  TableName: "technova-users",
  Item: { userId: "u-001", createdAt: "2026-11-09T10:00:00Z",
    name: "Ayesha Rahman", email: "ayesha@technova.example" }
}));
console.log("Put OK");

// 2. পড়া
const got = await client.send(new GetCommand({
  TableName: "technova-users",
  Key: { userId: "u-001", createdAt: "2026-11-09T10:00:00Z" }
}));
console.log("Got:", got.Item);
```

```
// 3. GSI দিয়ে ইমেইলে খোঁজা
const byEmail = await client.send(new QueryCommand({
  TableName: "technova-users",
  IndexName: "EmailIndex",
  KeyConditionExpression: "email = :e",
  ExpressionAttributeValues: { ":e": "ayesha@technova.example" }
}));
console.log("By email:", byEmail.Items.length, "item(s)");
```

```
node app.mjs
```

ধাপ ৪ — Console-এ যাচাই ও TTL

`DynamoDB → Tables → technova-users → Explore table items`-এ Item দেখুন। *Additional settings → Time to Live* (*TTL*)-এ `expiresAt` অ্যাট্রিবিউট দিয়ে TTL চালু করার পদ্ধতি দেখুন — সেশন/অস্থায়ী ডেটা আপনা-আপনি মুছে যায়।

৬. যাচাই

- টেবিল Status *ACTIVE*।
- `node app.mjs` তিনটি ধাপেই সফল (Put OK, Got Item, By email 1 item)।
- Console-এ Item দেখা যায়।
- ভুল Partition Key দিয়ে Get করলে খালি ফল আসে (কোনো এরর নয়)।

৭. ট্রাবলশুটিং

সমস্যা	কারণ ও সমাধান
`AccessDeniedException`	IAM পলিসিতে টেবিল ARN বা GSI ARN ভুল। Index-এর ARN-এ `/index/*` লাগে।
`ValidationException: Query key condition not supported`	Query-তে Partition Key-ই দিতে হয়; Sort Key ছাড়া Query হয় না। Scan ব্যবহার করুন (তবে বড় টেবিলে ধীর)।
Item পাওয়া যায় না, যদিও Put সফল	Sort Key-র মান হুবহু মিলছে কি না দেখুন — সময়ের ফরম্যাটে এক সেকেন্ডের পার্থক্যও মিলবে না।

৮. নিরাপত্তা বিবেচনা

DynamoDB-তে *Encryption at rest* ডিফল্ট ON — যাচাই করুন। সংবেদনশীল ফিল্ড (পাসওয়ার্ড হ্যাশ) কখনো প্লেইন টেক্সটে রাখবেন না। `Scan` অপারেশন পুরো টেবিল পড়ে — প্রোডাকশনে এড়িয়ে চলুন, Query ও GSI ব্যবহার করুন। Point-in-time recovery (PITR) প্রোডাকশন টেবিলে চালু করুন।

৯. পরিষ্কার করা

```
aws dynamodb delete-table --table-name technova-users
aws dynamodb wait table-not-exists --table-name technova-users
# প্রজেক্ট ফোল্ডারটি নিজের মেশিনে রেখে দিতে পারেন (কোডে কোনো Secret নেই কি না যাচাই করুন)
```

১০. চিন্তার প্রশ্ন

1. Partition Key খারাপভাবে বাছলে (যেমন সব Item-এ একই Key) কী সমস্যা হয়?
2. On-demand বনাম Provisioned — TechNova-র হঠাৎ ভাইরাল হওয়া অ্যাপে কোনটি নিরাপদ?
3. RDS-র বদলে DynamoDB কখন বেছে নেবেন — আর কখন নেবেন না?

ল্যাব ১৫ • সার্ভারলেস • ৭৫ মিনিট • কস্ট রিস্ক: Low — Free Tier-এর ভেতরে থাকে সাধারণত

Lab 15 — Lambda and API Gateway

১. উদ্দেশ্য

- Node.js Lambda ফাংশন বানিয়ে API Gateway-এর সঙ্গে যুক্ত করতে পারবেন।
- HTTP API দিয়ে REST এন্ডপয়েন্ট (`GET /users`, `POST /users`) তৈরি করতে পারবেন।
- Lambda-কে DynamoDB-র (Lab 14) সঙ্গে যুক্ত করে সার্ভারলেস API বানাতে পারবেন।
- CloudWatch Logs দিয়ে Lambda-র লগ দেখে ডিবাগ করতে পারবেন।

২. পূর্বশর্ত

Lab 14-এর DynamoDB ধারণা। Node.js ও JSON-এর মৌলিক জ্ঞান।

৩. প্রয়োজনীয় AWS সেবা

`Lambda`, `API Gateway (HTTP API)`, `DynamoDB`, `CloudWatch Logs`, `IAM`।

৪. আর্কিটেকচার

```
            [Client / curl] ──HTTPS──► [API Gateway: technova-users-api]
        GET /users  ──► [Lambda: technova-get-users] ──► [DynamoDB technova-users]
       POST /users ──► [Lambda: technova-create-user] ──► [DynamoDB technova-users]
                  সার্ভার নেই — রিকোয়েস্ট এলেই Lambda চলে, না এলে খরচ শূন্য।
          Lambda Execution Role: শুধু DynamoDB টেবিলে পড়া/লেখা + CloudWatch Logs।
```

৫. ধাপে ধাপে

ধাপ ১ — Lambda Execution Role (CLI)

```
aws iam create-role --role-name technova-lambda-dynamo-role \
  --assume-role-policy-document file://lambda-trust.json
# lambda-trust.json: Service lambda.amazonaws.com-কে AssumeRole
aws iam attach-role-policy --role-name technova-lambda-dynamo-role \
  --policy-arn arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole
aws iam put-role-policy --role-name technova-lambda-dynamo-role \
  --policy-name TechnovaDynamoAccess --policy-document file://dynamo-policy.json
# dynamo-policy.json: Lab 14-এর টেবিল-নির্দিষ্ট নীতি
```

ধাপ ২ — Lambda ফাংশন তৈরি (Console)

`Lambda → Create function → Author from scratch` → নাম `technova-create-user` → Runtime: Node.js 20.x → Execution role: *Use an existing role* → `technova-lambda-dynamo-role` → Create। কোড এডিটরে নিচের কোড বসান:

```
import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
import { DynamoDBDocumentClient, PutCommand } from "@aws-sdk/lib-dynamodb";

const client = DynamoDBDocumentClient.from(
  new DynamoDBClient({ region: "ap-south-1" }));

export const handler = async (event) => {
  const body = JSON.parse(event.body || "{}");
  if (!body.userId || !body.email) {
    return { statusCode: 400,
      body: JSON.stringify({ error: "userId and email are required" }) };
  }
  const item = { userId: body.userId,
    createdAt: new Date().toISOString(),
    name: body.name || "Unknown",
    email: body.email };
  await client.send(new PutCommand({
    TableName: "technova-users", Item: item }));
  return { statusCode: 201, body: JSON.stringify(item) };
};
```

Deploy করুন। একইভাবে `technova-get-users` ফাংশন বানান (Scan করে সব Item ফেরত দেয় — ল্যাবের জন্য ঠিক আছে, প্রোডাকশনে Query ব্যবহার করুন)।

ধাপ ৩ — API Gateway HTTP API (Console)

`API Gateway → Create API → HTTP API → Build` → নাম `technova-users-api` → *Add integration* → *Lambda* → `technova-create-user` → Route: `POST /users` → আরেকটি Route: `GET /users` → Integration: `technova-get-users` → Stage: `$default` (Auto-deploy ON) → Create। Invoke URL কপি করুন।

ধাপ ৪ — API পরীক্ষা (curl)

```
API=CHANGE-ME-API-INVOKE-URL
curl -s -X POST $API/users -H "Content-Type: application/json" \
  -d '{"userId":"u-002","name":"Tanvir Hasan","email":"tanvir@technova.example"}'
echo
curl -s $API/users | head -c 500
echo
```

প্রথমটিতে 201 ও তৈরি হওয়া Item, দ্বিতীয়টিতে Item-এর তালিকা আসা উচিত।

ধাপ ৫ — লগ দেখা ও ডিবাগ

`CloudWatch → Log groups → /aws/lambda/technova-create-user`-এ সাম্প্রতিক Log Stream খুলুন। কোনো এরর থাকলে এখানেই দেখা যাবে। Lambda Console-এর *Monitor* ট্যাবে Invocations, Duration ও Errors-এর গ্রাফ দেখুন।

৬. যাচাই

- `POST /users` 201 দেয়, DynamoDB-তে Item তৈরি হয়।
- `GET /users` তৈরি করা Item ফেরত দেয়।
- ভুল বডি (ইমেইল ছাড়া) 400 দেয়।
- CloudWatch Logs-এ প্রতিটি Invocation-এর লগ আছে।

৭. ট্রাবলশুটিং

সমস্যা	কারণ ও সমাধান
`500 Internal Server Error`	CloudWatch Logs-এ আসল এরর দেখুন — সাধারণত IAM অনুমতি (DynamoDB) বা টেবিলের নাম ভুল।
`403 / Missing Authentication Token`	Route-এর পাথ ভুল (যেমন `/user` বনাম `/users`) বা Stage-এর URL ভুল। Invoke URL হুবহু কপি করুন।
Lambda Timeout (3s ডিফল্ট)	Configuration → General → Timeout বাড়ান (যেমন 10s)। তবে আসল কারণ খুঁজুন — DynamoDB-র রিজিয়ন ভুল হলে Timeout হয়।
প্রথম রিকোয়েস্ট ধীর (Cold Start)	স্বাভাবিক। বারবার চালালে গরম থাকে। প্রোডাকশনে Provisioned Concurrency বিবেচনা করুন।

৮. নিরাপত্তা বিবেচনা

API Gateway-এ Throttling (হার সীমা) সেট করুন — না হলে কেউ ইচ্ছা করে লাখ লাখ রিকোয়েস্ট পাঠিয়ে বিল বাড়াতে পারে। সংবেদনশীল API-তে API Key বা JWT Authorizer (Cognito) যোগ করুন — এই ল্যাবে খোলা রাখা হয়েছে শুধু শেখার জন্য। Lambda-র পরিবেশ ভেরিয়েবলে Secret লিখবেন না — Secrets Manager ব্যবহার করুন।

৯. পরিষ্কার করা

```
aws apigatewayv2 delete-api --api-id CHANGE-ME-API-ID
aws lambda delete-function --function-name technova-create-user
aws lambda delete-function --function-name technova-get-users
aws dynamodb delete-table --table-name technova-users
aws iam delete-role-policy --role-name technova-lambda-dynamo-role \
  --policy-name TechnovaDynamoAccess
aws iam detach-role-policy --role-name technova-lambda-dynamo-role \
  --policy-arn arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole
aws iam delete-role --role-name technova-lambda-dynamo-role
```

CloudWatch Log Group নিজে থেকে মুছে যায় না — `/aws/lambda/technova-*` Log Group দুটি Console থেকে মুছে ফেলুন।

১০. চিন্তার প্রশ্ন

1. Lambda-র বিল কীভাবে হয় — আর কেন "ব্যবহার না করলে খরচ শূন্য" বলা হয়?

2. API Gateway ছাড়া Lambda-কে সরাসরি ইন্টারনেটে খোলা যায় কি? কী অসুবিধা?

3. Cold Start কী? TechNova-র লগইন API-তে Cold Start সমস্যা হলে কী করবেন?

ল্যাব ১৬ • মেসেজিং ও ইভেন্ট • ৬০ মিনিট • কস্ট রিস্ক: Low

Lab 16 — SQS, SNS and EventBridge

১. উদ্দেশ্য

- SQS Queue বানিয়ে মেসেজ পাঠানো ও পড়া করতে পারবেন।
- SNS Topic দিয়ে এক মেসেজে একাধিক গ্রাহককে (ইমেইল + SQS) জানাতে পারবেন।
- EventBridge Rule দিয়ে সময়সূচি (schedule) অনুযায়ী Lambda চালাতে পারবেন।
- Queue, Topic ও Event Bus-এর ব্যবহারের পার্থক্য ব্যাখ্যা করতে পারবেন।

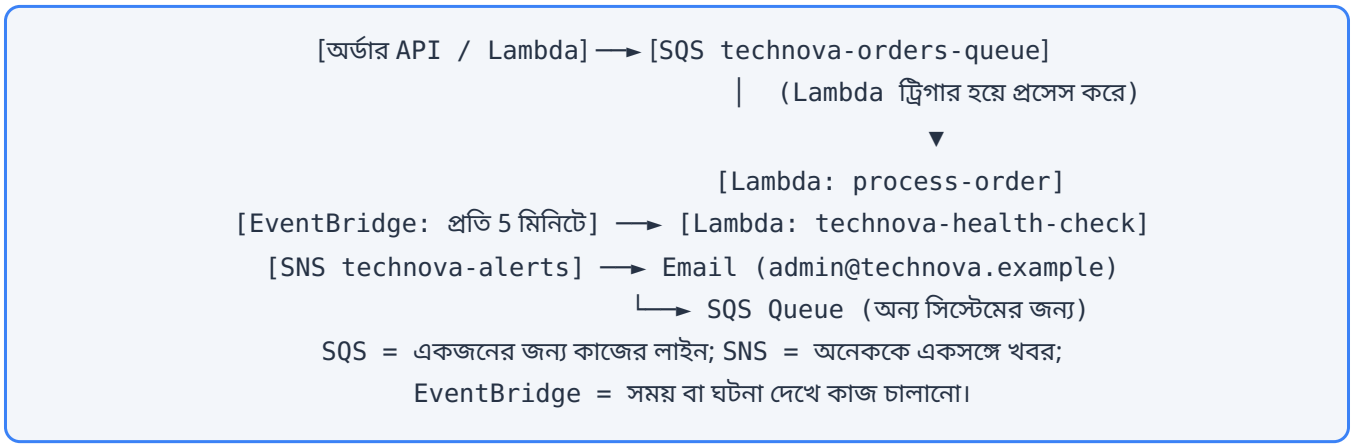
২. পূর্বশর্ত

Lab 15-এর Lambda ধারণা। একটি ইমেইল ঠিকানা (SNS সাবস্ক্রিপশন যাচাইয়ের জন্য)।

৩. প্রয়োজনীয় AWS সেবা

`SQS`, `SNS`, `EventBridge`, `Lambda`।

৪. আর্কিটেকচার



৫. ধাপে ধাপে

ধাপ ১ — SQS Queue তৈরি ও মেসেজ পাঠানো

```
aws sqs create-queue --queue-name technova-orders-queue \
  --attributes VisibilityTimeout=60,MessageRetentionPeriod=86400
QUEUE_URL=$(aws sqs get-queue-url --queue-name technova-orders-queue \
  --query 'QueueUrl' --output text)
echo $QUEUE_URL
aws sqs send-message --queue-url $QUEUE_URL \
  --message-body '{"orderId":"ord-1001","userId":"u-001","amount":1500}'
aws sqs receive-message --queue-url $QUEUE_URL \
  --max-number-of-messages 5 --wait-time-seconds 5
```

মেসেজটি পাওয়ার পর তার `ReceiptHandle` দিয়ে মুছুন — না মুছলে Visibility Timeout শেষে আবার দেখা যাবে:

```
aws sqs delete-message --queue-url $QUEUE_URL \
  --receipt-handle CHANGE-ME-RECEIPT-HANDLE
aws sqs get-queue-attributes --queue-url $QUEUE_URL \
  --attribute-names ApproximateNumberOfMessages
```

ধাপ ২ — Dead Letter Queue (DLQ)

Console-এ আরেকটি Queue বানান: `technova-orders-dlq`। তারপর মূল Queue-র *Dead-letter queue* সেটিংসে DLQ যুক্ত করুন — Maximum receives: 3। বারবার ব্যর্থ হওয়া মেসেজ DLQ-তে যাবে, মূল Queue আটকে থাকবে না।

ধাপ ৩ — SNS Topic ও সাবস্ক্রিপশন

```
TOPIC_ARN=$(aws sns create-topic --name technova-alerts \
  --query 'TopicArn' --output text)
echo $TOPIC_ARN
aws sns subscribe --topic-arn $TOPIC_ARN --protocol email \
  --notification-endpoint admin@technova.example
# ইমেইলে আসা Confirm লিংকে ক্লিক করুন — তারপর:
aws sns publish --topic-arn $TOPIC_ARN \
  --subject "TechNova Test Alert" \
  --message "Order ord-1001 processed successfully."
aws sns subscribe --topic-arn $TOPIC_ARN --protocol sqs \
  --notification-endpoint CHANGE-ME-QUEUE-ARN
```

> **ইমেইল যাচাই:** SNS ইমেইল সাবস্ক্রিপশন Confirm না করা পর্যন্ত মেইল যাবে না। স্প্যাম ফোল্ডারও দেখুন। `admin@technova.example` একটি উদাহরণ ঠিকানা — নিজের আসল ইমেইল ব্যবহার করুন।

ধাপ ৪ — EventBridge সময়সূচি দিয়ে Lambda

Console-এ একটি সাধারণ Lambda বানান (`technova-health-check`) — শুধু `console.log("Health check OK")` করে। তারপর:

```
aws events put-rule --name technova-health-check-5min \
  --schedule-expression "rate(5 minutes)" \
  --description "TechNova health check every 5 minutes"
aws events put-targets --rule technova-health-check-5min \
  --targets '[{"Id":"1","Arn":"CHANGE-ME-LAMBDA-ARN"}]'
aws lambda add-permission --function-name technova-health-check \
  --statement-id technova-events-permission --action lambda:InvokeFunction \
  --principal events.amazonaws.com \
  --source-arn CHANGE-ME-RULE-ARN
```

৫ মিনিট অপেক্ষা করে CloudWatch Logs-এ Lambda-র লগ দেখুন — সময়মতো চলেছে কি না।

ধাপ ৫ — SQS থেকে Lambda ট্রিগার (ধারণা + সেটআপ)

Console-এ Lambda `technova-health-check`-এর *Add trigger* → *SQS* → Queue: `technova-orders-queue` → Batch size: 5 → Add। এরপর Queue-তে মেসেজ পাঠালে Lambda আপনা-আপনি চলবে — মেসেজ প্রসেস করে মুছে ফেলবে।

৬. যাচাই

- SQS-এ মেসেজ পাঠানো, পড়া ও মোছা — তিনটিই সফল।
- SNS Publish করলে ইমেইল আসে (Confirm করার পর)।
- EventBridge Rule *Enabled*, Lambda ৫ মিনিট পরপর চলে (Logs-এ প্রমাণ)।
- DLQ সেটিংস মূল Queue-তে দেখা যায়।

৭. ট্রাবলশুটিং

সমস্যা	কারণ ও সমাধান
মেসেজ বারবার ফিরে আসে	Receive করার পর Delete করেননি, অথবা Lambda এরর করে মেসেজ মুছছে না। Visibility Timeout ও DLQ দেখুন।
SNS ইমেইল আসে না	সাবস্ক্রিপশন PendingConfirmation অবস্থায় আছে। ইমেইলের Confirm লিংকে ক্লিক করুন।
EventBridge Lambda চালায় না	`add-permission` দেওয়া হয়নি, বা Rule Disabled। Rule-র ARN ও Lambda Permission দেখুন।

৮. নিরাপত্তা বিবেচনা

SQS Queue Policy-তে শুধু নির্দিষ্ট Role/অ্যাকাউন্টকে Send/Receive দিন — `*` Principal দেবেন না। SNS Topic-এ সংবেদনশীল ডেটা পাঠাবেন না — ইমেইল এনক্রিপ্টেড নয়। EventBridge Rule-র Target-এ শুধু প্রয়োজনীয় Lambda-ই রাখুন।

৯. পরিষ্কার করা

```
aws events remove-targets --rule technova-health-check-5min --ids "1"
aws events delete-rule --name technova-health-check-5min
aws lambda delete-function --function-name technova-health-check
aws sns unsubscribe --subscription-arn CHANGE-ME-SUBSCRIPTION-ARN
aws sns delete-topic --topic-arn $TOPIC_ARN
aws sqs delete-queue --queue-url $QUEUE_URL
aws sqs delete-queue --queue-url CHANGE-ME-DLQ-URL
```

SQS Queue মুছতে কয়েক মিনিট লাগতে পারে। Console-এ তিনটি সেবার তালিকাই খালি দেখে নিন।

১০. চিন্তার প্রশ্ন

1. অর্ডার প্রসেসিংয়ে সরাসরি API কল না করে SQS ব্যবহার করলে কী লাভ — পেমেন্ট সার্ভার ডাউন থাকলে কী হবে?
2. SNS বনাম EventBridge — কখন কোনটি? একটি উদাহরণ দিয়ে বোঝান।
3. Visibility Timeout খুব ছোট বা খুব বড় হলে কী সমস্যা হয়?

ল্যাব ১৭ • কন্টেইনার • ৯০ মিনিট • কস্ট রিস্ক: Medium — Fargate টাস্ক চালু থাকলে চার্জ; শেষে Service 0 করুন

Lab 17 — Docker Image, ECR and ECS Fargate

১. উদ্দেশ্য

- Node.js অ্যাপের Dockerfile লিখে লোকালে ইমেজ বানাতে ও চালাতে পারবেন।
- ECR রিপোজিটরিতে ইমেজ পুশ করতে পারবেন।
- ECS Fargate-এ সার্ভার ছাড়াই কন্টেইনার চালাতে পারবেন।
- ALB-র পেছনে Fargate Service চালিয়ে পাবলিক URL-এ অ্যাপ দেখতে পারবেন।

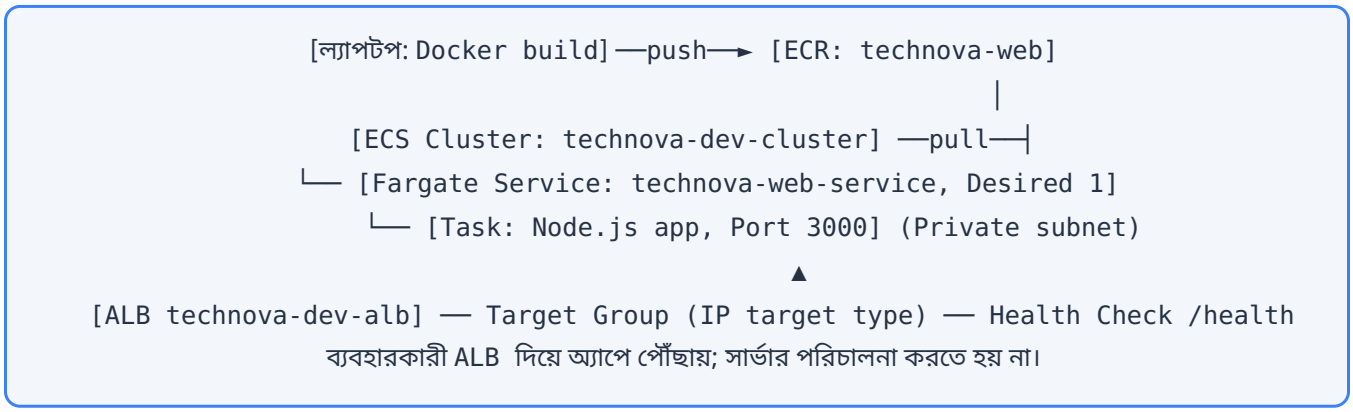
২. পূর্বশর্ত

Docker ইনস্টল করা। Lab 9-এর ALB ধারণা। Lab 4-এর VPC ও সাবনেট। Node.js-এর মৌলিক জ্ঞান।

৩. প্রয়োজনীয় AWS সেবা

`ECR`, `ECS (Fargate)`, `ALB`, `CloudWatch Logs`, `IAM (Task Execution Role)`।

৪. আর্কিটেকচার



৫. ধাপে ধাপে

ধাপ ১ — Node.js অ্যাপ ও Dockerfile

```
mkdir technova-web && cd technova-web
npm init -y
npm install express
```

`server.js`:

```
const express = require("express");
const app = express();
app.get("/", (req, res) =>
  res.send("Hello from TechNova on Fargate!"));
```

```
app.get("/health", (req, res) =>
  res.json({ status: "ok" }));
app.listen(3000, () =>
  console.log("Listening on 3000"));
```

`Dockerfile`:

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
USER node
CMD ["node", "server.js"]
```

```
docker build -t technova-web:local .
docker run -d -p 3000:3000 --name technova-test technova-web:local
curl http://localhost:3000/health
docker stop technova-test && docker rm technova-test
```

ধাপ ২ — ECR রিপোজিটরি ও পুশ

```
aws ecr create-repository --repository-name technova-web \
  --image-scanning-configuration scanOnPush=true
aws ecr get-login-password --region ap-south-1 | \
  docker login --username AWS --password-stdin \
  111122223333.dkr.ecr.ap-south-1.amazonaws.com
docker tag technova-web:local \
  111122223333.dkr.ecr.ap-south-1.amazonaws.com/technova-web:v1
docker push 111122223333.dkr.ecr.ap-south-1.amazonaws.com/technova-web:v1
aws ecr describe-images --repository-name technova-web
```

> **অ্যাকাউন্ট ID:** উপরের `111122223333` কাল্পনিক উদাহরণ — নিজের আসল অ্যাকাউন্ট ID ব্যবহার করুন (`aws sts get-caller-identity` দিয়ে দেখুন)।

ধাপ ৩ — ECS Cluster ও Task Definition (Console)

`ECS → Clusters → Create cluster` → নাম `technova-dev-cluster` → Infrastructure: AWS Fargate → Create। তারপর `Task Definitions → Create new task definition` → নাম `technova-web-task` → Launch type: Fargate → CPU 0.25 vCPU, Memory 0.5 GB → Container: নাম `web`, Image: ECR-র `:v1` URI, Port 3000 → Log: CloudWatch (`/ecs/technova-web`) → Create।

ধাপ ৪ — Fargate Service + ALB

Cluster-এ *Create Service* → Task Definition: `technova-web-task` → Desired tasks: 1 → VPC, প্রাইভেট সাবনেট দুটি → Security Group: `technova-sg-app` (3000 শুধু ALB SG থেকে) → Load balancer: ALB `technova-dev-alb`, Target Group (IP type, Port 3000, Health path `/health`) → Create। ২–৩ মিনিট পর Task *RUNNING* হবে।

ধাপ ৫ — পরীক্ষা ও লগ

```
curl -s http://CHANGE-ME-ALB-DNS/
aws ecs describe-services --cluster technova-dev-cluster \
  --services technova-web-service \
  --query 'services[].{Desired:desiredCount,Running:runningCount}'
aws logs tail /ecs/technova-web --follow
```

৬. যাচাই

- লোকালে Docker-এ `/health` 200 দেয়।
- ECR-এ `:v1` ইমেজ আছে, স্ক্যান সম্পন্ন।
- ECS Task RUNNING, Target Group-এ healthy।
- ALB DNS-এ "Hello from TechNova on Fargate!" দেখা যায়।
- CloudWatch Logs-এ অ্যাপের লগ আছে।

৭. ট্রাবলশুটিং

সমস্যা	কারণ ও সমাধান
Task বারবার PENDING → STOPPED	ECS Console-এ Stopped Task-এর *Details* → *Stopped reason* দেখুন। সাধারণত: ECR থেকে Pull ব্যর্থ (Execution Role-এ অনুমতি নেই), বা পোর্ট ভুল।
`CannotPullContainerError`	Task Execution Role-এ ECR Read অনুমতি আছে কি না, প্রাইভেট সাবনেটে NAT বা ECR Endpoint আছে কি না দেখুন।
Target unhealthy	`/health` পাথ ঠিক আছে কি না, SG-তে ALB থেকে 3000 খোলা আছে কি না দেখুন।
Docker push ব্যর্থ — `denied`	`docker login` আবার করুন (টোকেন ১২ ঘণ্টা মেয়াদি)। রিপোজিটরি নাম ও রিজিয়ন মিলিয়ে দেখুন।

৮. নিরাপত্তা বিবেচনা

ইমেজে Root ব্যবহারকারী দিয়ে অ্যাপ চালাবেন না — `USER node` ব্যবহার করুন (উপরের Dockerfile-এ আছে)। ECR-এ *Scan on push* চালু রাখুন — পরিচিত দুর্বলতা ধরা পড়ে। Secret (DB পাসওয়ার্ড) ইমেজে বা Task Definition-এর পরিবেশ ভেরিয়েবলে প্লেইন টেক্সটে লিখবেন না — Secrets Manager থেকে নিন।

৯. পরিষ্কার করা

```
aws ecs update-service --cluster technova-dev-cluster \
  --service technova-web-service --desired-count 0
aws ecs delete-service --cluster technova-dev-cluster \
  --service technova-web-service --force
aws ecs delete-cluster --cluster technova-dev-cluster
aws elbv2 delete-load-balancer --load-balancer-arn CHANGE-ME-ALB-ARN
aws ecr batch-delete-image --repository-name technova-web \
```

```
  --image-ids imageTag=v1
aws ecr delete-repository --repository-name technova-web --force
```

Service 0 করার পর Task থামা পর্যন্ত অপেক্ষা করুন, তারপর মুছুন। ALB-র Target Group-ও মুছে ফেলুন।

১০. চিন্তার প্রশ্ন

1. Fargate বনাম EC2-তে ECS চালানো — কোনটি কখন সুবিধাজনক?
2. ইমেজ ট্যাগ `:latest` ব্যবহার করলে ডেপ্লয়মেন্টে কী সমস্যা হয়?
3. Task Definition-এর নতুন Revision দিয়ে রোলিং ডেপ্লয়মেন্ট কীভাবে হয়?

ল্যাব ১৮ • কুবারনেটিস • ১২০ মিনিট • কস্ট রিস্ক: High — EKS ক্লাস্টার ও নোড প্রতি ঘণ্টায় চার্জ; ল্যাব শেষেই মুছুন

Lab 18 — EKS Fundamentals and Kubernetes Deployment

১. উদ্দেশ্য

- EKS ক্লাস্টার তৈরি করে `kubectl` দিয়ে সংযোগ করতে পারবেন।
- Deployment, Service ও Ingress-এর ধারণা প্রয়োগ করতে পারবেন।
- Lab 17-এর কন্টেইনার অ্যাপ Kubernetes-এ চালাতে পারবেন।
- Pod লগ ও `kubectl describe` দিয়ে সমস্যা নির্ণয় করতে পারবেন।

২. পূর্বশর্ত

Lab 17 সম্পন্ন (Docker + ECR)। `kubectl` ও `eksctl` (ঐচ্ছিক, তবে সুবিধাজনক) ইনস্টল করা।

৩. প্রয়োজনীয় AWS সেবা

`EKS`, `EC2 (Managed Node Group)`, `ECR`, `IAM (EKS Roles)`, `CloudWatch (Container Insights)`।

৪. আর্কিটেকচার

```
[kubectl — আপনার ল্যাপটপ]
        |
        ▼
[EKS Control Plane — AWS পরিচালিত] (ap-south-1)
        |
        ▼
[Managed Node Group: 2 × t3.small] (Private subnets)
    ├── Pod: technova-web (Deployment, replicas 2)
    └── Service: LoadBalancer → AWS NLB/ALB → Internet
        Namespace: technova-dev — সব রিসোর্স এখানে।
```

খরচ সতর্কতা: EKS Control Plane ও Node Group — দুটোতেই প্রতি ঘণ্টায় চার্জ হয়। এই ল্যাব এক বসায় শেষ করুন — ক্লাস্টার বানিয়ে রেখে ঘুমাবেন না। দাম AWS Pricing Calculator-এ যাচাই করুন। শেখার জন্য ছোট Node (`t3.small`, ২টি) ব্যবহার করুন।

৫. ধাপে ধাপে

ধাপ ১ — ক্লাস্টার তৈরি (eksctl — সবচেয়ে সহজ পথ)

```
eksctl create cluster --name technova-dev-eks \
  --region ap-south-1 --version 1.31 \
  --nodegroup-name technova-nodes --node-type t3.small \
  --nodes 2 --nodes-min 1 --nodes-max 3 \
  --managed --vpc-id CHANGE-ME-VPC-ID \
  --subnet-ids CHANGE-ME-PRIV-A,CHANGE-ME-PRIV-B
# তৈরি হতে ১৫–২০ মিনিট লাগতে পারে — অপেক্ষা করুন।
kubectl get nodes
kubectl cluster-info
```

Console পথ: `EKS → Create cluster` → নাম, IAM Role (`AmazonEKSClusterPolicy`), VPC ও সাবনেট → তারপর Node Group যোগ করুন। eksctl এই সব ধাপ এক কমান্ডে করে দেয় — তাই ল্যাবে eksctl ব্যবহার করা হয়েছে।

ধাপ ২ — Namespace ও Deployment

```
kubectl create namespace technova-dev
kubectl config set-context --current --namespace=technova-dev
```

`deployment.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: technova-web
  namespace: technova-dev
spec:
  replicas: 2
  selector:
    matchLabels: { app: technova-web }
  template:
    metadata:
      labels: { app: technova-web }
    spec:
      containers:
        - name: web
          image: 111122223333.dkr.ecr.ap-south-1.amazonaws.com/technova-web:v1
          ports: [{ containerPort: 3000 }]
          resources:
            requests: { cpu: "100m", memory: "128Mi" }
            limits: { cpu: "250m", memory: "256Mi" }
          livenessProbe:
            httpGet: { path: /health, port: 3000 }
            initialDelaySeconds: 10
            periodSeconds: 10
```

```
kubectl apply -f deployment.yaml
kubectl get deployments -n technova-dev
kubectl get pods -n technova-dev -w
```

ধাপ ৩ — Service দিয়ে প্রকাশ করা

`service.yaml`:

```
apiVersion: v1
kind: Service
metadata:
  name: technova-web-svc
  namespace: technova-dev
spec:
  type: LoadBalancer
  selector: { app: technova-web }
  ports:
    - port: 80
      targetPort: 3000
```

```
kubectl apply -f service.yaml
kubectl get svc -n technova-dev
# EXTERNAL-IP আসা পর্যন্ত অপেক্ষা করুন (২–৩ মিনিট):
curl http://CHANGE-ME-LOADBALANCER-DNS/
```

ধাপ ৪ — স্কেল ও রোলিং আপডেট

```
kubectl scale deployment technova-web --replicas=3 -n technova-dev
kubectl get pods -n technova-dev
# নতুন ইমেজ (v2) দিয়ে রোলিং আপডেট:
kubectl set image deployment/technova-web \
  web=111122223333.dkr.ecr.ap-south-1.amazonaws.com/technova-web:v2 \
  -n technova-dev
kubectl rollout status deployment/technova-web -n technova-dev
```

ধাপ ৫ — লগ ও ডিবাগ

```
kubectl logs -l app=technova-web -n technova-dev --tail=20
kubectl describe pod CHANGE-ME-POD-NAME -n technova-dev
kubectl get events -n technova-dev --sort-by=.lastTimestamp | tail -20
```

৬. যাচাই

- `kubectl get nodes`-এ 2টি Node Ready।
- Deployment-এ 2টি Pod Running।
- LoadBalancer DNS-এ অ্যাপ খোলে।
- Scale করে 3টি Pod হয়; রোলিং আপডেটে ডাউনটাইম ছাড়াই নতুন ইমেজ আসে।

৭. ট্রাবলশুটিং

সমস্যা	কারণ ও সমাধান
Pod `ImagePullBackOff`	

	ECR URI ভুল, বা Node-র IAM Role-এ ECR Read অনুমতি নেই। `kubectl describe pod` -এ আসল এরর দেখুন।
Pod `Pending`	Node-এ CPU/Memory কম, বা সাবনেটে IP শেষ। `kubectl describe pod` -এ Events দেখুন।
Service-র EXTERNAL-IP আসে না	সাবনেটে পাবলিক IP ট্যাগ (`kubernetes.io/role/elb`) নেই, বা Load Balancer তৈরিতে সময় লাগছে। ৫ মিনিট অপেক্ষা করুন।
`kubectl` সংযোগ হয় না	`aws eks update-kubeconfig --name technova-dev-eks --region ap-south-1` আবার চালান।

৮. নিরাপত্তা বিবেচনা

EKS-এ IAM Roles for Service Accounts (IRSA) ব্যবহার করুন — Pod-কে Node-র IAM Role-এর অনুমতি দেবেন না। Kubernetes Secret-এ সংবেদনশীল ডেটা প্লেইন টেক্সটে থাকে (base64 শুধু এনকোডিং, এনক্রিপশন নয়) — Secrets Manager বা KMS এনক্রিপশন ব্যবহার করুন। RBAC দিয়ে কে কী করতে পারে তা সীমিত করুন।

৯. পরিষ্কার করা

> **EKS মুছতে ভুলবেন না — Control Plane ও Node — দুটোতেই চার্জ হয়।**

```
kubectl delete -f service.yaml -f deployment.yaml
eksctl delete cluster --name technova-dev-eks --region ap-south-1
# eksctl না থাকলে Console: EKS → Node Groups মুছুন → Cluster মুছুন
# তারপর Load Balancer মুছে গেছে কি না যাচাই করুন:
aws elbv2 describe-load-balancers --query 'LoadBalancers[].LoadBalancerName'
aws ecr batch-delete-image --repository-name technova-web --image-ids imageTag=v1
aws ecr delete-repository --repository-name technova-web --force
```

ক্লাস্টার মুছতে ১০–১৫ মিনিট লাগতে পারে। Console-এ EKS তালিকা খালি ও EC2 তালিকায় Node ইনস্ট্যান্স নেই — নিশ্চিত হয়ে নিন।

১০. চিন্তার প্রশ্ন

1. Deployment বনাম StatefulSet — কখন কোনটি? TechNova-র ডাটাবেস কি Deployment-এ চালানো উচিত?
2. Liveness Probe ব্যর্থ হলে Kubernetes কী করে — আর Readiness Probe ব্যর্থ হলে কী করে?
3. ECS Fargate বনাম EKS — ছোট টিমের জন্য কোনটি সহজ, আর বড় সিস্টেমে কোনটি শক্তিশালী?

ল্যাব ১৯ • Infrastructure as Code • ৯০ মিনিট • কস্ট রিস্ক: Low — Terraform দিয়ে বানানো রিসোর্স destroy করুন

Lab 19 — Terraform and CloudFormation Provisioning

১. উদ্দেশ্য

- Terraform দিয়ে VPC, সাবনেট, Security Group ও EC2 — পুরো অবকাঠামো কোড হিসেবে বানাতে পারবেন।
- `plan` ও `apply`-এর পার্থক্য বুঝে নিরাপদে পরিবর্তন করতে পারবেন।
- CloudFormation দিয়ে একই কাজের দ্বিতীয় পদ্ধতি দেখবেন।
- State ফাইলের গুরুত্ব ও সুরক্ষা বুঝবেন।

২. পূর্বশর্ত

Terraform ≥ 1.5 ইনস্টল করা। Lab 4–7-এর হাতে-বানানো অবকাঠামোর অভিজ্ঞতা (তাহলে কোড কী করছে বুঝবেন)।

৩. প্রয়োজনীয় AWS সেবা

`VPC`, `EC2`, `S3 (State Backend)`, `CloudFormation`।

৪. আর্কিটেকচার

```
[Terraform Code: main.tf, variables.tf, outputs.tf]
            |  terraform plan → কী বদলাবে দেখায়
               |  terraform apply → বানায়
                              ▼
   [AWS: VPC 10.2.0.0/16 + 2 Subnets + SG + EC2 t2.micro]
State: terraform.tfstate (লোকালে; প্রোডাকশনে S3 + DynamoDB Lock)
     CloudFormation: একই জিনিস YAML টেমপ্লেটে (template.yaml)
```

৫. ধাপে ধাপে

ধাপ ১ — প্রজেক্ট কাঠামো

```
mkdir technova-terraform && cd technova-terraform
cat > variables.tf <<'EOF'
variable "region" { default = "ap-south-1" }
variable "vpc_cidr" { default = "10.2.0.0/16" }
variable "project" { default = "technova" }
EOF
cat > outputs.tf <<'EOF'
output "vpc_id" { value = aws_vpc.main.id }
output "public_subnet_id" { value = aws_subnet.public_a.id }
```

```
output "instance_public_ip" { value = aws_instance.web.public_ip }
EOF
```

ধাপ ২ — মূল কনফিগারেশন (`main.tf`)

```
terraform {
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
  required_version = ">= 1.5"
}
provider "aws" { region = var.region }

resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_hostnames = true
  tags = { Name = "technova-tf-vpc", Project = "TechNova" }
}
resource "aws_subnet" "public_a" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.2.1.0/24"
  availability_zone       = "ap-south-1a"
  map_public_ip_on_launch = true
  tags = { Name = "technova-tf-public-a" }
}
resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id
  tags   = { Name = "technova-tf-igw" }
}
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main.id
  }
}
resource "aws_route_table_association" "public_a" {
  subnet_id      = aws_subnet.public_a.id
  route_table_id = aws_route_table.public.id
}
resource "aws_security_group" "web" {
  name   = "technova-tf-sg-web"
  vpc_id = aws_vpc.main.id
  ingress {
    from_port = 80; to_port = 80; protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    from_port = 22; to_port = 22; protocol = "tcp"
    cidr_blocks = ["203.0.113.10/32"]
  }
  egress {
    from_port = 0; to_port = 0; protocol = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
resource "aws_instance" "web" {
  ami                    = "CHANGE-ME-AMAZON-LINUX-2023-AMI-ID"
```

```
  instance_type          = "t2.micro"
  subnet_id              = aws_subnet.public_a.id
  vpc_security_group_ids = [aws_security_group.web.id]
  key_name               = "technova-dev-key"
  user_data              = <<-EOT
    #!/bin/bash
    dnf update -y
    dnf install -y httpd
    systemctl start httpd
    systemctl enable httpd
    echo "<h1>Hello from Terraform!</h1>" > /var/www/html/index.html
  EOT
  tags = { Name = "technova-tf-web" }
}
```

> **নিজের IP দিন:** SSH নিয়মে `203.0.113.10/32`-এর বদলে নিজের আসল পাবলিক IP দিন। AMI ID নিজের রিজিয়নের Amazon Linux 2023-এরটি বসান — `aws ec2 describe-images` দিয়ে খুঁজুন।

ধাপ ৩ — Init, Plan, Apply

```
terraform init
terraform fmt
terraform validate
terraform plan -out=tfplan
# plan আউটপুট পড়ুন: কয়টি রিসোর্স বানাবে? কোনো মুছে ফেলা আছে কি?
terraform apply tfplan
terraform output
```

`instance_public_ip` আউটপুটে পাওয়া IP ব্রাউজারে খুলুন — "Hello from Terraform!" দেখা উচিত।

ধাপ ৪ — পরিবর্তন ও পুনরায় Apply

`main.tf`-এ EC2-র ট্যাগ বদলে `terraform plan` চালান — Terraform শুধু ট্যাগ বদলাবে, ইনস্ট্যান্স নতুন করে বানাবে না। তারপর Instance Type `t2.micro` থেকে `t3.micro`-তে বদলে plan দেখুন — কী বদলায়, বুঝুন। আসল পরিবর্তন না করে ফাইলটি আগের অবস্থায় ফিরিয়ে আনুন।

ধাপ ৫ — CloudFormation দিয়ে একই কাজ (তুলনা)

একটি ছোট CloudFormation টেমপ্লেট (`template.yaml`) লিখুন — শুধু একটি S3 বাকেট:

```
AWSTemplateFormatVersion: "2010-09-09"
Description: TechNova demo bucket via CloudFormation
Resources:
  TechNovaBucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: technova-cfn-demo-CHANGE-ME
      VersioningConfiguration:
        Status: Enabled
      PublicAccessBlockConfiguration:
        BlockPublicAcls: true
        BlockPublicPolicy: true
        IgnorePublicAcls: true
        RestrictPublicBuckets: true
```

```
Outputs:
  BucketName:
    Value: !Ref TechNovaBucket
```

```
aws cloudformation create-stack --stack-name technova-cfn-demo \
  --template-body file://template.yaml
aws cloudformation describe-stacks --stack-name technova-cfn-demo \
  --query 'Stacks[].StackStatus'
aws cloudformation delete-stack --stack-name technova-cfn-demo
```

৬. যাচাই

- `terraform plan`-এ ৪টি রিসোর্স বানানোর পরিকল্পনা দেখায়, কোনো ধ্বংস নয়।
- `terraform apply`-র পর ব্রাউজারে "Hello from Terraform!" দেখা যায়।
- `terraform state list`-এ সব রিসোর্স আছে।
- CloudFormation Stack *CREATE_COMPLETE* হয়, তারপর মুছে ফেলা যায়।

৭. ট্রাবলশুটিং

সমস্যা	কারণ ও সমাধান
`Error acquiring the state lock`	আগের Terraform প্রসেস আটকে আছে। `terraform force-unlock` (Lock ID দিয়ে) — তবে নিশ্চিত হয়ে নিন অন্য কেউ চালাচ্ছে না।
`InvalidAMIID.NotFound`	AMI ID ভুল রিজিয়নের। `ap-south-1`-এর AMI ID খুঁজে বসান।
Apply-র পরও ওয়েবসাইট খোলে না	Lab 7-এর মতো: User Data লগ `/var/log/cloud-init-output.log` দেখুন, SG-তে 80 খোলা আছে কি না দেখুন।
CloudFormation `ROLLBACK_COMPLETE`	বাকেট নাম ইতিমধ্যে নেওয়া। নাম বদলে আবার চেষ্টা করুন। Events ট্যাবে আসল এরর দেখুন।

৮. নিরাপত্তা বিবেচনা

Terraform State ফাইলে সংবেদনশীল তথ্য (যেমন পাসওয়ার্ড, IP) থাকতে পারে — কখনো Git-এ তুলবেন না। প্রোডাকশনে State S3 বাকেটে রাখুন (Versioning + Encryption ON) ও DynamoDB দিয়ে Lock করুন। `terraform plan` না দেখে কখনো `apply` করবেন না — বিশেষ করে প্রোডাকশনে।

৯. পরিষ্কার করা

```
terraform destroy
# নিশ্চিত হয়ে "yes" লিখুন। তারপর:
aws cloudformation delete-stack --stack-name technova-cfn-demo
aws ec2 describe-instances --filters Name=tag:Name,Values=technova-tf-web \
  --query 'Reservations[].Instances[].State.Name'
aws ec2 describe-vpcs --filters Name=tag:Name,Values=technova-tf-vpc
```

দুটি কমান্ডই খালি ফল দেওয়া উচিত (কোনো চলমান ইনস্ট্যান্স বা VPC নেই)। `.terraform/` ও `terraform.tfstate` ফাইল নিজের মেশিনে রেখে দিতে পারেন — তবে State ফাইলে কোনো Secret আছে কি না যাচাই করুন।

১০. চিন্তার প্রশ্ন

1. হাতে Console-এ বানানো আর Terraform-এ বানানো — দশটি সার্ভার বানাতে হলে কোনটি নিরাপদ, কেন?
2. State ফাইল হারিয়ে গেলে কী হয়? Terraform তখন কী করবে?
3. Terraform বনাম CloudFormation — TechNova-র টিমে কোনটি বেছে নেবেন, কেন?

ল্যাব ২০ • CI/CD • ৯০ মিনিট • কস্ট রিস্ক: Low — GitHub Actions ফ্রি টিয়ার; AWS-এ শুধু S3/ECR খরচ

Lab 20 — GitHub Actions to AWS Deployment

১. উদ্দেশ্য

- GitHub Actions Workflow লিখে Push করলেই স্বয়ংক্রিয় ডেপ্লয়মেন্ট চালাতে পারবেন।
- OIDC দিয়ে Secret Key ছাড়াই GitHub থেকে AWS-এ লগইন করতে পারবেন।
- Docker ইমেজ বানিয়ে ECR-এ পুশ ও ECS Service আপডেট করতে পারবেন।
- ডেপ্লয়মেন্ট ব্যর্থ হলে লগ দেখে ঠিক করতে পারবেন।

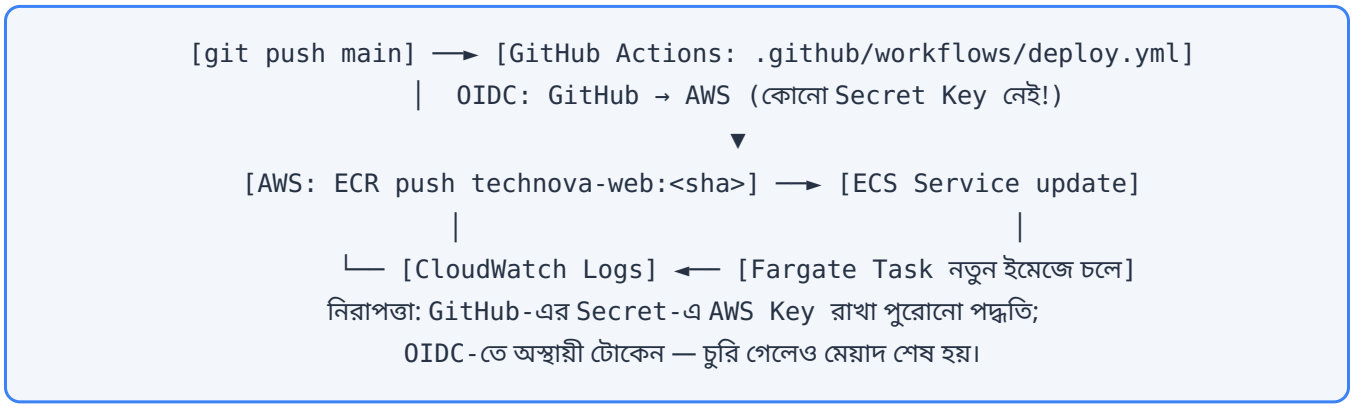
২. পূর্বশর্ত

GitHub অ্যাকাউন্ট ও একটি রিপোজিটরি (Lab 17-এর `technova-web` কোডসহ)। Lab 17-এর ECR ও ECS ধারণা।

৩. প্রয়োজনীয় AWS সেবা

`IAM (OIDC Identity Provider)`, `ECR`, `ECS`, `GitHub Actions`।

৪. আর্কিটেকচার



৫. ধাপে ধাপে

ধাপ ১ — IAM OIDC Provider ও রোল (Console)

`IAM → Identity providers → Add provider → OpenID Connect` → Provider URL: `https://token.actions.githubusercontent.com` → Audience: `sts.amazonaws.com` → Add। তারপর `Roles → Create role → Web identity` → Identity provider: token.actions.githubusercontent.com → Audience: sts.amazonaws.com → GitHub organization: নিজের GitHub ব্যবহারকারী নাম → Repository: `technova-web` → Branch: main → Permissions: `AmazonEC2ContainerRegistryPowerUser` + ECS Deploy নীতি → নাম `technova-github-deploy-role` → Create।

Trust Policy-তে শর্তটি যাচাই করুন — শুধু `repo:YOUR-USERNAME/technova-web:ref:refs/heads/main` থেকেই AssumeRole করা যাবে:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"Federated": "arn:aws:iam::111122223333:oidc-provider/
token.actions.githubusercontent.com"},
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
        "token.actions.githubusercontent.com:sub": "repo:CHANGE-ME-GITHUB-USER/technova-
web:ref:refs/heads/main"
      }
    }
  }]
}
```

ধাপ ২ — Workflow ফাইল লেখা

রিপোজিটরিতে `.github/workflows/deploy.yml` তৈরি করুন:

```
name: Deploy to AWS ECS
on:
  push:
    branches: [main]
permissions:
  id-token: write
  contents: read
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::111122223333:role/technova-github-deploy-role
          aws-region: ap-south-1
      - name: Login to Amazon ECR
        id: login-ecr
        uses: aws-actions/amazon-ecr-login@v2
      - name: Build and push image
        env:
          ECR_REGISTRY: ${{ steps.login-ecr.outputs.registry }}
          IMAGE_TAG: ${{ github.sha }}
        run: |
          docker build -t $ECR_REGISTRY/technova-web:$IMAGE_TAG .
          docker push $ECR_REGISTRY/technova-web:$IMAGE_TAG
      - name: Deploy to ECS
        run: |
          aws ecs update-service --cluster technova-dev-cluster \
            --service technova-web-service --force-new-deployment
```

> **খেয়াল করুন:** এই Workflow-এ ECR-এ নতুন ইমেজ পুশ হয়, কিন্তু ECS Task Definition-এর ইমেজ ট্যাগ আপডেট হয় না — `--force-new-deployment` পুরোনো ট্যাগেই নতুন করে চালায়। সম্পূর্ণ সঠিক পদ্ধতিতে Task Definition JSON রেন্ডার করে নতুন Revision রেজিস্টার করতে হয় (AWS-এর `amazon-ecs-render-task-definition` অ্যাকশন দেখুন) — Project 02-এ সেটাই করবেন। এই ল্যাবে CI/CD-র প্রবাহ বোঝাই মূল লক্ষ্য।

ধাপ ৩ — Push করে Workflow চালানো

```
git add .github/workflows/deploy.yml
git commit -m "Add GitHub Actions deploy workflow"
git push origin main
# GitHub → Repository → Actions ট্যাবে Workflow চলছে দেখুন।
```

ধাপ ৪ — ব্যর্থতা দেখা ও ঠিক করা

ইচ্ছা করে একটি ভুল করুন (যেমন Dockerfile-এ ভুল পাথ) — Workflow ব্যর্থ হবে। Actions ট্যাবে লগ খুলে কোন ধাপে ব্যর্থ হলো দেখুন, ঠিক করুন, আবার Push করুন — সবুজ টিক আসা উচিত। এই চক্রটাই CI/CD-র দৈনন্দিন কাজ।

ধাপ ৫ — Secrets Manager-এ ডাটাবেস পাসওয়ার্ড (ধারণা)

Console-এ `Secrets Manager → Store a new secret → Other type of secret` → Key: `DB_PASSWORD`, Value: `CHANGE-ME-Db-Password-123!` → নাম `technova/dev/db` → Store। ECS Task Definition-এ `secrets` অংশে এই Secret-এর ARN দিলে কন্টেইনার পরিবেশ ভেরিয়েবল হিসেবে পায় — কোডে বা ইমেজে পাসওয়ার্ড থাকে না।

৬. যাচাই

- GitHub Actions-এ Workflow সবুজ (Success)।
- ECR-এ নতুন ইমেজ ট্যাগ (Commit SHA) দেখা যায়।
- ECS Service-এ নতুন Deployment সম্পন্ন হয়।
- GitHub Secrets-এ কোনো AWS Access Key নেই — OIDC ব্যবহার হয়েছে।

৭. ট্রাবলশুটিং

সমস্যা	কারণ ও সমাধান
`Not authorized to perform sts:AssumeRoleWithWebIdentity`	Trust Policy-র `sub` শর্ত ভুল — রিপোজিটরি নাম, ব্যবহারকারী নাম বা ব্রাঞ্চ মিলছে না। হুবহু মিলিয়ে দেখুন।
`permissions: id-token: write` নেই	Workflow-এ permissions ব্লক যোগ করুন — না হলে OIDC টোকেন পাওয়া যায় না।
Docker build ব্যর্থ	লোকালে `docker build` চালিয়ে দেখুন — GitHub-এর এরর আর লোকাল এরর একই হওয়া উচিত।

৮. নিরাপত্তা বিবেচনা

GitHub Secrets-এ দীর্ঘমেয়াদি AWS Access Key রাখবেন না — OIDC-ই সঠিক পদ্ধতি। OIDC রোলের Trust Policy-তে শুধু নির্দিষ্ট রিপোজিটরি ও ব্রাঞ্চ দিন — `*` দিলে যেকোনো রিপোজিটরি AssumeRole করতে পারবে। Workflow-এ `pull_request_target` ব্যবহারে সতর্ক থাকুন — এতে Secret ফাঁসের ঝুঁকি থাকে।

৯. পরিষ্কার করা

Lab 17-এর মতো ECS Service 0 করে মুছুন, ECR ইমেজ মুছুন। OIDC রোলটি রেখে দিতে পারেন (কোনো খরচ নেই) — Project 02-এ আবার লাগবে। GitHub Workflow ফাইলটি রিপোজিটরিতে রেখে দিন — এটাই আপনার CI/CD প্রমাণ।

১০. চিন্তার প্রশ্ন

1. OIDC কেন Access Key-এর চেয়ে নিরাপদ — টোকেন চুরি গেলেও কী হবে?
2. Push করলেই প্রোডাকশনে ডেপ্লয় হওয়া কি সবসময় ভালো? কখন Manual Approval দরকার?
3. ডেপ্লয়মেন্ট ব্যর্থ হলে আগের ভার্সনে ফিরে যাওয়া (rollback) কীভাবে করবেন?

ল্যাব ২১ • পর্যবেক্ষণ ও অডিট • ৬০ মিনিট • কস্ট রিস্ক: Low

Lab 21 — CloudWatch Monitoring and CloudTrail Auditing

১. উদ্দেশ্য

- CloudWatch Dashboard বানিয়ে EC2-র CPU, নেটওয়ার্ক ও ডিস্ক এক জায়গায় দেখতে পারবেন।
- অ্যালার্ম বানিয়ে CPU বেশি হলে ইমেইল/SNS নোটিফিকেশন পেতে পারবেন।
- CloudTrail দিয়ে "কে, কখন, কী করল" — API ইতিহাস দেখতে পারবেন।
- VPC Flow Logs চালু করে নেটওয়ার্ক ট্র্যাফিক নিরীক্ষণ করতে পারবেন।

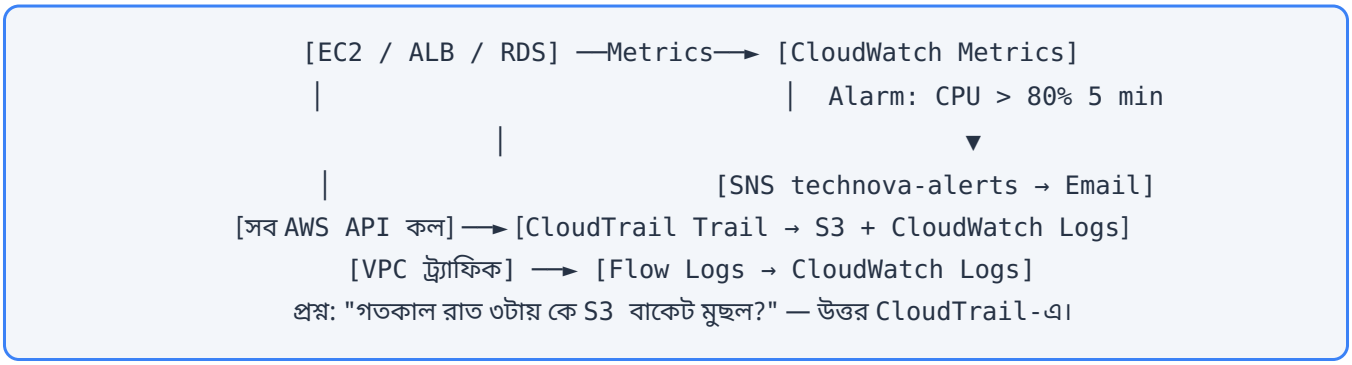
২. পূর্বশর্ত

Lab 7-এর EC2 চলছে (বা নতুন করে চালু করুন)। Lab 16-এর SNS Topic ধারণা।

৩. প্রয়োজনীয় AWS সেবা

`CloudWatch (Metrics, Alarms, Dashboards, Logs)`, `CloudTrail`, `SNS`, `VPC Flow Logs`।

৪. আর্কিটেকচার



৫. ধাপে ধাপে

ধাপ ১ — CloudWatch Dashboard (Console)

`CloudWatch → Dashboards → Create dashboard` → নাম `technova-dev-overview` → Add widget → Line → Metrics: EC2 → Per-Instance Metrics → ইনস্ট্যান্সের `CPUUtilization`, `NetworkIn`, `NetworkOut` → Create widget। একইভাবে `DiskReadOps` ও `StatusCheckFailed` যোগ করুন।

ধাপ ২ — CPU অ্যালার্ম + SNS (CLI)

```
aws cloudwatch put-metric-alarm --alarm-name technova-high-cpu \
  --alarm-description "TechNova dev EC2 CPU above 80 percent" \
  --metric-name CPUUtilization --namespace AWS/EC2 \
```

```
  --statistic Average --period 300 --threshold 80 \
  --comparison-operator GreaterThanThreshold \
  --evaluation-periods 1 --dimensions Name=InstanceId,Value=CHANGE-ME-INSTANCE-ID \
  --alarm-actions CHANGE-ME-SNS-TOPIC-ARN
aws cloudwatch describe-alarms --alarm-names technova-high-cpu \
  --query 'MetricAlarms[].StateValue'
```

ধাপ ৩ — অ্যালার্ম পরীক্ষা (লোড দিয়ে)

```
ssh -i ~/technova-dev-key.pem ec2-user@CHANGE-ME-PUBLIC-IP
sudo dnf install -y stress
stress --cpu 2 --timeout 400 &
exit
# ৫ মিনিট পর: CloudWatch-এ Alarm state IN_ALARM হওয়া উচিত,
# আর SNS থেকে ইমেইল আসা উচিত। তারপর killall stress।
```

ধাপ ৪ — CloudTrail Trail তৈরি

`CloudTrail → Create trail` → নাম `technova-audit-trail` → Storage: নতুন S3 বাকেট (`technova-cloudtrail-logs-CHANGE-ME`) → Log file validation: Enable → CloudWatch Logs: Enable (নতুন Log Group) → Create। কয়েক মিনিট পর:

```
aws cloudtrail lookup-events --lookup-attributes
AttributeKey=ResourceName,AttributeValue=technova-dev-web-01 --max-results 5
aws cloudtrail lookup-events --lookup-attributes
AttributeKey=EventName,AttributeValue=RunInstances --max-results 5
```

ধাপ ৫ — VPC Flow Logs চালু

`VPC → Your VPCs → technova-dev-vpc → Actions → Create flow log` → Filter: All → Destination: CloudWatch Logs → Log group: নতুন (`technova-vpc-flow-logs`) → IAM Role: নতুন তৈরি করুন → Create। কয়েক মিনিট পর Log Group-এ ফ্লো লগ দেখুন — কোন IP থেকে কোন পোর্টে সংযোগ এসেছে, তা পড়া যায়।

ধাপ ৬ — Logs Insights দিয়ে খোঁজা

`CloudWatch → Logs Insights` → Log group: `technova-vpc-flow-logs` → কুয়েরি:

```
fields @timestamp, srcAddr, dstPort, action
| filter action = "REJECT"
| sort @timestamp desc
| limit 20
```

REJECT হওয়া সংযোগগুলো দেখুন — কেউ বন্ধ পোর্টে ঢোকার চেষ্টা করেছে কি না বোঝা যায়।

৬. যাচাই

- Dashboard-এ CPU গ্রাফ দেখা যায়।
- লোড দিলে অ্যালার্ম IN_ALARM হয় ও ইমেইল আসে।
- CloudTrail-এ `RunInstances` ইভেন্টে ব্যবহারকারীর নাম ও সময় দেখা যায়।
- Flow Logs-এ ট্র্যাফিক রেকর্ড আছে।

৭. ট্রাবলশুটিং

সমস্যা	কারণ ও সমাধান
অ্যালার্ম IN_ALARM হয় না	Period ও Evaluation Periods দেখুন; CPU সত্যিই 80% পেরিয়েছে কি না Dashboard-এ দেখুন। stress চলছে কি না যাচাই করুন।
CloudTrail-এ ইভেন্ট দেখা যায় না	Trail তৈরির আগের ইভেন্ট দেখা যায় না। ৫–১০ মিনিট অপেক্ষা করুন; Lookup-এ সময়সীমা (৯০ দিন) মনে রাখুন।
Flow Logs খালি	Log Group-এ IAM Role-র অনুমতি আছে কি না দেখুন; ট্র্যাফিক না থাকলে লগও থাকে না — EC2-তে curl করুন।

৮. নিরাপত্তা বিবেচনা

CloudTrail কখনো বন্ধ করবেন না — এটি আপনার অডিট প্রমাণ। Trail-এর S3 বাকেটে *Log file validation* ও *MFA Delete* চালু করুন — কেউ লগ বদলাতে পারবে না। অ্যালার্মের SNS Topic-এ শুধু দায়িত্বশীলদের ইমেইল রাখুন — সবাইকে রাখলে অ্যালার্ট উপেক্ষিত হয় (alert fatigue)।

৯. পরিষ্কার করা

```
aws cloudwatch delete-alarms --alarm-names technova-high-cpu
aws cloudwatch delete-dashboards --dashboard-names technova-dev-overview
aws cloudtrail delete-trail --name technova-audit-trail
aws ec2 delete-flow-logs --flow-log-ids CHANGE-ME-FLOW-LOG-ID
aws ec2 terminate-instances --instance-ids CHANGE-ME-INSTANCE-ID
# CloudTrail S3 বাকেটটি খালি করে মুছুন (Lab 11-এর পদ্ধতি)
```

CloudWatch Log Group-গুলো (`/aws/lambda/*`, Flow Logs) Console থেকে মুছে ফেলুন — না হলে ছোট স্টোরেজ চার্জ চলতে থাকে।

১০. চিন্তার প্রশ্ন

1. Metrics, Logs ও Traces — তিনটির পার্থক্য কী? একটি ধীর API ডিবাগ করতে কোনটি আগে দেখবেন?
2. অ্যালার্মের Threshold 80% না 95% — কোনটি ভালো, কেন?
3. CloudTrail বন্ধ করে কেউ খারাপ কাজ করলে ধরা যাবে কীভাবে? কী সুরক্ষা দরকার?

ল্যাব ২২ • ব্যাকআপ ও DR • ৭৫ মিনিট • কস্ট রিস্ক: Low — Backup Vault-এর স্টোরেজ খরচ সামান্য

Lab 22 — Backup and Disaster Recovery

১. উদ্দেশ্য

- AWS Backup দিয়ে EC2 ও RDS-র স্বয়ংক্রিয় ব্যাকআপ পরিকল্পনা (Backup Plan) বানাতে পারবেন।
- Recovery Point Objective (RPO) ও Recovery Time Objective (RTO) বুঝে পরিকল্পনা লিখতে পারবেন।
- ব্যাকআপ থেকে পুনরুদ্ধার (Restore) পরীক্ষা করে প্রমাণ করতে পারবেন।
- দ্বিতীয় রিজিয়নে ব্যাকআপ কপি (Cross-Region) করার পদ্ধতি বুঝবেন।

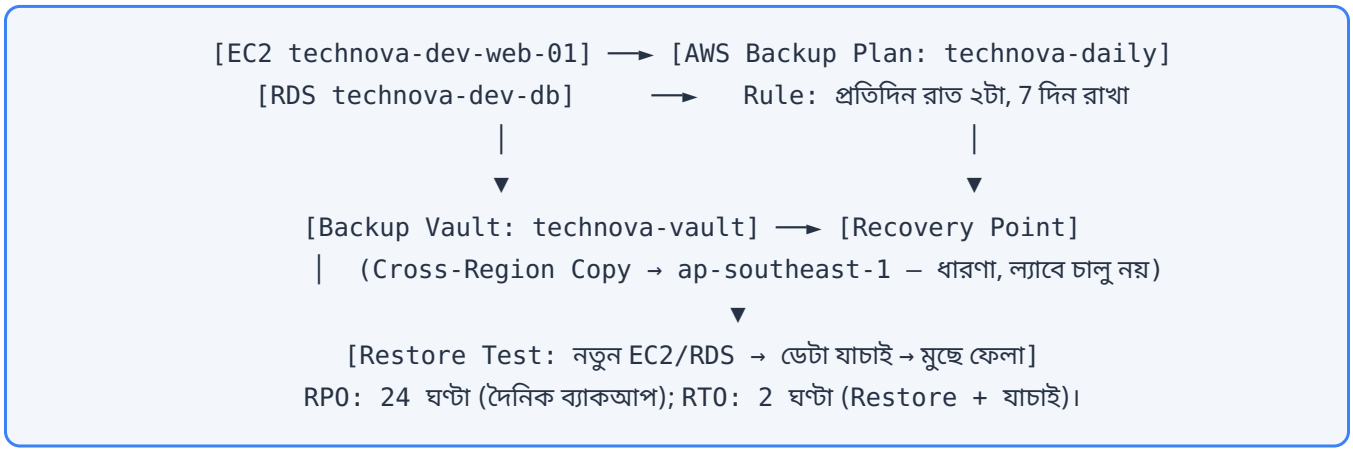
২. পূর্বশর্ত

Lab 8 (EBS স্ন্যাপশট) ও Lab 13 (RDS ব্যাকআপ) সম্পন্ন। একটি চলমান EC2 (পরীক্ষার জন্য)।

৩. প্রয়োজনীয় AWS সেবা

`AWS Backup`, `EBS Snapshots`, `RDS Snapshots`, `S3 (Cross-Region Replication — ধারণা)`।

৪. আর্কিটেকচার



৫. ধাপে ধাপে

ধাপ ১ — RPO/RTO ঠিক করা (কাগজে)

নোটবুকে লিখুন: TechNova-র ওয়েবসাইটের জন্য RPO 24 ঘণ্টা (এক দিনের ডেটা হারানো চলবে) ও RTO 2 ঘণ্টা (দুই ঘণ্টায় সাইট ফিরিয়ে আনতে হবে)। ডাটাবেসের জন্য RPO 1 ঘণ্টা — তাই RDS-র অটোমেটেড ব্যাকআপ আলাদা করে ১ ঘণ্টা পরপর নয়, বরং Point-in-time recovery ব্যবহার করুন। এই দুটি সংখ্যাই ব্যাকআপ পরিকল্পনার ভিত্তি — সংখ্যা ছাড়া পরিকল্পনা হয় না।

ধাপ ২ — Backup Vault ও Plan তৈরি (Console)

`AWS Backup → Backup vaults → Create backup vault` → নাম `technova-vault` → Encryption: Default → Create। তারপর `Backup plans → Create backup plan → Build a new plan` → নাম `technova-daily` →

Backup rule: নাম `daily-2am`, Vault: `technova-vault`, Schedule: প্রতিদিন ২:০০ AM, Retention: 7 দিন → Create plan।

ধাপ ৩ — রিসোর্স Assign করা

Plan-এর পাতায় *Assign resources* → নাম `technova-ec2-rds` → Resource selection: Tag `Backup=Daily` → Save। তারপর EC2-তে ট্যাগ যোগ করুন:

```
aws ec2 create-tags --resources CHANGE-ME-INSTANCE-ID \
  --tags Key=Backup,Value=Daily Key=Project,Value=TechNova
```

ধাপ ৪ — On-Demand ব্যাকআপ নেওয়া (CLI)

```
aws backup start-backup-job --backup-vault-name technova-vault \
  --resource-arn arn:aws:ec2:ap-south-1:111122223333:instance/CHANGE-ME-INSTANCE-ID \
  --iam-role-arn arn:aws:iam::111122223333:role/service-role/AWSBackupDefaultServiceRole
aws backup list-recovery-points-by-backup-vault --backup-vault-name technova-vault \
  --query 'RecoveryPoints[].{Status:Status,Created:CreationDate}'
```

Status *COMPLETED* হওয়া পর্যন্ত অপেক্ষা করুন।

ধাপ ৫ — Restore পরীক্ষা (সবচেয়ে গুরুত্বপূর্ণ ধাপ)

Console-এ Recovery Point নির্বাচন → *Restore* → নতুন EC2 হিসেবে Restore করুন (আলাদা নামে)। Restore হওয়া EC2-তে SSH করে দেখুন — আগের ফাইল আছে কি না। যাচাই শেষে Restore করা EC2 Terminate করুন।

> **ব্যাকআপ পরীক্ষা না করলে ব্যাকআপ নেই:** অনেক টিম বছরের পর বছর ব্যাকআপ নেয়, কিন্তু Restore কখনো পরীক্ষা করে না — বিপদের দিন দেখে ব্যাকআপ নষ্ট। এই ল্যাবের Restore পরীক্ষাটাই আসল শিক্ষা।

ধাপ ৬ — Cross-Region Copy (ধারণা + সেটআপ দেখা)

Backup Plan-এর Rule-এ *Copy to another Region* যোগ করা যায় — যেমন `ap-southeast-1` (Singapore)। ল্যাবে এটি চালু করবেন না (অতিরিক্ত খরচ), তবে Console-এ অপশনটি খুলে দেখুন — Destination Region, Vault ও Lifecycle কীভাবে সেট করতে হয়। S3-র Cross-Region Replication (CRR)-এর ধারণাও একই — একটি রিজিয়ন পুরো ডাউন হলেও ডেটা অন্য রিজিয়নে থাকে।

৬. যাচাই

- Backup Plan *technova-daily* সক্রিয়, Rule দৈনিক ২টা।
- On-Demand ব্যাকআপ Status COMPLETED।
- Restore করা EC2-তে আগের ডেটা আছে।
- RPO/RTO নোটবুকে লেখা আছে।

৭. ট্রাবলশুটিং

সমস্যা	কারণ ও সমাধান

ব্যাকআপ Job ব্যর্থ	AWS Backup-এর Service Role-এ অনুমতি আছে কি না দেখুন; EC2-র ট্যাগ ঠিক আছে কি না দেখুন।
Restore-এ নতুন EC2 চালু হয় না	Restore-এর সময় সাবনেট ও Security Group নির্বাচন করতে হয় — ডিফল্ট VPC-তে Restore হতে পারে। সঠিক সাবনেট বেছে নিন।
Recovery Point দেখা যায় না	Vault-এর নাম ও রিজিয়ন ঠিক আছে কি না দেখুন; ব্যাকআপ সম্পূর্ণ হতে সময় লাগে।

৮. নিরাপত্তা বিবেচনা

Backup Vault-এ *Vault Lock* (WORM — লেখা যায়, মোছা যায় না) চালু করলে Ransomware-ও ব্যাকআপ মুছতে পারবে না। ব্যাকআপের Access শুধু দায়িত্বশীলদের দিন — ব্যাকআপে পুরো সিস্টেমের ডেটা থাকে। নিয়মিত Restore Drill-এর তারিখ ক্যালেন্ডারে রাখুন (মাসে একবার)।

৯. পরিষ্কার করা

```
aws backup delete-recovery-point --backup-vault-name technova-vault \
  --recovery-point-arn CHANGE-ME-RECOVERY-POINT-ARN
aws backup delete-backup-plan --backup-plan-id CHANGE-ME-PLAN-ID
aws backup delete-backup-vault --backup-vault-name technova-vault
aws ec2 terminate-instances --instance-ids CHANGE-ME-INSTANCE-ID CHANGE-ME-RESTORED-ID
```

Restore করা EC2 ও মূল EC2 — দুটোই Terminate করুন। Vault মুছতে আগে সব Recovery Point মুছতে হয়।

১০. চিন্তার প্রশ্ন

1. RPO 1 ঘণ্টা আর RPO 24 ঘণ্টা — খরচের পার্থক্য কোথায়?
2. একটি AZ ডাউন হলে কী করবেন, আর পুরো রিজিয়ন ডাউন হলে কী করবেন — পার্থক্য কী?
3. ব্যাকআপ নেওয়া আর Disaster Recovery পরিকল্পনা — এক জিনিস নয় কেন?

ল্যাব ২৩ • নিরাপত্তা ও খরচ • ৬০ মিনিট • কস্ট রিস্ক: Low

Lab 23 — Security Audit and Cost Analysis

১. উদ্দেশ্য

- AWS Security Hub ও Trusted Advisor দিয়ে অ্যাকাউন্টের নিরাপত্তা ঘাটতি খুঁজে বের করতে পারবেন।
- Cost Explorer দিয়ে কোন সেবায় কত খরচ হচ্ছে তা বিশ্লেষণ করতে পারবেন।
- অপ্রয়োজনীয় রিসোর্স (Idle EC2, অব্যবহৃত EIP, পুরোনো স্ন্যাপশট) চিহ্নিত করে খরচ কমাতে পারবেন।
- একটি সংক্ষিপ্ত নিরাপত্তা ও খরচ প্রতিবেদন লিখতে পারবেন।

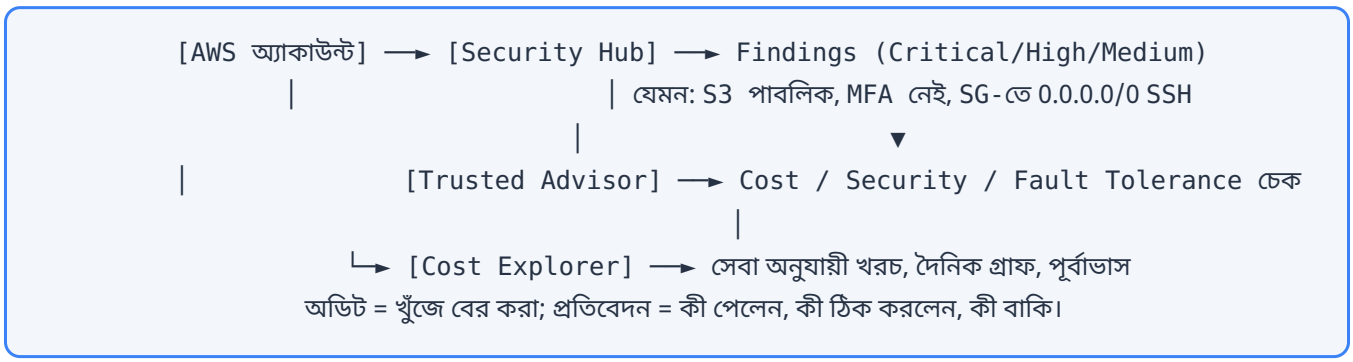
২. পূর্বশর্ত

Lab 1 (MFA, বিলিং অ্যালার্ট) ও Lab 21 (CloudTrail) সম্পন্ন। কয়েকটি ল্যাবের রিসোর্স চালু আছে — অডিটের জন্য সেটাই দরকার।

৩. প্রয়োজনীয় AWS সেবা

`Security Hub`, `Trusted Advisor`, `Cost Explorer`, `AWS Budgets`, `GuardDuty (ধারণা)`, `Config (ধারণা)`।

৪. আর্কিটেকচার



৫. ধাপে ধাপে

ধাপ ১ — Security Hub চালু ও Findings দেখা

`Security Hub → Enable Security Hub` → Standards: AWS Foundational Security Best Practices ON → Enable। ১০–১৫ মিনিট অপেক্ষা করুন। তারপর *Findings* দেখুন — Severity অনুযায়ী সাজান। সাধারণ Findings: "MFA not enabled for IAM users", "S3 Block Public Access disabled", "Security Group allows unrestricted SSH"। প্রতিটি Finding খুলে *Remediation* পড়ুন।

ধাপ ২ — Trusted Advisor চেক

`Trusted Advisor → Dashboard` → চারটি বিভাগ দেখুন: Cost Optimization, Security, Fault Tolerance, Performance। *Cost Optimization*-এ "Idle EC2", "Unassociated Elastic IP", "Old EBS Snapshots" চেকগুলো

দেখুন — প্রতিটিতে আনুমানিক মাসিক সাশ্রয় লেখা থাকে। *Security*-তে "MFA on Root", "IAM Access Key Rotation" দেখুন।

ধাপ ৩ — Cost Explorer বিশ্লেষণ

`Cost Explorer → Cost & Usage` → Date range: গত ৩০ দিন → Group by: Service → দেখুন কোন সেবায় সবচেয়ে বেশি খরচ (EC2? NAT? RDS?)। তারপর Group by: Tag (`Project=TechNova`) করে দেখুন। *Forecast* দেখুন — এই মাস শেষে কত বিল হতে পারে।

```
aws ce get-cost-and-usage --time-period Start=2026-10-09,End=2026-11-09 \
  --granularity MONTHLY --metrics BlendedCost \
  --group-by Type=DIMENSION,Key=SERVICE \
  --query 'ResultsByTime[].Groups[].{Service:Keys[0],Cost:Metrics.BlendedCost.Amount}'
```

ধাপ ৪ — অপ্রয়োজনীয় রিসোর্স খুঁজে বের করা (হাতে-কলমে)

```
# অব্যবহৃত Elastic IP:
aws ec2 describe-addresses --query 'Addresses[?AssociationId==null]'
# পুরোনো স্ন্যাপশট:
aws ec2 describe-snapshots --owner-ids self \
  --query 'Snapshots[].{Id:SnapshotId,Created:StartTime}'
# বন্ধ থাকা (stopped) EC2 — তবুও EBS চার্জ হয়:
aws ec2 describe-instances --filters Name=instance-state-name,Values=stopped \
  --query 'Reservations[].Instances[].InstanceId'
# খালি/অব্যবহৃত S3 বাকেটের আকার:
aws s3 ls --recursive s3://CHANGE-ME-BUCKET/ --summarize | tail -3
```

ধাপ ৫ — প্রতিবেদন লেখা

নোটবুকে এক পাতার প্রতিবেদন লিখুন — এই ছকটি ব্যবহার করুন:

বিভাগ	যা পেলেন	যা করলেন
নিরাপত্তা	যেমন: ২টি IAM ব্যবহারকারীর MFA নেই	MFA চালু করলেন / পরিকল্পনা লিখলেন
খরচ	যেমন: NAT Gateway সবচেয়ে বেশি খরচ	ল্যাব শেষে NAT মুছলেন
অপ্রয়োজনীয় রিসোর্স	যেমন: ১টি অব্যবহৃত EIP, ৩টি পুরোনো স্ন্যাপশট	Release/Delete করলেন
বাকি ঝুঁকি	যেমন: CloudTrail এক রিজিয়নে মাত্র	সব রিজিয়নে চালুর পরিকল্পনা

৬. যাচাই

- Security Hub-এ Findings দেখা যায় ও কমপক্ষে ২টির Remediation বোঝা গেছে।
- Trusted Advisor-এ Cost Optimization-এর সাশ্রয়ের পরিমাণ পড়া গেছে।
- Cost Explorer-এ সবচেয়ে ব্যয়বহুল সেবাটি চিহ্নিত করা গেছে।
- কমপক্ষে ১টি অপ্রয়োজনীয় রিসোর্স মুছে খরচ কমানো গেছে।
- এক পাতার প্রতিবেদন লেখা সম্পন্ন।

৭. ট্রাবলশুটিং

সমস্যা	কারণ ও সমাধান
Security Hub-এ কোনো Finding নেই	প্রথম স্ক্যানে সময় লাগে (১৫–৩০ মিনিট)। অপেক্ষা করুন; Standards চালু আছে কি না দেখুন।
Cost Explorer-এ ডেটা নেই	Cost Explorer প্রথমবার চালু করতে হয় (*Enable Cost Explorer*) — ডেটা আসতে ২৪ ঘণ্টা লাগতে পারে।
Trusted Advisor-এর অনেক চেক "—" দেখায়	পূর্ণ Trusted Advisor Business/Enterprise Support ছাড়া সীমিত। Free-তে Cost ও Security-র মৌলিক চেকগুলোই দেখা যায় — সেগুলো দিয়েই কাজ করুন।

৮. নিরাপত্তা বিবেচনা

অডিটের ফল কাউকে শেয়ার করার আগে সংবেদনশীল তথ্য (অ্যাকাউন্ট ID, IP) সরিয়ে নিন। GuardDuty চালু করলে অস্বাভাবিক API কল ও ম্যালওয়্যার সতর্কতা পাওয়া যায় — প্রোডাকশনে চালু করুন। AWS Config দিয়ে "নিয়ম ভাঙলেই সতর্কতা" (যেমন S3 পাবলিক হলেই অ্যালার্ট) সেট করুন।

৯. পরিষ্কার করা

এই ল্যাবে কোনো বিলযোগ্য রিসোর্স বানানো হয়নি — তবে অডিটে পাওয়া অপ্রয়োজনীয় রিসোর্সগুলো (অব্যবহৃত EIP, পুরোনো স্ন্যাপশট, বন্ধ EC2) এখনই মুছে ফেলুন। Security Hub চালু রেখে দিতে পারেন (প্রথম ৩০ দিন ফ্রি, তারপর চার্জ — AWS Pricing Calculator-এ যাচাই করুন; চাইলে Disable করুন)।

১০. চিন্তার প্রশ্ন

1. Security Hub-এর Critical Finding আর Trusted Advisor-এর Cost সতর্কতা — কোনটি আগে ঠিক করবেন, কেন?
2. একটি টিমে কেউ NAT Gateway ভুলে চালু রেখে গেলে খরচের দায় কার? কীভাবে ঠেকাবেন?
3. Cost Explorer-এর Forecast কতটা নির্ভরযোগ্য? কী কী বদলালে Forecast ভুল হবে?

ল্যাব ২৪ • সমন্বিত ডেপ্লয়মেন্ট • ১৮০ মিনিট • কস্ট রিস্ক: High — সব সেবা একসঙ্গে চালু থাকে; ল্যাব শেষেই সব মুছুন

Lab 24 — Complete Production-Style MERN Deployment

১. উদ্দেশ্য

- আগের সব ল্যাবের দক্ষতা একত্র করে একটি সম্পূর্ণ MERN অ্যাপ প্রোডাকশন-স্টাইলে ডেপ্লয় করতে পারবেন।
- Frontend (React) S3 + CloudFront-এ, Backend (Node/Express) ECS Fargate-এ, ডাটাবেস RDS-এ চালাতে পারবেন।
- CI/CD, Secrets Manager, CloudWatch মনিটরিং ও HTTPS — সব একসঙ্গে যুক্ত করতে পারবেন।
- ডেপ্লয়মেন্টের শেষে সম্পূর্ণ সিস্টেম যাচাই করে ডকুমেন্ট লিখতে পারবেন।

২. পূর্বশর্ত

Lab 1–23 সম্পন্ন (বা সমমানের অভিজ্ঞতা)। MERN অ্যাপের কোড (নিজের প্রজেক্ট — যেমন Bookora / TeamFlow — বা নিচের নমুনা)। GitHub রিপোজিটরি।

৩. প্রয়োজনীয় AWS সেবা

`VPC, ALB, ECS Fargate, ECR, RDS (PostgreSQL — MERN-এ MongoDB-র বদলে ল্যাবে PostgreSQL; Atlas ব্যবহার করলে MongoDB Atlas), S3, CloudFront, Route 53, ACM, Secrets Manager, CloudWatch, GitHub Actions।`

৪. আর্কিটেকচার

```
                [User — HTTPS] ──► [Route 53: app.technova.example]
                   ├──► [CloudFront] ──► [S3: technova-frontend] (React build)
                   └──► [ALB] ──► [ECS Fargate: technova-api] (Node/Express, 2 tasks)
                                          │  Secrets Manager থেকে DB পাসওয়ার্ড
                                          ▼
                                  [RDS PostgreSQL] (Private subnet, SG-db)
                                          │
                                  [CloudWatch: Logs + Alarms + Dashboard]
[GitHub Push] ──► [GitHub Actions] ──► [ECR → ECS] + [S3 Sync → CloudFront Invalidation]
```

নোট: MERN-এ M = MongoDB; এই ল্যাবে RDS PostgreSQL ব্যবহার করা হয়েছে কারণ Lab 13-এ সেটাই শেখা হয়েছে। MongoDB Atlas ব্যবহার করলে সংযোগ স্ট্রিং Secrets Manager-এ রাখুন — আর্কিটেকচার একই থাকে।

৫. ধাপে ধাপে

ধাপ ১ — ভিত্তি তৈরি (Terraform বা Console)

Lab 19-এর Terraform কোড প্রসারিত করুন, অথবা Lab 4–6-এর VPC ব্যবহার করুন। নিশ্চিত করুন: VPC, ৪টি সাবনেট, IGW, NAT (এই ল্যাবের জন্য), ৩টি Security Group (`sg-alb` , `sg-app` , `sg-db`) — সব প্রস্তুত।

ধাপ ২ — ডাটাবেস ও Secret

Lab 13-এর মতো RDS PostgreSQL বানান (`technova-prod-db`)। পাসওয়ার্ড Secrets Manager-এ রাখুন (`technova/prod/db`)। ECS Task Execution Role-কে Secret পড়ার অনুমতি দিন।

ধাপ ৩ — Backend: ECR + ECS Fargate

Express অ্যাপের নমুনা (`server.js`):

```
const express = require("express");
const { Pool } = require("pg");
const app = express();
app.use(express.json());
const pool = new Pool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: "technova",
  port: 5432
});
app.get("/health", (req, res) => res.json({ status: "ok" }));
app.get("/api/users", async (req, res) => {
  const { rows } = await pool.query(
    "SELECT id, name, email FROM users ORDER BY id LIMIT 20");
  res.json(rows);
});
app.post("/api/users", async (req, res) => {
  const { name, email } = req.body;
  const { rows } = await pool.query(
    "INSERT INTO users (name, email) VALUES ($1, $2) RETURNING *",
    [name, email]);
  res.status(201).json(rows[0]);
});
app.listen(3000, () => console.log("API listening on 3000"));
```

Dockerfile বানিয়ে ECR-এ পুশ করুন (Lab 17), ECS Service বানান Desired 2, ALB-র পেছনে (Lab 9–10)। Task Definition-এ `secrets` অংশে DB পাসওয়ার্ডের ARN দিন — পরিবেশ ভেরিয়েবলে প্লেইন টেক্সটে নয়।

ধাপ ৪ — Frontend: React Build → S3 → CloudFront

```
cd frontend
npm run build
aws s3 sync build/ s3://technova-frontend-CHANGE-ME/ --delete
aws cloudfront create-invalidation --distribution-id CHANGE-ME-DIST-ID \
  --paths "/*"
```

CloudFront Distribution-এর Origin: S3 (OAC), Default root: `index.html`। React Router ব্যবহার করলে CloudFront-এ Custom Error Response: 404 → `/index.html` 200 সেট করুন — না হলে রিফ্রেশে 404 আসবে।

ধাপ ৫ — HTTPS ও DNS (আসল ডোমেইন থাকলে)

ACM সার্টিফিকেট (`us-east-1`) → CloudFront-এ যুক্ত করুন; ALB-তেও HTTPS Listener যোগ করুন। Route 53-এ `app.technova.example` → CloudFront Alias, `api.technova.example` → ALB Alias।

ধাপ ৬ — CI/CD যুক্ত করা

Lab 20-এর Workflow প্রসারিত করুন — দুটি Job: `deploy-api` (ECR → ECS) ও `deploy-frontend` (S3 Sync + Invalidation)। `main` ব্রাঞ্চে Push করলেই দুটো ডেপ্লয় হয়।

ধাপ ৭ — মনিটরিং ও অ্যালার্ম

Lab 21-এর মতো Dashboard বানান: ALB Request Count, ECS CPU/Memory, RDS CPU ও Free Storage, API Latency। অ্যালার্ম: ALB 5xx > 10 (৫ মিনিটে), RDS Free Storage < 5 GB, ECS Running Task < 1।

ধাপ ৮ — সম্পূর্ণ যাচাই (End-to-End Test)

```
# 1. Frontend খোলে কি না:
curl -s https://CHANGE-ME-CLOUDFRONT-DOMAIN/ | head -5
# 2. API Health:
curl -s http://CHANGE-ME-ALB-DNS/health
# 3. ব্যবহারকারী তৈরি:
curl -s -X POST http://CHANGE-ME-ALB-DNS/api/users \
  -H "Content-Type: application/json" \
  -d '{"name":"Ayesha Rahman","email":"ayesha@technova.example"}'
# 4. তালিকা:
curl -s http://CHANGE-ME-ALB-DNS/api/users
```

৬. যাচাই

- Frontend HTTPS-এ খোলে, React অ্যাপ লোড হয়।
- `/api/users` থেকে ডেটা আসে — Frontend থেকে API কল সফল।
- নতুন ব্যবহারকারী তৈরি করলে RDS-তে রো যোগ হয়।
- একটি ECS Task বন্ধ করলে অন্যটি ট্র্যাফিক নেয়; ASG/ECS নিজে নতুন Task বানায়।
- GitHub-এ Push করলে নতুন ভার্সন ডেপ্লয় হয়।
- Dashboard-এ সব মেট্রিক দেখা যায়; অ্যালার্ম সেট আছে।

৭. ট্রাবলশুটিং

সমস্যা	কারণ ও সমাধান
Frontend থেকে API কলে CORS এরর	Express-এ `cors` মিডলওয়্যার যোগ করুন — শুধু নিজের ডোমেইনকে অনুমতি দিন (`origin: https://app.technova.example`)।
API থেকে DB সংযোগ ব্যর্থ	

	Secrets Manager-এর Secret-এ DB_HOST ভুল, বা SG-db-তে SG-app থেকে 5432 বন্ধ। ECS Task-এর লগ CloudWatch-এ দেখুন।
React রিফ্রেশে 404	CloudFront Custom Error Response সেট করা হয়নি (ধাপ ৪ দেখুন)।
ডেপ্লয়মেন্টের পর পুরোনো Frontend দেখায়	CloudFront Invalidation করা হয়নি। `/*` Invalidation করুন।

৮. নিরাপত্তা বিবেচনা

এই ল্যাবের নিরাপত্তা চেকলিস্ট: (১) DB প্রাইভেট সাবনেটে, পাবলিক অ্যাক্সেস No; (২) সব Secret Secrets Manager-এ; (৩) S3 শুধু CloudFront OAC দিয়ে; (৪) HTTPS সর্বত্র; (৫) SG-তে `0.0.0.0/0` শুধু 80/443-এ, SSH শুধু নিজের IP; (৬) CloudTrail ও GuardDuty চালু; (৭) WAF ALB-র সামনে (Project 02-এ যোগ করবেন)।

৯. পরিষ্কার করা

> **এই ল্যাবের Cleanup সবচেয়ে গুরুত্বপূর্ণ — সব সেবা একসঙ্গে চালু আছে।** নিচের ক্রম মেনে চলুন — উল্টো করলে নির্ভরতা (dependency) এরর আসবে।

```
# 1. ECS Service 0 করে মুছুন, Cluster মুছুন
aws ecs update-service --cluster technova-dev-cluster \
  --service technova-web-service --desired-count 0
aws ecs delete-service --cluster technova-dev-cluster \
  --service technova-web-service --force
aws ecs delete-cluster --cluster technova-dev-cluster
# 2. ALB + Target Group মুছুন
aws elbv2 delete-load-balancer --load-balancer-arn CHANGE-ME-ALB-ARN
aws elbv2 delete-target-group --target-group-arn CHANGE-ME-TG-ARN
# 3. CloudFront Disable → Delete; Route 53 রেকর্ড মুছুন
# 4. RDS মুছুন (skip final snapshot বা final snapshot নিন)
aws rds delete-db-instance --db-instance-identifier technova-prod-db \
  --skip-final-snapshot
# 5. ECR, S3 বাকেট, Secrets Manager Secret মুছুন
aws ecr delete-repository --repository-name technova-web --force
aws s3 rm s3://technova-frontend-CHANGE-ME --recursive
aws s3api delete-bucket --bucket technova-frontend-CHANGE-ME
aws secretsmanager delete-secret --secret-id technova/prod/db --force-delete-without-recovery
# 6. NAT Gateway + Elastic IP ছেড়ে দিন
aws ec2 delete-nat-gateway --nat-gateway-id CHANGE-ME-NAT-ID
aws ec2 release-address --allocation-id CHANGE-ME-EIP-ALLOC-ID
# 7. বিলিং যাচাই: Billing Dashboard-এ আজকের খরচ দেখুন
```

সব মুছে ফেলার পর Cost Explorer-এ আগামীকাল দেখুন — কোনো চলমান চার্জ আছে কি না।

১০. চিন্তার প্রশ্ন

1. এই আর্কিটেকচারে একটিমাত্র ব্যর্থতার বিন্দু (single point of failure) কোথায় কোথায় আছে?
2. ট্র্যাফিক দশ গুণ বাড়লে প্রথমে কোন অংশ ভাঙবে — Frontend, Backend, নাকি ডাটাবেস? কেন?
3. এই সিস্টেমকে "প্রোডাকশন-রেডি" বলতে আর কী কী লাগবে? (ব্যাকআপ পরীক্ষা, Runbook, On-call — ভাবুন।)

প্রজেক্ট ০১ • নেটওয়ার্ক ও কম্পিউট ভিত্তি • আনুমানিক ২–৩ সপ্তাহ • কস্ট রিস্ক: Medium

Project 01 — Secure AWS Network & Compute Foundation

TechNova-র সব সেবার ভিত্তি হবে এই প্রজেক্ট — একটি নিরাপদ, Multi-AZ নেটওয়ার্ক ও কম্পিউট প্ল্যাটফর্ম, যার উপরে Project 02-এর MERN অ্যাপ ও ক্যাপস্টোনের সম্পূর্ণ প্ল্যাটফর্ম দাঁড়াবে। এই প্রজেক্টে কোনো অ্যাপ্লিকেশন কোড নেই — শুধু ভিত্তি। ভিত্তি দুর্বল হলে উপরের সবকিছুই ঝুঁকিতে থাকে।

১. প্রয়োজনীয়তা (Requirements)

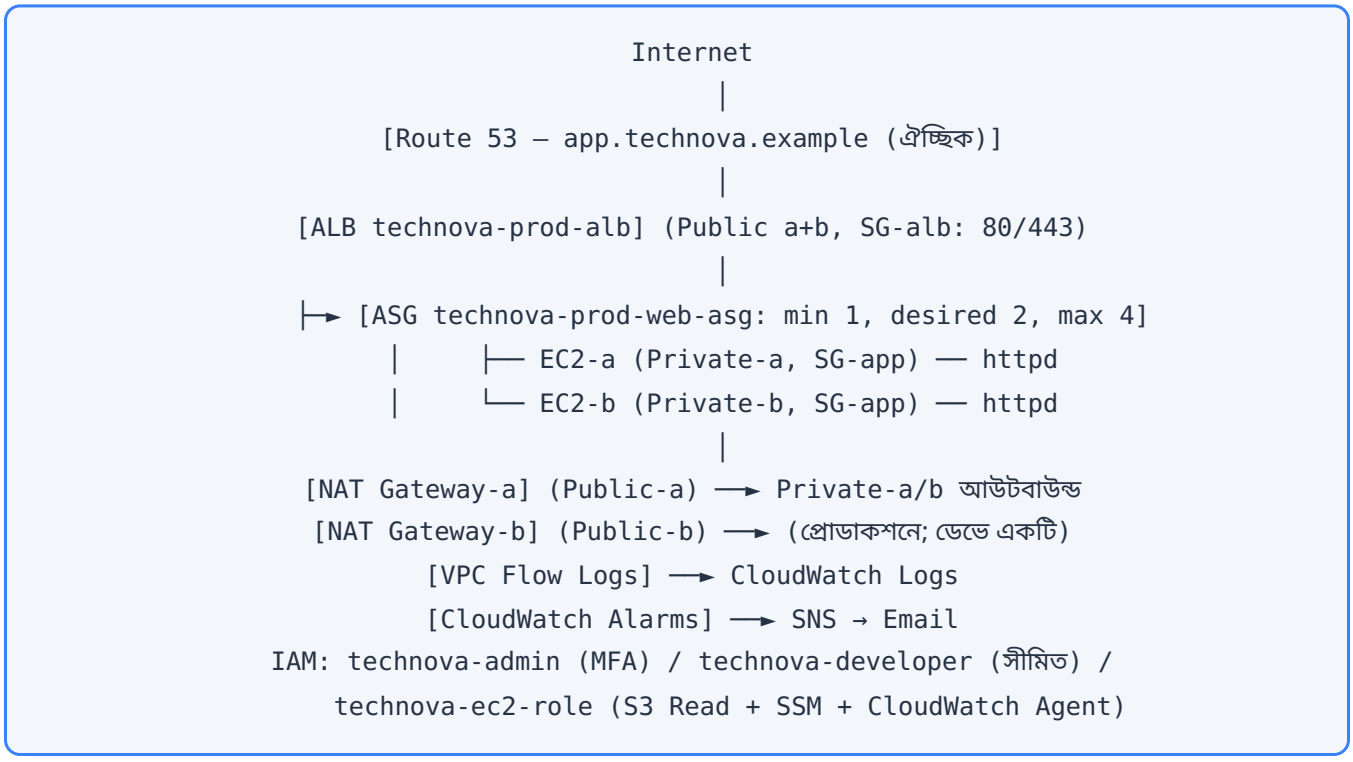
কার্যকরী প্রয়োজনীয়তা

- Multi-AZ VPC (`10.0.0.0/16`, `ap-south-1a` ও `ap-south-1b`) — প্রতিটি AZ-তে একটি পাবলিক ও একটি প্রাইভেট সাবনেট।
- পাবলিক সাবনেটে Internet Gateway; প্রাইভেট সাবনেটে NAT Gateway (প্রতি AZ-তে একটি — প্রোডাকশন মান)।
- ALB (পাবলিক) → Auto Scaling Group (প্রাইভেট, Desired 2, Min 1, Max 4) → EC2 (Amazon Linux 2023, httpd নমুনা অ্যাপ)।
- IAM: অ্যাডমিন, ডেভেলপার ও EC2-র জন্য আলাদা রোল — প্রতিটি Least-Privilege।
- নেটওয়ার্ক নিরাপত্তা: তিন স্তরের Security Group + NACL + VPC Flow Logs।
- মনিটরিং: CloudWatch Dashboard + CPU অ্যালার্ম + SNS নোটিফিকেশন।

অ-কার্যকরী প্রয়োজনীয়তা

- উপলব্ধতা: একটি AZ ডাউন হলেও সাইট চলবে (ALB + ASG দুটি AZ-তে)।
- নিরাপত্তা: ডাটাবেস/অ্যাপ স্তর ইন্টারনেটে সরাসরি খোলা নয়; SSH শুধু নির্দিষ্ট IP থেকে; সব ভলিউম এনক্রিপ্টেড।
- খরচ: ল্যাব/ডেভ পরিবেশে NAT একটি AZ-তে (খরচ বাঁচাতে) — তবে ডকুমেন্টে লিখতে হবে প্রোডাকশনে দুটি লাগে কেন।
- পুনরুৎপাদনযোগ্যতা: পুরো ভিত্তি Terraform কোডে থাকবে — হাতে ক্লিক করে বানানো অংশের ডকুমেন্ট থাকবে।

২. আর্কিটেকচার



৩. বাস্তবায়নের ধাপ

পর্যায় ১ — পরিকল্পনা ও ডকুমেন্ট (২ দিন)

CIDR পরিকল্পনা, সাবনেট টেবিল, Security Group ম্যাট্রিক্স ও খরচের হিসাব (AWS Pricing Calculator) — এক পাতার *Architecture Decision Record* (*ADR*) লিখুন: কেন `10.0.0.0/16`, কেন দুটি AZ, কেন NAT Gateway (NAT Instance নয়)।

পর্যায় ২ — নেটওয়ার্ক (৩ দিন)

Lab 4–6 অনুসরণ করে Terraform-এ VPC, সাবনেট, IGW, NAT, রাউট টেবিল, SG ও NACL বানান। `terraform plan` আউটপুট সংরক্ষণ করুন।

পর্যায় ৩ — কম্পিউট (৩ দিন)

Launch Template + ASG + Target Group + ALB (Lab 9–10)। Health Check ও স্কেলিং পলিসি সেট করুন। Failover পরীক্ষা করুন: একটি AZ-র ইনস্ট্যান্স Terminate করে দেখুন।

পর্যায় ৪ — IAM ও নিরাপত্তা (২ দিন)

Lab 1–2 অনুসরণ করে রোল ও পলিসি বানান। Policy Simulator দিয়ে যাচাই করুন। VPC Flow Logs চালু করুন। Security Hub-এর প্রথম Findings দেখে ঠিক করুন।

পর্যায় ৫ — মনিটরিং ও ডকুমেন্টেশন (২ দিন)

Dashboard, অ্যালার্ম, Runbook (ইনস্ট্যান্স মারা গেলে কী করবেন — ধাপে ধাপে) ও আর্কিটেকচার ডায়াগ্রাম (হাতে আঁকা বা draw.io) তৈরি করুন।

৪. যাচাই (Verification)

পরীক্ষা	প্রত্যাশিত ফল
ALB DNS-এ ব্রাউজার	নমুনা পাতা খোলে, রিফ্রেশে দুটি সার্ভারের উত্তর আসে
একটি EC2 Terminate	ASG ৫ মিনিটে নতুন বানায়; ডাউনটাইম নেই
একটি AZ-র সাবনেটে ট্র্যাফিক বন্ধ (পরীক্ষামূলক)	অন্য AZ-তে ট্র্যাফিক চলে
প্রাইভেট EC2 থেকে `curl`	ইন্টারনেট যায় (NAT); বাইরে থেকে SSH ব্যর্থ হয়
ডেভেলপার ব্যবহারকারী দিয়ে EC2 Terminate চেষ্টা	AccessDenied — Least-Privilege প্রমাণ
CloudWatch অ্যালার্ম পরীক্ষা	CPU চাপে IN_ALARM ও ইমেইল আসে

৫. নিরাপত্তা পর্যালোচনা

- কোনো Security Group-এ `0.0.0.0/0` থেকে SSH বা ডাটাবেস পোর্ট খোলা নেই।
- Root-এ MFA আছে; কোনো Root Access Key নেই।
- সব EBS ভলিউম এনক্রিপ্টেড; S3 বাকেট (যদি থাকে) পাবলিক নয়।
- VPC Flow Logs চালু; CloudTrail চালু।
- Security Hub-এর Critical/High Finding শূন্য — অথবা প্রতিটির জন্য লিখিত ব্যতিক্রম (exception) ও কারণ আছে।

৬. খরচ পর্যালোচনা

AWS Pricing Calculator-এ মাসিক হিসাব বের করুন — NAT Gateway, ALB, EC2 (২টি), EBS ও ডেটা ট্রান্সফার আলাদা আলাদা লিখুন। "AWS Pricing Calculator-এ যাচাই করুন" — এই বইয়ে নির্দিষ্ট দাম দেওয়া হয়নি। খরচ কমানোর তিনটি উপায় লিখুন: (১) ডেভ পরিবেশে রাতে ASG Desired 0 করা, (২) একটি NAT ব্যবহার করা (ডেভে), (৩) Savings Plans / Spot (প্রোডাকশনে বিবেচনা)।

৭. চূড়ান্ত ডেলিভারেবল

- Terraform কোড (GitHub রিপোজিটরি) — `README`-সহ, চালানোর নির্দেশনা।
- আর্কিটেকচার ডায়াগ্রাম (PNG/PDF) + ADR ডকুমেন্ট।
- নিরাপত্তা চেকলিস্ট (সবুজ/লাল অবস্থাসহ)।
- খরচের হিসাব (Pricing Calculator এক্সপোর্ট)।
- Runbook: Failover, স্কেলিং ও ইনস্ট্যান্স প্রতিস্থাপনের ধাপ।
- যাচাইয়ের প্রমাণ: স্ক্রিনশট / CLI আউটপুট (৬টি পরীক্ষা)।

> **সমাপ্তির মানদণ্ড:** ALB দিয়ে সাইট চলে, একটি AZ ডাউন হলেও চলে, ডেভেলপার Least-Privilege প্রমাণিত, Terraform দিয়ে পুরো ভিত্তি আবার বানানো যায় (`destroy → apply`), এবং সব ডকুমেন্ট জমা দেওয়া হয়েছে।

প্রজেক্ট ০২ • সম্পূর্ণ MERN অ্যাপ • আনুমানিক ৩–৪ সপ্তাহ • কস্ট রিস্ক: High — RDS + ALB + ECS একসঙ্গে চলে

Project 02 — Production MERN Application on AWS

Project 01-এর ভিত্তির উপরে এবার আসল অ্যাপ — TechNova-র MERN প্ল্যাটফর্ম: React/Next.js Frontend, Node.js/Express Backend, প্রোডাকশন ডাটাবেস, S3 + CloudFront, Route 53 + ACM + HTTPS, CI/CD, Secrets Management, মনিটরিং ও লগিং। Lab 24-এর সম্প্রসারিত, প্রোডাকশন-মানের সংস্করণ।

১. প্রয়োজনীয়তা (Requirements)

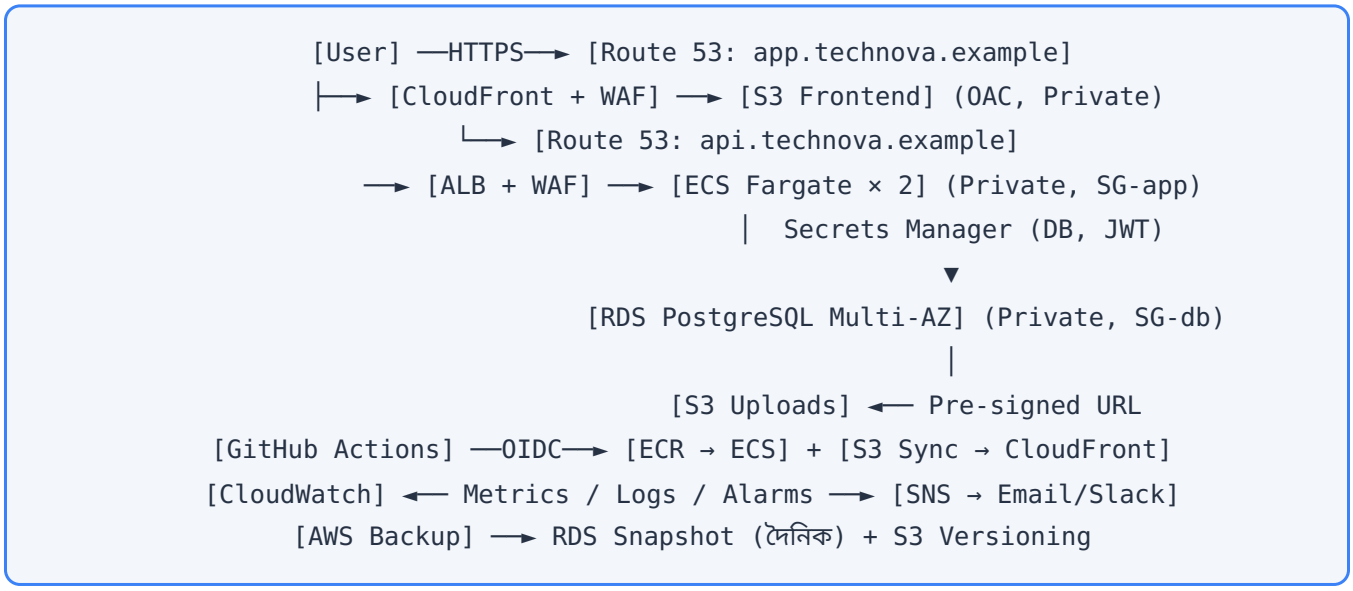
কার্যকরী প্রয়োজনীয়তা

- Frontend: React (বা Next.js) — Build হয়ে S3-তে, CloudFront দিয়ে পরিবেশিত, Route 53 + ACM HTTPS।
- Backend: Node.js/Express API — ECS Fargate-এ (Desired 2, দুটি AZ), ALB-র পেছনে, `/health` ও `/api/*` এন্ডপয়েন্ট।
- ডাটাবেস: RDS PostgreSQL (Multi-AZ — প্রোডাকশন মান; ডেভে Single-AZ) — অথবা MongoDB Atlas; সংযোগ Secrets Manager থেকে।
- ফাইল আপলোড: S3 (প্রি-সাইন্ড URL), CloudFront দিয়ে পরিবেশন।
- CI/CD: GitHub Actions — Push → Test → Build → ECR → ECS Deploy; Frontend → S3 Sync + CloudFront Invalidation।
- Secrets Management: সব পাসওয়ার্ড/API Key Secrets Manager-এ; কোডে কোনো Secret নেই।
- মনিটরিং ও লগিং: CloudWatch Dashboard, অ্যালার্ম (5xx, Latency, CPU, DB Storage), অ্যাপ লগ CloudWatch Logs-এ (JSON ফরম্যাট)।

অ-কার্যকরী প্রয়োজনীয়তা

- নিরাপত্তা: HTTPS সর্বত্র, WAF ALB-র সামনে, CORS শুধু নিজের ডোমেইনে, Rate Limiting, Security Headers।
- উপলব্ধতা: Multi-AZ; ডেপ্লয়মেন্টে ডাউনটাইম শূন্য (Rolling Deployment)।
- পর্যবেক্ষণযোগ্যতা: প্রতিটি রিকোয়েস্টে Correlation ID; এরর লগে Stack Trace।
- খরচ: মাসিক বাজেট $50 (উদাহরণ — নিজের বাজেট ঠিক করুন) ও অ্যালার্ট 80%।

২. আর্কিটেকচার



৩. বাস্তবায়নের ধাপ

পর্যায় ১ — পরিকল্পনা (৩ দিন)

API চুক্তি (এন্ডপয়েন্ট তালিকা), ডাটা মডেল (টেবিল/কালেকশন), পরিবেশ কৌশল (dev/staging/prod), খরচের হিসাব। ADR লিখুন: কেন ECS Fargate (EKS নয়), কেন RDS PostgreSQL (DynamoDB নয় — এই অ্যাপের জন্য)।

পর্যায় ২ — Backend ও ডাটাবেস (৫ দিন)

Express API + PostgreSQL (Lab 13, 24)। মাইগ্রেশন স্ক্রিপ্ট, Seed ডেটা, `/health` ও `/api/users` CRUD। Secrets Manager সংযোগ। লোকালে Docker-এ পরীক্ষা।

পর্যায় ৩ — Frontend (৪ দিন)

React অ্যাপ — API ক্লায়েন্ট (Correlation ID হেডারসহ), লগইন পাতা, ব্যবহারকারী তালিকা। Build → S3 → CloudFront (Lab 12)। Custom Error Response (SPA রাউটিং)।

পর্যায় ৪ — CI/CD (৩ দিন)

Lab 20-এর Workflow সম্পূর্ণ করুন — Task Definition রেন্ডার করে নতুন Revision ডেপ্লয় (সঠিক পদ্ধতি), Frontend Job আলাদা। Staging পরিবেশে প্রথমে ডেপ্লয়, যাচাই, তারপর Production।

পর্যায় ৫ — নিরাপত্তা ও মনিটরিং (৩ দিন)

WAF Rules (SQLi, XSS, Rate Limit), Security Headers (Helmet), CORS, CloudWatch Dashboard + অ্যালার্ম, Log ফরম্যাট JSON করা। Security Hub-এর Findings ঠিক করা।

পর্যায় ৬ — লোড পরীক্ষা ও ডকুমেন্টেশন (২ দিন)

সাধারণ লোড পরীক্ষা (যেমন `ab` বা `k6` দিয়ে 100 concurrent ব্যবহারকারী) — কোথায় ভাঙে দেখুন। Runbook, আর্কিটেকচার ডায়াগ্রাম, খরচের প্রতিবেদন ও ডেপ্লয়মেন্ট গাইড লিখুন।

৪. যাচাই (Verification)

পরীক্ষা	প্রত্যাশিত ফল
Frontend HTTPS-এ খোলে	React অ্যাপ লোড হয়, Console-এ কোনো এরর নেই
API CRUD (Postman/curl)	ব্যবহারকারী তৈরি, পড়া, আপডেট, মোছা — সব 2xx
একটি ECS Task বন্ধ	অন্যটি ট্র্যাফিক নেয়; নতুন Task আসে; ডাউনটাইম নেই
GitHub Push	Staging-এ ডেপ্লয় হয়; যাচাইয়ের পর Prod-এ যায়
ডাটাবেস Failover (Multi-AZ)	স্বয়ংক্রিয়ভাবে স্ট্যান্ডবাইয়ে যায়; অ্যাপ পুনরায় সংযোগ করে
ভুল পাসওয়ার্ড দিয়ে API	401; লগে প্রচেষ্টা রেকর্ড হয়
লোড পরীক্ষা	100 concurrent-এ p95 Latency < 500ms (নিজের লক্ষ্য ঠিক করুন)

৫. নিরাপত্তা পর্যালোচনা

- WAF চালু — SQLi/XSS Rules, Rate Limit 100 req/min/IP (নিজের সীমা ঠিক করুন)।
- JWT Secret Secrets Manager-এ; টোকেনের মেয়াদ ১ ঘণ্টা; Refresh Token আলাদা।
- কোডে কোনো Secret নেই — `git log --all -p | grep -i "password\|secret\|key"` যাচাই করুন।
- S3 বাকেট পাবলিক নয়; CloudFront OAC; আপলোডে Content-Type যাচাই।
- Security Hub Critical/High শূন্য; GuardDuty চালু।

৬. খরচ পর্যালোচনা

মাসিক খরচের ভাঙন (Pricing Calculator): RDS, ECS Fargate, ALB, NAT, CloudFront, S3, Route 53 — আলাদা আলাদা। বাজেট $50-এর বিপরীতে আসল খরচ — বেশি হলে কোন সেবা কমানো যায় লিখুন (যেমন: ডেভে RDS Single-AZ, Fargate CPU কমানো, CloudFront ক্যাশ বাড়ানো)।

৭. চূড়ান্ত ডেলিভারেবল

- চলমান অ্যাপের URL (Frontend + API) — HTTPS-সহ।
- GitHub রিপোজিটরি: Frontend, Backend, Terraform/IaC, CI/CD Workflow।
- আর্কিটেকচার ডায়াগ্রাম + ADR (কেন এই সিদ্ধান্তগুলো)।
- API ডকুমেন্টেশন (এন্ডপয়েন্ট, উদাহরণ রিকোয়েস্ট/রেসপন্স)।
- Runbook: ডেপ্লয়, রোলব্যাক, ডাটাবেস Failover, ঘটনা (Incident) প্রতিক্রিয়া।
- নিরাপত্তা ও খরচ প্রতিবেদন (Lab 23-এর ছক)।
- লোড পরীক্ষার ফলাফল ও সীমা।

> **সমাপ্তির মানদণ্ড:** HTTPS-এ চলমান MERN অ্যাপ, Push করলেই ডেপ্লয়, একটি Task/একটি AZ ডাউন হলেও চলে, কোনো Secret কোডে নেই, এবং সব ডকুমেন্ট জমা দেওয়া হয়েছে।

প্রজেক্ট ০৩ • সম্পূর্ণ ক্লাউড প্ল্যাটফর্ম • আনুমানিক ৪–৬ সপ্তাহ • কস্ট রিস্ক: High — মাল্টি-অ্যাকাউন্ট ডিজাইন; ল্যাবে এক অ্যাকাউন্টে অনুকরণ করুন

Project 03 — TechNova AWS Cloud Platform

TechNova এখন একটি কোম্পানি — একাধিক টিম, একাধিক পরিবেশ (dev/staging/prod), একাধিক অ্যাপ। এই প্রজেক্টে আপনি একজন ক্লাউড আর্কিটেক্টের মতো পুরো প্ল্যাটফর্মের নকশা করবেন: মাল্টি-অ্যাকাউন্ট গভর্ন্যান্স, নেটওয়ার্ক ভিত্তি, IAM ও নিরাপত্তা, কম্পিউট ও কন্টেইনার, স্টোরেজ ও ডাটাবেস, পর্যবেক্ষণ, ব্যাকআপ ও DR, খরচ অপ্টিমাইজেশন, Runbook ও আর্কিটেকচার ডকুমেন্টেশন। এটি Project 01 + 02-এর উপরে দাঁড়িয়ে থাকা প্ল্যাটফর্ম-স্তরের কাজ।

১. প্রয়োজনীয়তা (Requirements)

গভর্ন্যান্স

- মাল্টি-অ্যাকাউন্ট কাঠামো: Management অ্যাকাউন্ট + Dev / Staging / Prod অ্যাকাউন্ট + Shared Services (লগ, ব্যাকআপ) + Security (Security Hub, GuardDuty কেন্দ্রীয়)।
- AWS Organizations + Service Control Policies (SCP): যেমন Prod অ্যাকাউন্টে `ec2:TerminateInstances` শুধু নির্দিষ্ট রোল, কোনো অ্যাকাউন্টে Root Access Key নয়।
- ট্যাগিং নীতি: প্রতিটি রিসোর্সে `Project`, `Env`, `Owner`, `CostCenter` — ট্যাগ ছাড়া রিসোর্স বানানো যাবে না (Config Rule)।

নেটওয়ার্ক ভিত্তি

- প্রতি পরিবেশে আলাদা VPC (CIDR ওভারল্যাপ নয় — যেমন Dev `10.0.0.0/16`, Staging `10.1.0.0/16`, Prod `10.2.0.0/16`)।
- Shared Services VPC-র সঙ্গে VPC Peering / Transit Gateway (ধারণা + ডিজাইন; ল্যাবে Peering অনুকরণ)।
- প্রতি AZ-তে NAT Gateway (Prod), Centralized Egress-এর ধারণা।

IAM ও নিরাপত্তা

- SSO (IAM Identity Center): টিমের সবাই SSO দিয়ে লগইন; দীর্ঘমেয়াদি Access Key নয়।
- ভূমিকা-ভিত্তিক অ্যাক্সেস: Developer (Dev-এ Deploy), DevOps (Staging), Release Manager (Prod Deploy — Approval-সহ)।
- কেন্দ্রীয় লগ: সব অ্যাকাউন্টের CloudTrail → Shared Services S3 (Vault Lock)।
- GuardDuty + Security Hub + Config — সব অ্যাকাউন্টে চালু, কেন্দ্রীয় Dashboard।

কম্পিউট ও কন্টেইনার

- ECS Fargate (স্ট্যান্ডার্ড অ্যাপ) + EKS (বড়/জটিল সেবার জন্য) — কখন কোনটি, তার সিদ্ধান্ত-ছক।
- Lambda (ইভেন্ট-ভিত্তিক কাজ: ছবি প্রসেসিং, নোটিফিকেশন)।
- CI/CD: GitHub Actions (OIDC) → Dev (স্বয়ংক্রিয়) → Staging (স্বয়ংক্রিয় + পরীক্ষা) → Prod (Manual Approval)।

স্টোরেজ ও ডাটাবেস

- S3: প্রতি পরিবেশে আলাদা বাকেট; Versioning + Lifecycle + Replication (Prod → DR রিজিয়ন)।
- RDS: Prod-এ Multi-AZ + Read Replica; ব্যাকআপ ও PITR।
- DynamoDB: সেশন/ক্যাশ-জাতীয় ডেটা; PITR চালু।

পর্যবেক্ষণ, ব্যাকআপ ও খরচ

- কেন্দ্রীয় CloudWatch Dashboard + অ্যালার্ম + On-call Runbook।
- AWS Backup Plan: সব পরিবেশে দৈনিক ব্যাকআপ; মাসে একবার Restore Drill।
- DR: RPO/RTO প্রতি সেবায় লেখা; Prod-এর জন্য Cross-Region কপি।
- খরচ: প্রতি টিমের বাজেট, Cost Explorer-এ ট্যাগ-ভিত্তিক প্রতিবেদন, মাসিক খরচ পর্যালোচনা সভা।

২. আর্কিটেকচার

```
                    [AWS Organizations]
          ├── [Management Account] — Billing, SCP, SSO
   ├── [Security Account] — Security Hub, GuardDuty (কেন্দ্রীয়), CloudTrail S3
      ├── [Shared Services] — ECR, Terraform State, CI/CD, DNS
       ├── [Dev Account] — VPC 10.0.0.0/16, ECS, RDS Single-AZ
     ├── [Staging Account] — VPC 10.1.0.0/16, ECS, RDS Single-AZ
     └── [Prod Account] — VPC 10.2.0.0/16, ECS + EKS, RDS Multi-AZ
        [Transit Gateway] — অ্যাকাউন্টগুলোর VPC সংযুক্ত (প্রয়োজনে)
    [GitHub Actions OIDC] ──► Dev → Staging → Prod (Approval)
         [CloudWatch + SNS] ──► On-call (Runbook অনুযায়ী)
```

৩. বাস্তবায়নের ধাপ

পর্যায় ১ — গভর্ন্যান্স ডিজাইন (১ সপ্তাহ)

অ্যাকাউন্ট কাঠামোর ডায়াগ্রাম, SCP-র তালিকা, ট্যাগিং নীতি, SSO ভূমিকা-ছক। ল্যাবে একটিমাত্র অ্যাকাউন্ট ব্যবহার করে ডিজাইনটি লিখুন — মাল্টি-অ্যাকাউন্ট বানানো এই প্রজেক্টের লক্ষ্য নয়, নকশা করাই লক্ষ্য।

পর্যায় ২ — Shared Services (১ সপ্তাহ)

ECR, Terraform State বাকেট, কেন্দ্রীয় লগ বাকেট, DNS Hosted Zone — একটি Shared VPC-তে বানান।

পর্যায় ৩ — পরিবেশ তৈরি (১ সপ্তাহ)

Terraform Workspace বা আলাদা ভেরিয়েবল ফাইল দিয়ে Dev ও Staging VPC + ECS + RDS বানান। Prod-এর জন্য `plan` তৈরি করুন — Apply করবেন শুধু যাচাইয়ের পর।

পর্যায় ৪ — নিরাপত্তা ও পর্যবেক্ষণ (১ সপ্তাহ)

Security Hub, GuardDuty, Config Rules, কেন্দ্রীয় Dashboard, অ্যালার্ম, Backup Plan — সব চালু করুন।

পর্যায় ৫ — CI/CD ও DR পরীক্ষা (১ সপ্তাহ)

তিন ধাপের পাইপলাইন (Dev → Staging → Prod Approval) বানান। DR Drill: Staging-এ ব্যাকআপ থেকে Restore করে সময় মাপুন — RTO-র সঙ্গে মিলিয়ে দেখুন।

পর্যায় ৬ — ডকুমেন্টেশন ও হস্তান্তর (৩ দিন)

আর্কিটেকচার ডকুমেন্ট, Runbook (প্রতিটি সেবার জন্য), Onboarding গাইড (নতুন ডেভেলপার প্রথম দিনে কী করবে), খরচের প্রতিবেদন।

৪. যাচাই (Verification)

পরীক্ষা	প্রত্যাশিত ফল
Dev-এ Push	Dev-এ স্বয়ংক্রিয় ডেপ্লয়; Prod-এ কিছু হয় না
Prod Deploy-এ Approval ছাড়া	পাইপলাইন থেমে থাকে — Approval ছাড়া যায় না
ট্যাগ ছাড়া রিসোর্স	Config Rule Non-compliant দেখায়
কেন্দ্রীয় লগ	সব অ্যাকাউন্টের CloudTrail এক জায়গায় দেখা যায়
Restore Drill	RTO-র ভেতরে সেবা ফিরে আসে; সময় রেকর্ড করা আছে
খরচ প্রতিবেদন	প্রতি পরিবেশ/টিমের খরচ আলাদা দেখা যায়

৫. নিরাপত্তা পর্যালোচনা

- কোনো অ্যাকাউন্টে Root Access Key নেই; সব লগইন SSO + MFA।
- SCP দিয়ে Prod-এ বিপজ্জনক কাজ (যেমন সব S3 বাকেট মোছা) আটকানো আছে।
- সব লগ Immutable (Vault Lock / Object Lock)।
- নিয়মিত Access Review: ৯০ দিন ব্যবহার না হওয়া Access Key/Role মুছে ফেলা।

৬. খরচ পর্যালোচনা

প্রতি পরিবেশের মাসিক খরচের ভাঙন ও বাজেট। খরচ অপ্টিমাইজেশনের পরিকল্পনা: Dev-এ রাতে রিসোর্স বন্ধ (Scheduler), Prod-এ Savings Plans, S3 Lifecycle, পুরোনো স্ন্যাপশট পরিষ্কার। "AWS Pricing Calculator-এ যাচাই করুন" — নির্দিষ্ট দাম এই বইয়ে দেওয়া হয়নি।

৭. চূড়ান্ত ডেলিভারেবল

- প্ল্যাটফর্ম আর্কিটেকচার ডকুমেন্ট (অ্যাকাউন্ট, নেটওয়ার্ক, IAM, নিরাপত্তা, খরচ — সব)।
- Terraform কোড (মডিউল-ভিত্তিক: VPC, ECS, RDS — পুনঃব্যবহারযোগ্য)।
- CI/CD পাইপলাইন (তিন পরিবেশ, Approval-সহ)।
- Runbook সংগ্রহ (অন্তত ৮টি: Deploy, Rollback, Failover, Restore, Incident, Access, Cost Alert, Security Finding)।

- DR পরীক্ষার প্রতিবেদন (RPO/RTO বনাম আসল সময়)।
- Onboarding গাইড (নতুন টিম সদস্যের প্রথম দিনের চেকলিস্ট)।

> **সমাপ্তির মানদণ্ড:** মাল্টি-পরিবেশ ডিজাইন ডকুমেন্ট সম্পূর্ণ, Dev/Staging চলমান, Prod-এর plan প্রস্তুত, নিরাপত্তা ও খরচ পর্যালোচনা সম্পন্ন, এবং Runbook দিয়ে নতুন কেউ সিস্টেম চালাতে পারে।

ক্যাপস্টোন • স্বাধীন প্রজেক্ট • আনুমানিক ৪–৬ সপ্তাহ • কস্ট রিস্ক: High — পরিকল্পনা করে বাজেট সেট করুন, প্রতিদিন Cleanup যাচাই করুন

Capstone — TechNova: Production-Ready AWS Cloud Platform

এই ক্যাপস্টোনে আপনি একা — একজন ক্লাউড ইঞ্জিনিয়ারের মতো — TechNova-র জন্য একটি সম্পূর্ণ, প্রোডাকশন-স্টাইলের MERN প্ল্যাটফর্ম ডিজাইন, ডেপ্লয়, সুরক্ষা, পর্যবেক্ষণ, সমস্যা সমাধান ও ডকুমেন্ট করবেন। আগের ২৪টি ল্যাব ও ৩টি প্রজেক্টে যা শিখেছেন, তার সবকিছু এখানে একসঙ্গে ব্যবহার হবে। সাহায্য নেওয়া যাবে বই ও নিজের নোট থেকে — তবে সিদ্ধান্ত আপনার, বাস্তবায়ন আপনার, দায়িত্বও আপনার।

> **ক্যাপস্টোনের নিয়ম:** (১) শুরুতে বাজেট সেট করুন (যেমন $50) ও বিলিং অ্যালার্ট চালু করুন। (২) প্রতিদিন কাজ শেষে চলমান রিসোর্সের তালিকা যাচাই করুন। (৩) কোনো Secret কোডে বা Git-এ রাখবেন না। (৪) সমাধান গাইড (`#capstone-solution`) পড়ার আগে নিজে চেষ্টা করুন — আগে পড়লে শেখা হবে না।

১. পরিস্থিতি (Scenario)

TechNova একটি স্টার্টআপ — তাদের MERN অ্যাপ (`technova.example`) এখন একজন ডেভেলপারের ল্যাপটপে চলে। ব্যবহারকারী বাড়ছে, সাইট মাঝে মাঝে ডাউন হয়, ডাটাবেসের ব্যাকআপ নেই, পাসওয়ার্ড কোডে লেখা। আপনার কাজ: অ্যাপটিকে AWS-এ প্রোডাকশন-রেডি করা — যাতে তা নিরাপদ, স্কেলযোগ্য, পর্যবেক্ষণযোগ্য ও পুনরুদ্ধারযোগ্য হয়। আপনার কাছে আছে: MERN অ্যাপের কোড, একটি AWS অ্যাকাউন্ট, GitHub, এবং এই বইয়ের সব ল্যাব।

২. যা যা করতে হবে (Scope)

২.১ আর্কিটেকচার ডায়াগ্রাম

সম্পূর্ণ সিস্টেমের ডায়াগ্রাম আঁকুন: ব্যবহারকারী → Route 53 → CloudFront (Frontend) ও ALB (API) → ECS Fargate → RDS → S3। প্রতিটি সংযোগে প্রোটোকল ও পোর্ট লিখুন। ডায়াগ্রামে দেখান: কোনটি পাবলিক, কোনটি প্রাইভেট, কোন AZ-তে কী আছে, কোথায় WAF, কোথায় Secrets Manager।

২.২ প্রয়োজনীয়তা ডকুমেন্ট (Requirements Document)

কার্যকরী ও অ-কার্যকরী প্রয়োজনীয়তা লিখুন — ব্যবহারকারী, ট্র্যাফিক অনুমান (যেমন: ১,০০০ দৈনিক ব্যবহারকারী, সর্বোচ্চ ১০০ concurrent), উপলব্ধতার লক্ষ্য (যেমন 99.5%), RPO/RTO (যেমন RPO ১ ঘণ্টা, RTO ২ ঘণ্টা), বাজেট (মাসিক $50), নিরাপত্তার প্রয়োজনীয়তা (HTTPS, MFA, Least-Privilege)। অনুমানগুলো কেন করলেন — যুক্তি লিখুন।

২.৩ নেটওয়ার্ক ও IAM ডিজাইন

VPC CIDR পরিকল্পনা (সাবনেট টেবিলসহ), Security Group ম্যাট্রিক্স, NACL নিয়ম, IAM ভূমিকা ও পলিসি (অ্যাডমিন, ডেভেলপার, CI/CD, EC2/ECS Task Role) — প্রতিটি পলিসির জন্য Least-Privilege যুক্তি লিখুন।

২.৪ Infrastructure as Code

পুরো অবকাঠামো Terraform-এ লিখুন (মডিউল-ভিত্তিক হলে ভালো)। `terraform plan` ও `apply` দিয়ে বানান। State S3-তে রাখুন (Versioning + Encryption)। কোড GitHub-এ রাখুন — `README`-তে চালানোর নির্দেশনা।

২.৫ অ্যাপ্লিকেশন ডেপ্লয়মেন্ট

Frontend (React/Next.js) → S3 + CloudFront + HTTPS; Backend (Node/Express) → ECR + ECS Fargate + ALB; ডাটাবেস → RDS (Multi-AZ — ডেভে Single-AZ, তবে ডিজাইনে Multi-AZ)। ফাইল আপলোড S3-তে (প্রি-সাইন্ড URL)।

২.৬ ডাটাবেস ও স্টোরেজ

ডাটা মডেল, মাইগ্রেশন, Seed ডেটা, ব্যাকআপ পরিকল্পনা (অটোমেটেড ব্যাকআপ + স্ন্যাপশট), S3 Versioning ও Lifecycle। ব্যাকআপ থেকে Restore পরীক্ষা করে প্রমাণ জমা দিন।

২.৭ CI/CD

GitHub Actions (OIDC) — Push → Test → Build → Deploy। Staging ও Production — Production-এ Manual Approval। Rollback পদ্ধতি লিখুন ও পরীক্ষা করুন (আগের ইমেজ ট্যাগে ফিরে যাওয়া)।

২.৮ পর্যবেক্ষণ (Observability)

CloudWatch Dashboard (Frontend, API, DB — এক জায়গায়), অ্যালার্ম (5xx, Latency, CPU, DB Storage, Billing), লগ (JSON, Correlation ID), CloudTrail। একটি কৃত্রিম সমস্যা (যেমন একটি Task বন্ধ করা) তৈরি করে অ্যালার্ম আসে কি না প্রমাণ করুন।

২.৯ নিরাপত্তা মূল্যায়ন

নিজের সিস্টেমের নিরাপত্তা অডিট করুন (Lab 23-এর ছক): Security Hub Findings, Trusted Advisor, হাতে-কলমে চেকলিস্ট (MFA, SG, S3 পাবলিক, Secret, WAF, CloudTrail)। প্রতিটি Finding-এর জন্য: ঠিক করলেন, নাকি লিখিতভাবে গ্রহণ করলেন (risk acceptance) — কারণসহ।

২.১০ খরচের অনুমান

AWS Pricing Calculator-এ মাসিক হিসাব — সেবা অনুযায়ী ভাঙন, বাজেটের সঙ্গে তুলনা। খরচ কমানোর তিনটি উপায় ও সেগুলোর ঝুঁকি লিখুন। বিলিং অ্যালার্ট ও বাজেট সেট করুন।

২.১১ ব্যাকআপ ও পুনরুদ্ধার

ব্যাকআপ পরিকল্পনা (কী, কখন, কতদিন), Restore Drill (সময় মেপে), RPO/RTO-র সঙ্গে তুলনা। একটি DR Runbook লিখুন: "RDS মুছে গেলে কী করবেন" — ধাপে ধাপে।

২.১২ রোলব্যাক কৌশল

ডেপ্লয়মেন্ট ব্যর্থ হলে কী করবেন — ECS-এ আগের Task Definition-এ ফিরে যাওয়া, Frontend-এ আগের Build-এ ফিরে যাওয়া, ডাটাবেস মাইগ্রেশন ব্যর্থ হলে কী করবেন — তিনটিরই ধাপ লিখুন ও একটি পরীক্ষা করুন।

২.১৩ অপারেশনাল Runbook

কমপক্ষে ৮টি Runbook: (১) ডেপ্লয় করা, (২) রোলব্যাক করা, (৩) সাইট ডাউন হলে, (৪) ডাটাবেস ডাউন হলে, (৫) বেশি ট্র্যাফিক এলে, (৬) বিলিং অ্যালার্ট এলে, (৭) নিরাপত্তা সতর্কতা এলে, (৮) ব্যাকআপ থেকে ফেরানো। প্রতিটি Runbook: লক্ষণ → নির্ণয় → ধাপ → যাচাই → কখন সাহায্য চাইবেন।

২.১৪ চূড়ান্ত আর্কিটেকচার পর্যালোচনা

নিজের আর্কিটেকচারের সমালোচনা করুন: কোথায় Single Point of Failure আছে? ট্র্যাফিক দশ গুণ বাড়লে কী ভাঙবে? কোন সিদ্ধান্তটি পরে বদলাতে হতে পারে? — সৎ উত্তর লিখুন। নিখুঁত আর্কিটেকচার হয় না; সীমা জানা থাকাই পেশাদারিত্ব।

৩. সময়সূচি (সুপারিশ)

সপ্তাহ	কাজ	ডেলিভারেবল
১	প্রয়োজনীয়তা, আর্কিটেকচার ডায়াগ্রাম, নেটওয়ার্ক ও IAM ডিজাইন	Requirements Doc + ডায়াগ্রাম + ADR
২	Terraform দিয়ে ভিত্তি + ডাটাবেস + Backend ডেপ্লয়	চলমান API + Terraform কোড
৩	Frontend + HTTPS + CI/CD + মনিটরিং	চলমান অ্যাপ + পাইপলাইন + Dashboard
৪	নিরাপত্তা অডিট, ব্যাকআপ/DR পরীক্ষা, রোলব্যাক পরীক্ষা	অডিট প্রতিবেদন + Drill প্রমাণ
৫	Runbook, খরচের হিসাব, চূড়ান্ত পর্যালোচনা	Runbook + খরচ প্রতিবেদন
৬	পর্যালোচনা, সংশোধন, জমা, Cleanup	চূড়ান্ত জমা + Cleanup প্রমাণ

৪. পেশাদার গ্রেডিং রুব্রিক

মূল্যায়নের ক্ষেত্র	ওজন	Excellent (৯০–১০০%)	Good (৭০–৮৯%)	Needs Work (<৭০%)
আর্কিটেকচার ও ডিজাইন	১৫%	Multi-AZ, স্তরভিত্তিক, সিদ্ধান্তের যুক্তিসহ ADR	কাজ করে, তবে কিছু সিদ্ধান্তের যুক্তি নেই	এক AZ, বা ডায়াগ্রাম ছাড়া
Infrastructure as Code	১৫%	মডিউল-ভিত্তিক Terraform, State S3-তে, README-সহ	এক ফাইলে Terraform, কাজ করে	হাতে ক্লিক করে বানানো, কোড নেই
অ্যাপ্লিকেশন ডেপ্লয়মেন্ট	১৫%	Frontend + API + DB — সব HTTPS-এ চলে, CRUD কাজ করে	চলে, তবে কিছু এন্ডপয়েন্ট ভাঙা	অ্যাপ চলে না বা আংশিক
CI/CD ও রোলব্যাক	১০%	OIDC পাইপলাইন, Approval, রোলব্যাক পরীক্ষিত	পাইপলাইন চলে, রোলব্যাক পরীক্ষা হয়নি	হাতে ডেপ্লয়, পাইপলাইন নেই
নিরাপত্তা	১৫%	Security Hub Critical/High শূন্য, WAF, Secrets Manager, অডিট প্রতিবেদন	মৌলিক নিরাপত্তা ঠিক, কিছু Finding বাকি	Secret কোডে, বা S3 পাবলিক, বা SSH খোলা
পর্যবেক্ষণ ও লগিং	১০%	Dashboard + অ্যালার্ম পরীক্ষিত, Correlation ID, Runbook	Dashboard আছে, অ্যালার্ম পরীক্ষা হয়নি	মনিটরিং নেই
ব্যাকআপ ও DR	১০%	Restore Drill সময়সহ প্রমাণিত, RPO/RTO মিলেছে	ব্যাকআপ আছে, Restore পরীক্ষা হয়নি	ব্যাকআপ নেই
ডকুমেন্টেশন ও খরচ	১০%	সম্পূর্ণ ডকুমেন্ট, খরচের ভাঙন, সৎ সীমা-স্বীকার	ডকুমেন্ট আছে, কিছু অংশ অসম্পূর্ণ	ডকুমেন্ট নেই বা অগোছালো

৫. পরিমাপযোগ্য গ্রহণযোগ্যতার মানদণ্ড (Acceptance Criteria)

ক্যাপস্টোন "পাস" করতে হলে নিচের সবগুলো সত্য হতে হবে — একটি মিথ্যা হলেও পুনরায় করতে হবে:

1. Frontend URL HTTPS-এ খোলে; API থেকে ডেটা আসে; নতুন ব্যবহারকারী তৈরি করা যায় ও RDS-তে দেখা যায়।
2. একটি ECS Task বন্ধ করলে ৫ মিনিটের মধ্যে সেবা নিজে থেকে ফিরে আসে (ডাউনটাইম প্রমাণসহ রেকর্ড করুন)।
3. GitHub-এ Push করলে Staging-এ ডেপ্লয় হয়; Production-এ Approval ছাড়া ডেপ্লয় হয় না।
4. কোডে কোনো Secret নেই (`git log` যাচাইসহ)। সব Secret Secrets Manager-এ।
5. ডাটাবেস প্রাইভেট সাবনেটে; পাবলিক অ্যাক্সেস No; S3 বাকেট পাবলিক নয়।
6. ব্যাকআপ থেকে Restore পরীক্ষা সফল — সময় রেকর্ড করা আছে ও RTO-র ভেতরে।
7. রোলব্যাক পরীক্ষা সফল — আগের ভার্সনে ফিরে যাওয়া যায়।
8. CloudWatch-এ কমপক্ষে ৩টি অ্যালার্ম সেট ও অন্তত ১টি পরীক্ষামূলকভাবে ট্রিগার করা হয়েছে।
9. Security Hub-এ কোনো Critical Finding নেই; High Finding থাকলে লিখিত কারণ আছে।
10. সম্পূর্ণ ডকুমেন্টেশন জমা: আর্কিটেকচার, Requirements, Runbook (৮টি), খরচের হিসাব, চূড়ান্ত পর্যালোচনা।
11. সব রিসোর্সের Cleanup সম্পন্ন ও Billing Dashboard-এ চলমান চার্জ শূন্যের প্রমাণ জমা দেওয়া হয়েছে।

৬. জমা দেওয়ার চেকলিস্ট

- ☐ GitHub রিপোজিটরি (কোড + Terraform + Workflow + README)
- ☐ চলমান অ্যাপের URL (জমা দেওয়ার সময় চালু থাকতে হবে)
- ☐ আর্কিটেকচার ডায়াগ্রাম (PDF/PNG)
- ☐ Requirements Document
- ☐ নিরাপত্তা অডিট প্রতিবেদন
- ☐ খরচের হিসাব (Pricing Calculator এক্সপোর্ট)
- ☐ ব্যাকআপ/DR Drill প্রমাণ (সময়সহ)
- ☐ রোলব্যাক পরীক্ষার প্রমাণ
- ☐ Runbook সংগ্রহ (৮টি)
- ☐ চূড়ান্ত আর্কিটেকচার পর্যালোচনা
- ☐ Cleanup প্রমাণ (খালি Console + Billing স্ক্রিনশট)

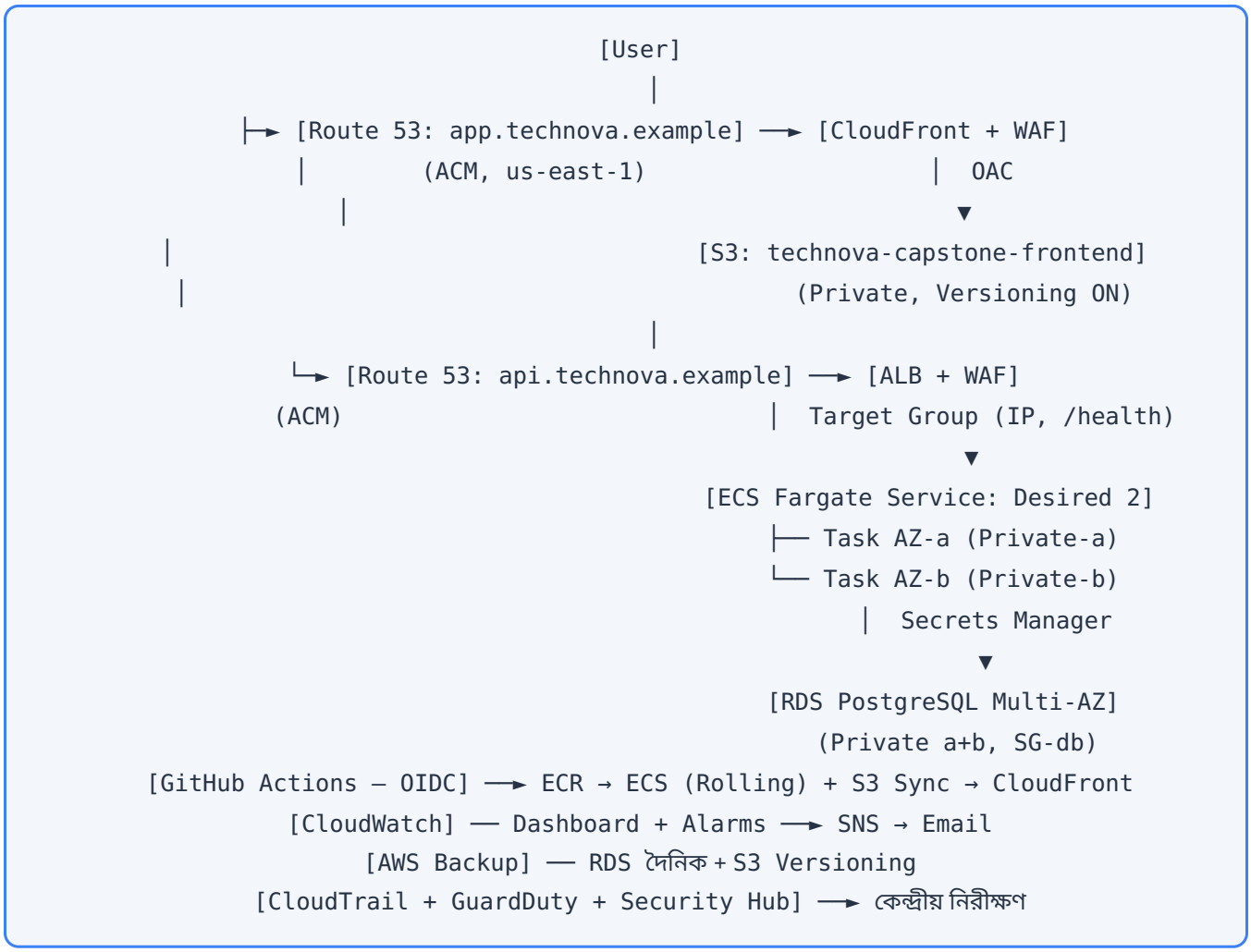
> **শেষ পরামর্শ:** ক্যাপস্টোন শেষে আপনার হাতে যা থাকবে — চলমান অ্যাপ, Terraform কোড, CI/CD পাইপলাইন, Runbook ও অডিট প্রতিবেদন — তা একটি চাকরির ইন্টারভিউতে দেখানোর মতো পোর্টফোলিও। GitHub রিপোজিটরির README-তে অ্যাপের URL, আর্কিটেকচার ডায়াগ্রাম ও "কী শিখলাম" — তিনটি রাখুন।

ক্যাপস্টোন সমাধান নির্দেশিকা • আগে নিজে চেষ্টা করুন, তারপর পড়ুন

Capstone Solution Guide — Reference Architecture and Key Decisions

> **আগে চেষ্টা করুন, তারপর পড়ুন।** এই অংশে ক্যাপস্টোনের রেফারেন্স আর্কিটেকচার ও মূল সিদ্ধান্তগুলো দেওয়া আছে — সম্পূর্ণ কোড নয়। নিজে ডিজাইন না করে এখান থেকে কপি করলে শেখা হবে না। আটকে গেলে একটি সিদ্ধান্ত পড়ুন, তারপর আবার নিজে চেষ্টা করুন।

১. রেফারেন্স আর্কিটেকচার (সংক্ষেপ)



২. মূল সিদ্ধান্ত ও যুক্তি

সিদ্ধান্ত	বেছে নেওয়া হয়েছে	কেন (যুক্তি)	বিকল্প ও তার অসুবিধা
কম্পিউট	ECS Fargate	সার্ভার পরিচালনা নেই; ছোট টিমের জন্য সহজ; ALB-র সঙ্গে ভালো সংযোগ	EKS — শক্তিশালী, তবে পরিচালনার ভার বেশি; EC2-তে ECS — সস্তা হতে পারে, তবে প্যাচিং নিজের দায়িত্ব

ডাটাবেস	RDS PostgreSQL Multi-AZ	পরিচিত SQL, অটোমেটেড ব্যাকআপ, Failover স্বয়ংক্রিয়	MongoDB Atlas — MERN-এর আসল M, তবে আলাদা সেবা ও খরচ; DynamoDB — স্কেল ভালো, তবে রিলেশনাল কুয়েরি কঠিন
Frontend হোস্টিং	S3 + CloudFront + OAC	সস্তা, দ্রুত, S3 পাবলিক নয়, HTTPS সহজ	EC2-তে Nginx — সার্ভার পরিচালনা লাগে; Amplify — সহজ, তবে নিয়ন্ত্রণ কম
IaC	Terraform (মডিউল-ভিত্তিক)	ক্লাউড-নিরপেক্ষ ধারণা, মডিউল পুনঃব্যবহার, State স্পষ্ট	CloudFormation — AWS-নেটিভ, তবে শুধু AWS-এ চলে; CDK — কোডে লেখা যায়, তবে শেখার বক্ররেখা বেশি
CI/CD	GitHub Actions + OIDC	কোডের পাশেই Workflow, Secret Key নেই, ফ্রি টিয়ার যথেষ্ট	Jenkins — নিজে হোস্ট করতে হয়; CodePipeline — AWS-নেটিভ, তবে GitHub-এর সঙ্গে OIDC সেটআপ বেশি কাজ
Secret	Secrets Manager	স্বয়ংক্রিয় Rotation, ECS-র সঙ্গে সরাসরি সংযোগ, অডিট হয়	Parameter Store — সস্তা, তবে Rotation নিজে করতে হয়; পরিবেশ ভেরিয়েবলে প্লেইন টেক্সট — কখনোই নয়
মনিটরিং	CloudWatch (Metrics + Logs + Alarms)	AWS-নেটিভ, সব সেবার মেট্রিক এক জায়গায়, অ্যালার্ম সহজ	Grafana/Prometheus — সুন্দর, তবে আলাদা হোস্টিং লাগে

৩. নেটওয়ার্ক পরিকল্পনা (রেফারেন্স)

রিসোর্স	CIDR / মান
VPC	`10.0.0.0/16`
Public-a / Public-b	`10.0.1.0/24` / `10.0.2.0/24`
Private-a / Private-b	`10.0.11.0/24` / `10.0.12.0/24`
SG-alb	80/443 — `0.0.0.0/0`
SG-app	3000 — শুধু SG-alb থেকে
SG-db	5432 — শুধু SG-app থেকে
NAT	প্রতি AZ-তে একটি (Prod); ডেভে একটি

৪. CI/CD প্রবাহ (রেফারেন্স)

```
Push main ──► [Test: npm test + lint]
                  │ ব্যর্থ হলে থেমে যায়
                  ▼
        [Build Docker → ECR :<sha>]
                  │
                  ▼
   [Deploy Staging: ECS update-service]
      │  Smoke Test: /health + /api/users
                  ▼
   [Manual Approval — GitHub Environments]
                  │
                  ▼
[Deploy Prod: ECS update-service (নতুন Task Definition Revision)]
      │  CloudWatch Alarm দেখা (৫ মিনিট)
                  ▼
[Frontend: npm run build → S3 Sync → CloudFront Invalidation]
     Rollback: আগের Task Definition Revision-এ ফিরে যাওয়া
(aws ecs update-service --task-definition technova-api:<prev>)
```

৫. নিরাপত্তা চেকলিস্ট (রেফারেন্স)

- Root MFA ON, Root Access Key নেই।
- সব লগইন SSO / IAM + MFA; দীর্ঘমেয়াদি Key নেই (OIDC)।
- DB প্রাইভেট, এনক্রিপ্টেড, ব্যাকআপ 7 দিন, Multi-AZ।
- S3 পাবলিক নয়; Versioning ON; Lifecycle ON।
- WAF ALB ও CloudFront-এর সামনে; Rate Limit সেট।
- CloudTrail ON (সব রিজিয়ন); GuardDuty ON; Security Hub ON।
- Secrets Manager-এ সব Secret; Rotation চালু (DB পাসওয়ার্ড)।
- Security Hub Critical/High শূন্য (বা লিখিত ব্যতিক্রম)।

৬. খরচের কাঠামো (রেফারেন্স — দাম নয়, ভাঙন)

মাসিক খরচের ভাঙন এইভাবে লিখুন (দাম AWS Pricing Calculator-এ যাচাই করুন): RDS (সবচেয়ে বড় অংশ — Multi-AZ হলে দ্বিগুণ), ECS Fargate (vCPU + Memory × সময়), ALB (ঘণ্টা + LCU), NAT Gateway (ঘণ্টা + ডেটা), CloudFront (ডেটা ট্রান্সফার), S3 (স্টোরেজ + রিকোয়েস্ট), Route 53 (Hosted Zone + কুয়েরি), CloudWatch Logs (স্টোরেজ)। খরচ কমানোর প্রথম তিনটি লিভার: (১) ডেভ/Staging-এ রিসোর্স বন্ধ রাখা, (২) Fargate CPU/Memory সঠিক মাপা, (৩) S3 Lifecycle ও পুরোনো স্ন্যাপশট পরিষ্কার।

৭. সাধারণ ভুল (আগে থেকে জেনে রাখুন)

- **CloudFront সার্টিফিকেট ভুল রিজিয়নে:** ACM সার্টিফিকেট `us-east-1`-এ নিতে হয় — `ap-south-1`-এ নিলে CloudFront-এ দেখা যায় না।
- **Task Definition-এ `:latest` ট্যাগ:** রোলব্যাক করা যায় না — কোন ভার্সন চলছে বোঝা যায় না। সবসময় Commit SHA ট্যাগ ব্যবহার করুন।
- **CORS ভুলে যাওয়া:** Frontend থেকে API কল ব্যর্থ হয় — ব্রাউজার Console-এ CORS এরর দেখুন, নেটওয়ার্ক ট্যাবে নয়।
- **Health Check পাথ ভুল:** `/health` না থাকলে Target unhealthy হয় — ALB 502 দেয়।
- **NAT Gateway ভুলে রেখে যাওয়া:** সবচেয়ে সাধারণ বিল-বিপর্যয় — Cleanup চেকলিস্টে NAT সবার উপরে রাখুন।
- **RDS পাবলিক করা:** "সাময়িক পরীক্ষার জন্য" পাবলিক করলে বট কয়েক মিনিটে খুঁজে পায় — কখনোই নয়।
- **Secret Git-এ:** একবার Push করলে ইতিহাসেও থেকে যায় — `git log --all` যাচাই করুন, প্রয়োজনে Secret ঘোরান।

৮. Cleanup — ক্যাপস্টোন শেষে

ক্যাপস্টোন জমা দেওয়ার পর সব বিলযোগ্য রিসোর্স মুছে ফেলুন — Lab 24-এর Cleanup ক্রম অনুসরণ করুন: ECS → ALB → CloudFront → RDS → ECR → S3 → NAT/EIP → VPC। তারপর Billing Dashboard-এ যাচাই করুন — আগামী ২৪ ঘণ্টায় নতুন চার্জ আসছে কি না দেখুন। কোড, ডকুমেন্ট ও স্ক্রিনশট GitHub-এ রেখে দিন — সেগুলোই আপনার স্থায়ী প্রমাণ।

> **অভিনন্দন — আপনি এখন প্রোডাকশন-স্টাইলে AWS-এ একটি সম্পূর্ণ প্ল্যাটফর্ম বানাতে পারেন।** এই দক্ষতা শুধু সার্টিফিকেটের জন্য নয় — বাস্তব টিমে, বাস্তব ব্যবহারকারীর জন্য সিস্টেম চালানোর ভিত্তি। পরের ধাপ: নিজের প্রজেক্টে (Bookora / TeamFlow / The Grand Sovereign) এই প্যাটার্ন প্রয়োগ করুন, আর AWS Solutions Architect পরীক্ষার প্রস্তুতি নিন।